Universidad de Castilla-La Mancha

Escuela de Ingeniería Industrial y Aeroespacial

TRABAJO FIN DE GRADO
Nº 25-B-225150

Diseño e implementación de una base de datos y entorno gráfico para la gestión de datos en la simulación de transporte ferroviario

Sergio Berraco Arellano

JULIO DE 2025

Título: Diseño e implementación de una base de datos y entorno gráfico para la gestión de datos en la simulación de transporte ferroviario
Autor: Sergio Berraco Arellano
Grado: Grado en Ingeniería Electrónica Industrial y Automática
Tutores: Juan Moreno García
David Muñoz Valero

E-mail: sergio.berraco@alu.uclm.es
Sitio web: https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos
Dirección: UCLM — Escuela de Ingeniería Industrial y Aeroespacial
Campus Universitario de la Real Fábrica de Armas
45071 Toledo – Spain

© 2025 Sergio Berraco Arellano

Esta obra está bajo una Licencia Creative Commons Atribución 4.0 Internacional.

Muchos de los nombres usados por las compañías para diferenciar sus productos y servicios son reclamados como marcas registradas. Allí donde estos nombres aparezcan en este documento, y cuando el autor haya sido informado de esas marcas registradas, los nombres estarán escritos en mayúsculas o como nombres propios.

Agradecimientos

Quisiera expresar mi agradecimiento a mis tutores D. Juan y D. David por ayudarme y guiarme en el desarrollo de este Trabajo de Fin de Grado.

También, quisiera agradecer a mi familia y amigos por brindarme su apoyo siempre que lo he necesitado.

Resumen

Los grupos *ORETO* y *MAT* de la Escuela Superior de Informática de la Universidad de Castilla-La Mancha han desarrollado Rail mOBIlity simulatioN (ROBIN) [1], un simulador de movilidad ferroviaria microscópico que permite modelar el flujo de viajeros en corredores ferroviarios. Este simulador emplea varios ficheros como entrada de datos y, al acabar la simulación, genera unos archivos donde se guardan los resultados de cada simulación. El manejo de estos ficheros es tedioso en ocasiones debido a la cantidad de ficheros que se deben gestionar. Por esto, se ha decidido la utilización de una base de datos para estos propósitos.

El objetivo de este Trabajo Fin de Grado consiste en el diseño e implementación de un conjunto de bases de datos destinadas a almacenar los distintos archivos de entrada de datos para la oferta y la demanda del simulador de movilidad ferroviaria ROBIN, así como los archivos de resultados generados. Además, se propone desarrollar una interfaz gráfica que facilite la gestión de dichas bases de datos, permitiendo la importación, regeneración y consulta de los archivos a través de sentencias Structured Query Languaje (SQL).

Para diseñar las bases de datos necesarias para la consecución de este TFG, primero se ha estudiado el funcionamiento y diseño de las bases de datos relacionales. A continuación, se han analizado los diferentes comandos empleados en el lenguaje SQL para crear las bases de datos con sus correspondientes tablas, editar estas tablas para añadir o eliminar información y ejecutar consultas que permitan visualizar los datos almacenados en ellas.

Una vez entendido el funcionamiento de las bases de datos relacionales, se ha pasado a estudiar los diferentes bloques que poseen los archivos Yaml y CSV para poder diseñar de las bases de datos necesarias que contengan toda la información que estos archivos tienen almacenada. En la fase de diseño se ha empleado la herramienta SQLiteStudio, que utiliza SQLite3 como motor para las bases de datos. SQLiteStudio permite generar bases de datos, con sus correspondientes tablas, e interactuar con ellas por medio de su interfaz o bien mediante sentencias SQL.

Después, se ha abordado la creación del diseño final de aplicación que manejará todo. Se ha diseñado una estructura modular, donde cada módulo tiene una función diferente, por ejemplo, hay un módulo encargado de leer y procesar la información de los archivos de entrada de datos para ROBIN, también hay otro módulo encargado de la gestión de las bases de datos y la ejecución de sentencias SQL en las bases de datos. Se ha comprobado el correcto funcionamiento de estos módulos.

Finalmente, se ha diseñado e implementado una interfaz que, utilizando los módulos que componen el programa, gestiona las bases de datos. Esta interfaz tiene la capacidad de importar los datos de los archivos a la base de datos, regenerar estos archivos de nuevo para su uso en el simulador y ejecutar sentencias SQL para consultar datos de las bases de datos. Estas consultas pueden almacenarse en el programa para su uso en ocasiones posteriores, también se pueden cargar archivos `.sql` que contengan la sentencia para ejecutarla con la aplicación o escribir las sentencias directamente en la entrada de texto que posee la aplicación.

Abstract

The *ORETO* and *MAT* research groups at the Higher School of Computer Science of the University of Castilla-La Mancha have developed Rail mOBIlity simulatioN (ROBIN) [1], a microscopic railway mobility simulator that allows modeling passenger flows in railway corridors. This simulator uses several files as input data and, upon completion of the simulation, generates files in which the results of each run are stored. Managing these files can be tedious at times due to the number of files that must be handled. Therefore, a database has been adopted for these purposes.

The objective of this Final Degree Project (TFG) is the design and implementation of a set of databases intended to store the various input data files for the supply and demand of ROBIN, as well as the generated result files. Additionally, a graphical user interface is proposed to facilitate the management of these databases, enabling the import, rebuilding, and querying of the files via Structured Query Languaje (SQL) statements.

To design the databases necessary for the completion of this TFG, the operation and design of relational databases have first been studied. Subsequently, the various commands employed in the Structured Query Languaje (SQL) language for creating databases with their corresponding tables, editing these tables to add or remove information, and performing queries to display the data stored within have been examined.

Once the operation of relational databases was understood, the different blocks present in Yaml and CSV files were analyzed to design the necessary databases that would contain all the information stored within these files. In the design phase, the SQLiteStudio tool was used, which employs SQLite3 as the database engine. SQLiteStudio allows creating databases with their corresponding tables and interacting with them either through its interface or via SQL statements.

Subsequently, the creation of the final application design that will manage everything has been undertaken. A modular structure has been designed, in which each module has a different function; for example, one module is responsible for reading and processing the input data files for ROBIN, and another module is responsible for database management and the execution of SQL statements. The correct operation of these modules has been verified.

Finally, an interface has been designed and implemented that, using the modules comprising the program, manages the databases. This interface is capable of importing data from the files into the database, reconstructing these files for use in the simulator, and executing SQL statements to query the databases. These queries can be stored within the application for later use; additionally, `.sql` files containing statements can be loaded for execution, or statements can be entered directly into the application's text input.

Índice general

1. Introducción **1**

1.1. Organización de la memoria 2

1.2. Repositorio de información 4

2. Objetivos **5**

3. Motivación y antecedentes **7**

4. Desarrollo del TFG **11**

4.1. Estudio del funcionamiento de bases de datos relacionales 11

4.2. Archivos de entrada y salida del simulador ROBIN 13

4.2.1. Estructura de los archivos de configuración de la oferta 14

4.2.1.1. Estaciones 16

4.2.1.2. Asientos 17

4.2.1.3. Corredores 18

4.2.1.4. Línea 20

4.2.1.5. Material rodante - `rollingStock` 21

4.2.1.6. Proveedores de Servicio 22

4.2.1.7. Franjas horarias - `timeSlot` 22

4.2.1.8. Servicio 23

4.2.2. Estructura de los archivos de configuración de la demanda 24

4.2.2.1. Mercado 25

4.2.2.2. Patrón de usuario 26

4.2.2.3. Patrón de demanda 30

4.2.2.4. Día 31

4.2.3. Estructura de los archivos de resultados 32

4.3. Diseño e implementación de las bases de datos 33

4.3.1. Base de datos para los archivos de entrada de datos de la oferta 37

4.3.1.1. Tabla principal 38

4.3.1.2. Tablas de datos 39

4.3.1.3. Tablas auxiliares 46

4.3.2. Base de datos para los archivos de entrada de datos de la demanda . . . 49

4.3.2.1. Tabla principal 50
4.3.2.2. Tablas de datos 51
4.3.2.3. Tablas auxiliares 54
4.3.3. Base de datos para la configuración de los resultados 56
4.3.3.1. Tabla principal 56
4.3.3.2. Tablas de datos 57
4.3.3.3. Tablas auxiliares 58
4.4. Desarrollo del software 58
4.4.1. Estructura funcional del software 58
4.4.1.1. Módulo de lectura de Yaml 60
4.4.1.2. Módulo de escritura de Yaml 75
4.4.1.3. Módulo de manejo de CSV 75
4.4.1.4. Módulo de manejo de bases de datos 81
4.4.1.5. Módulo de introducción de datos 92
4.4.1.6. Módulo de configuración 93
4.4.2. Interfaz gráfica 94
4.4.2.1. Ejemplo de uso de Tkinter 95
4.4.2.2. Interfaz del programa principal 101

5. Resultados y discusión **105**
5.1. Prueba de la interfaz 105
5.1.1. Importación de archivos 105
5.1.2. Exportación de archivos 107
5.1.3. Eliminación de archivos de la base de datos 107
5.1.4. Ejecución de sentencias SQL 108
5.1.5. Gestión de sentencias SQL 111
5.1.6. Generación de CSV a partir de los datos devueltos por sentencias SQL . 112
5.2. Prueba de las bases de datos 112

6. Conclusiones **119**

A. Esquemas de diseño relacional **121**

B. Estructura de los archivos manejados por la aplicación **125**
B.1. Estructura de los archivos Yaml de entrada de datos de la oferta 125
B.2. Estructura de los archivos Yaml de entrada de datos de la demanda 127

C. Introducción a la lógica difusa **129**
C.1. Conjuntos difusos 129
C.2. Variables lingüísticas 130
C.3. Proposiciones difusas 130
C.4. Reglas difusas 130

C.5. Modelos difusos 130
C.5.1. Modelo de Mamdani 130
C.5.2. Modelo Tsukamoto 131
C.6. Modelo TSK (Takagi-Sugeno-Kang) 131
C.7. Modelos difusos acumulativos 132

Bibliografía **133**

Índice de tablas

4.1. Tabla de ejemplo: CLIENTES 12
4.2. Tabla de ejemplo: PEDIDOS 12
4.3. Resultado de la sentencia del Listado 4.1 12
4.4. Resultados de la sentencia del Listado 4.2 13
4.5. Tabla con datos reales de un archivo de resultados 32
4.6. Primeras cinco filas de la salida de la sentencia SQL del Listado 4.28 40
4.7. Paradas de la línea 05065 44
4.8. Muestra de datos obtenidos de la sentencia del Listado 4.30 49

5.1. Datos exportados de las estaciones del ramal con identificador 10 112

Índice de figuras

3.1. Esquema de entrada y salida de archivos actual de ROBIN 8
3.2. Esquema de entrada y salida de archivos de ROBIN tras la realización del TFG 9

4.1. Diagrama de la estructura de las bases de datos. En naranja aparece la tabla principal, en amarillo las tablas auxiliares, en verde las tablas de datos y en azul las tablas de datos auxiliares. 33
4.2. Columnas de la tabla principal. 34
4.3. Columnas de la tabla de datos de la clave raíz 1. 35
4.4. Columnas de la tabla de datos de la clave raíz 2. 35
4.5. Columnas de la tabla de datos auxiliar de la clave raíz 2 con los datos asociados a la clave 2.7. 36
4.6. Columnas de la tabla auxiliar de la clave raíz 1. 36
4.7. Columnas de la tabla auxiliar de la clave raíz 2. 36
4.8. Esquema de la base de datos para archivos de entrada de datos de la oferta. En naranja aparece la tabla principal, en amarillo las tablas auxiliares, en verde las tablas de datos y en azul las tablas de datos auxiliares. 37
4.9. Tabla principal de la base de datos para archivos de oferta. 38
4.10. Ejemplo con datos reales de la tabla 4.9. 39
4.11. Tabla `STATIONS` 39
4.12. Tabla `TRAIN_SERVICE_PROVIDER` 40
4.13. Tabla `ROLLING_STOCK` 41
4.14. Tabla `TIME_SLOT` 41
4.15. Tabla `CORRIDOR` 42
4.16. Tabla `CORRIDOR_STATIONS` 42
4.17. Tabla `SEAT` 43
4.18. Tabla `LINE` 43
4.19. Tabla `STOPS` 44
4.20. Tabla `SERVICE` 45
4.21. Tabla `RESTRICTIONS` 45
4.22. Tabla `ORIGIN_DESTINATION_TUPLES` 46
4.23. Tabla `SEATS_PRICE` 46
4.24. Tabla `AUX_STATIONS` 47
4.25. Tabla `AUX_TRAIN_SERVICE_PROVIDER` 47

4.26. Tabla AUX_ROLLING_STOCK . . . 47
4.27. Tabla AUX_TIME_SLOT . . . 48
4.28. Tabla AUX_CORRIDOR . . . 48
4.29. Tabla AUX_SEAT . . . 48
4.30. Tabla AUX_SERVICE . . . 48
4.31. Esquema de la base de datos de la demanda. En naranja aparece la tabla principal, en amarillo las tablas auxiliares, en verde las tablas de datos y en azul las tablas de datos auxiliares. . . . 50
4.32. Tabla TESTS . . . 51
4.33. Tabla DEMAND_PATTERN . . . 51
4.34. Tabla MARKET . . . 51
4.35. Tabla MARKETS . . . 52
4.36. Tabla USER_PATTERN . . . 53
4.37. Tabla VARIABLE . . . 53
4.38. Tabla DAY . . . 54
4.39. Tabla USER_PATTERN_DISTRIBUTION . . . 54
4.40. Tabla AUX_DEMAND_PATTERN . . . 55
4.41. Tabla AUX_MARKET . . . 55
4.42. Tabla AUX_USER_PATTERN . . . 55
4.43. Tabla AUX_DAY . . . 56
4.44. Esquema de la base de datos de los resultados. En naranja aparece la tabla principal, en amarillo la tabla auxiliar y en verde la tabla de datos. . . . 56
4.45. Tabla TESTS . . . 57
4.46. Tabla RESULTS . . . 58
4.47. Tabla AUX_RESULTS . . . 58
4.48. Diagrama que muestra las relaciones entre los módulos en los que se ha dividido el programa . . . 59
4.49. Diagrama de flujo que muestra el proceso de lectura de un archivo Yaml mediante loadSupplyFile. . . . 63
4.50. Diagrama de flujo del proceso de extracción y procesado de los datos de la clave raíz stations con getStationsData. . . . 65
4.51. Diagrama de flujo de la función extractStationsFromCorridor . . . 68
4.52. Diagrama de flujo que muestra el proceso de lectura de un archivo Yaml mediante loadDemandFile. . . . 72
4.53. Diagrama de flujo que muestra el proceso de guardado de un archivo Yaml mediante saveFile. . . . 76
4.54. Diagrama de flujo que muestra el proceso de lectura de un archivo CSV mediante loadCsvFile. . . . 77
4.55. Diagrama de flujo que muestra el proceso de generación de un archivo CSV mediante saveFile. . . . 80
4.56. Diagrama de flujo para la creación de la tabla SERVICE. . . . 82
4.57. Diagrama de flujo de la función insertServiceData . . . 84

4.58. Diagrama de flujo de la función `deleteUnusedServiceData` 86
4.59. Diagrama de flujo de la función `getDataForTimeSlot` 89
4.60. Ventana de la aplicación de ejemplo 98
4.61. Prueba del botón "Añadir 1" 99
4.62. Prueba del botón "Restar 1" 100
4.63. Ventana principal con los campos de título y cuerpo del mensaje rellenos 101
4.64. Ventana de mensaje generada 101
4.65. Ventana principal de la aplicación 103
4.66. sentencia con error de columna faltante 103
4.67. Mensaje de error al ejecutar la sentencia con error de la Figura 4.66 104

5.1. Ventana de confirmación para añadir observaciones a la entrada del archivo. . . 105
5.2. Ventana para añadir observaciones. 106
5.3. Ventana de notificación para importación exitosa. 106
5.4. Ventana de notificación para importación fallida. 106
5.5. Ventana de notificación para archivo duplicado. 107
5.6. Ventana de selección de archivo. 107
5.7. Ventana de selección de la base de datos objetivo. 108
5.8. Desplegable con las sentencias almacenadas en la aplicación. 108
5.9. Ventana con los resultados de la sentencia "Mostrar test de oferta". 109
5.10. Ventana con los resultados de la sentencia del Listado 5.2. 110
5.11. Estaciones pertenecientes al ramal con identificador 10. 111
5.12. Ramales del corredor español. 111
5.13. Nueva lista de sentencias guardadas. 112

A.1. Esquema de diseño relacional para la base de datos de los archivos de entrada de datos de la oferta 122
A.2. Esquema de diseño relacional para la base de datos de los archivos de entrada de datos de la demanda 123
A.3. Esquema de diseño relacional para la base de datos de los archivos de resultados 123

C.1. Ejemplo de funciones trapezoidal, en azul, y triangular, en naranja. 129

Lista de acrónimos

API
Application Programming Interface. 94

CSV
Comma-Separated Values. I, II, V, VIII, 7–9, 13, 14, 32, 36, 40, 56, 75, 77–80, 109, 112, 113, 117–120

EDR
Esquema de diseño relacional. 37, 49, 56

GDI
Graphics Device Interface. 94

HTML
HyperText Markup Language. 13

JSON
JavaScript Object Notation. 38, 39, 41, 42, 46, 50–53, 93

ROBIN
Rail mOBIlity simulatioN. I, V, VII, 2, 3, 5, 7–9, 11, 13, 14, 25, 26, 36, 37, 45, 49, 56, 57, 120, 132

SQL
Structured Query Languaje. I, II, V, 9, 13, 34, 38–40, 44, 49, 79, 81, 83, 85, 88, 91, 93, 94, 102, 108–112, 119, 120

TB
Tera Byte. 13

Tcl
Tool Command Language. 94

TFG
Trabajo Fin de Grado. I, V, VII, 2, 3, 5, 8, 9, 11, 13, 15, 26, 36, 58, 94, 97, 101, 113, 119, 120, 132

XML
eXtensible Markup Language. 13

Yaml
YAML Ain't Markup Language. I, V, VIII, 8, 9, 13, 14, 20, 26, 34, 37, 38, 49, 50, 59, 60, 62, 63, 71, 72, 75, 76, 87, 107, 113, 114, 116, 119, 120

Capítulo 1

Introducción

A día de hoy, en el transporte ferroviario contemporáneo, alcanzar un equilibrio entre los asientos ofertados por las compañías que ofrecen servicios ferroviarios y la demanda real de pasajeros constituye un gran reto operativo. Esto se debe a la liberación y al aumento de la competencia en el sector ferroviario, sobre todo en el europeo, donde múltiples operadores luchan por captar a los mismos pasajeros. La venta de billetes ha de ajustarse a la capacidad efectiva de cada tren para así evitar tanto la saturación de dichos servicios como el desaprovechamiento de plazas, lo que produciría una bajada en el rendimiento del servicio.

La liberalización progresiva del sector ferroviario viene impulsada por la Unión Europea desde los años noventa, lo que abre el mercado a múltiples operadores. Se ha materializado en cuatro paquetes legislativos (1991, 2001, 2012 y 2016)[1] que establecen la separación de la infraestructura de la operación de servicios, la creación de organismos reguladores independientes y la armonización técnica para fomentar un espacio ferroviario europeo único. Estos marcos legales han permitido la entrada de nuevos competidores en corredores estratégicos, modificando profundamente la forma de asignar capacidad y gestionar la demanda.

En el caso concreto de España, tras su adhesión a la Unión Europea en 1986[2], la transposición de la política comunitaria se inició con la Ley 39/2003[3], que rompió el antiguo monopolio de RENFE, y separó las competencias de la empresa en dos: gestión de la infraestructura (ADIF[4]) y explotación (Grupo Renfe[5]). Posteriormente, la Ley 38/2015[6] y el Real Decreto-ley 23/2018[7] incorporaron íntegramente los paquetes ferroviarios europeos, fijando diciembre de 2020 como fecha de inicio efectiva de la liberalización del transporte de pasajeros en alta velocidad y larga distancia. En noviembre de 2019, ADIF adjudicó tres paquetes de capacidad (65 %, 30 % y 5 %) para los corredores Madrid – Barcelona – Frontera Francesa, Madrid – Levante y Madrid – Toledo – Sevilla – Málaga, reservándolos a Renfe, Iryo y Ouigo respecti-

[1] Recopilación de los diferentes paquetes legislativos del sector ferroviario por la Comisión Europea: https://transport.ec.europa.eu/transport-modes/rail/railway-packages_en (Último acceso: 8 de julio de 2025)

[2] Ley Orgánica 10/1985, de 2 de agosto, de Autorización para la Adhesión de España a las Comunidades Europeas, publicado en BOE num. 189, de 8/8/1985, páginas 25119 a 25119 (1 pág.), referencia BOE-A-1985-16659 https://www.boe.es/eli/es/lo/1985/08/02/10 (Último acceso: 8 de julio de 2025)

[3] Ley 39/2003, de 17 de noviembre, del Sector Ferroviario, publicado en BOE num. 276, de 18/11/2003 con referencia BOE-A-2003-20978 https://www.boe.es/eli/es/l/2003/11/17/39/con (Último acceso: 8 de julio de 2025)

[4] Sitio web de presentación de Adif: https://www.adif.es/sobre-adif/conoce-adif/quienes-somos (Último acceso: 8 de julio de 2025)

[5] Sitio web de presentación de *Grupo Renfe* y sus diferentes sociedades: https://www.renfe.com/es/es/grupo-renfe/grupo-renfe (Último acceso: 8 de julio de 2025)

[6] Ley 38/2015, de 29 de septiembre, del sector ferroviario, publicado en BOE num. 234, de 30/09/2015 con referencia BOE-A-2015-10440 https://www.boe.es/eli/es/l/2015/09/29/38/con (Último acceso: 8 de julio de 2025)

[7] Real Decreto-ley 23/2018, de 21 de diciembre, de transposición de directivas en materia de marcas, transporte ferroviario y viajes combinados y servicios de viaje vinculados, publicado en BOE núm. 312, de 27/12/2018 con referencia BOE-A-2018-17769 https://www.boe.es/eli/es/rdl/2018/12/21/23 (Último acceso: 8 de julio de 2025)

vamente. Estos operadores comenzaron sus servicios entre 2022 (Ouigo y AVLO, el servicio de bajo coste de Renfe) y diciembre de 2022 (Iryo), consolidando así la competencia en los principales corredores de alta velocidad.

La liberalización de cualquier mercado en general (y del ferroviario en particular) motiva la entrada de nuevas empresas que competirán por la demanda de los usuarios. En el caso del sector ferroviario, la infraestructura juega un papel fundamental y limitante a la hora de coordinar las peticiones de uso de la misma por parte de las compañías. Lo anterior supone un reto para el gestor de la infraestructura que debe organizar el uso de la infraestructura entre diferentes operadores considerando para ello tanto indicadores económicos como de equidad entre empresas. Por otra parte, las propias compañías deben enfrentar otra serie de desafíos motivados, entre otros, por la variabilidad en la demanda, lo que repercute, por ejemplo, en los servicios que se pondrán a disposición de la demanda así como sus precios.

Por este motivo, en la convocatoria nacional del Ministerio de Ciencia e Innovación de 2020, se solicitó, y fue concedido, el proyecto *Técnicas Analíticas para la transición hacia un transporte sostenible de pasajeros y mercancías en entornos de competitividad (ACoSeM@SusTran)*. Es un proyecto coordinado por las Universidades Universidad Politécnica de Cataluña, la Universidad Rey Juan Carlos y la Universidad de Castilla-La Mancha. Esta última ejecuta el proyecto titulado *Meeting Artificial Intelligence and Machine Learning for Rail Passenger Service Planning under Competition (PID2020-112967GB-C32)*. El objetivo principal de este proyecto consiste en el diseño de métodos de aprendizaje automático e inteligencia artificial para el análisis táctico de la competencia en los sistemas de transporte. Se pretende abordar los objetivos utilizando métodos de aprendizaje automático e inteligencia artificial que se basan en datos y no dependen del modelizador. La obtención de datos para utilizarlos en las investigaciones es un proceso complejo, dado que las compañías no ceden su información, y, en muchos casos, no existen. Por todo esto, se hace muy necesario el uso de herramientas de simulación robustas con este fin. Por este motivo, desde los grupos ORETO y MAT de la Escuela Superior de Informática de la Universidad de Castilla-La Mancha se ha desarrollado Rail mOBIlity simulatioN (ROBIN) [1]. ROBIN, un simulador de movilidad ferroviaria microscópico que permite modelar el flujo de viajeros en corredores ferroviarios. Este simulador emplea varios ficheros como entrada de datos y, al acabar la simulación, genera unos archivos donde se guardan los resultados de la misma. El manejo de estos archivos puede ser tedioso, debido a la cantidad de datos que se deben gestionar. Esto puede resultar un problema si se quieren tener todos los datos almacenados de manera ordenada en un mismo lugar.

Por ello, el objetivo de este TFG consiste en diseñar e implementar una serie de bases de datos encargadas de almacenar los ficheros que emplea ROBIN como entrada de información de la oferta y de la demanda, así como los archivos de resultados que se generan después de las simulaciones. Las bases de datos irán acompañadas de un programa destinado a su gestión por medio de una interfaz. Dicho programa tendrá la capacidad de importar los archivos a las bases de datos, exportarlos de las bases de datos, y eliminar los archivos que no sean útiles de dichas bases de datos. Esto permitirá ahorrar tiempo en la gestión de los archivos, ya que todos se encontrarán centralizados en un mismo punto.

1.1. Organización de la memoria

Este documento se ha estructurado en los siguientes capítulos:

Capítulo 2

Se definirá el objetivo principal del TFG, así como los diferentes objetivos que se deben llevar a cabo para completar el objetivo principal.

Capítulo 3
Se analizan los conocimientos previos y las tecnologías utilizadas por ROBIN y los problemas que plantea. Además, se exponen las posibles soluciones que podrían utilizarse para mejorar los problemas planteados, así como las tecnologías que podrían emplearse con este fin.

Capítulo 4
Se describe el proceso completo de diseño e implementación del proyecto. Este capítulo se ha dividido en 4 secciones:

Sección 4.1
Se exploran tanto el diseño como el funcionamiento de las bases de datos relacionales.

Sección 4.2
Se expone la estructura de los diferentes archivos empleados por ROBIN para la entrada y salida de datos.

Sección 4.3
Se detalla el diseño e implementación de las bases de datos necesarias para la consecución del proyecto.

Sección 4.4
Se muestra el diseño del software encargado de gestionar las bases de datos, así como de la creación de una interfaz para facilitar su uso.

Capítulo 5
Se analizan los resultados del proyecto obtenidos. Este capítulo se ha dividido en dos secciones.

Sección 5.1
Se realizarán pruebas para verificar el correcto funcionamiento del programa y verificar que es capaz de capturar ciertos errores que puedan surgir durante su uso.

Sección 5.2
Se comprueba que las bases de datos funcionan correctamente, es decir, que no corrompan o modifiquen los datos de los diferentes archivos almacenados en estas. Esto se ha hecho comparando los archivos originales con los que se exportan utilizando la aplicación.

Capítulo 6
Se recopilan las conclusiones del proyecto, los objetivos propuestos que se han cumplido, los objetivos académicos logrados durante el desarrollo del proyecto y se plantean posibles líneas de trabajo futuro.

Anexos
Se encuentran los esquemas de diseño relacional para cada una de las bases de datos, las plantillas estructurales de los archivos gestionados y una introducción a la lógica difusa.

Bibliografía
Recoge las referencias y recursos técnicos utilizados para el desarrollo y redacción de este TFG.

1.2. Repositorio de información

Todo el material generado durante la ejecución de este proyecto está disponible en el repositorio https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos. El material incluye el código LaTeX del presente documento, el código fuente de los programas realizados o modificados, y todos los datos generados en la evaluación de resultados.

Capítulo 2

Objetivos

El objetivo principal de este Trabajo Fin de Grado (TFG) consiste en el diseño e implementación de una serie de bases de datos destinadas a almacenar los diferentes datos empleados en la simulación de transporte ferroviario realizada por ROBIN, así como los resultados de las simulaciones que se realicen. Para ello, se utilizará el gestor de bases de datos relacionales SQLite3. Además, se diseñará e implementará una interfaz para interactuar con las diferentes bases de datos, con la ayuda de Python y la librería para generar interfaces Tkinter. Para la consecución de este objetivo se deben realizar los siguientes objetivos parciales:

- Estudiar el funcionamiento y diseño de las bases de datos relacionales y su aplicación concreta dentro de SQLite3.
- Diseñar e implementar las diferentes bases de datos.
- Diseñar y desarrollar un programa en Python que permita interactuar con las bases de datos, incluyendo la inserción, exportación, eliminación de datos y ejecución de sentencias SQL. El diseño será modular para poder añadir nuevas funcionalidades en el futuro.
- Investigar el proceso de creación de interfaces gráficas con Tkinter.
- Diseñar e implementar una interfaz gráfica para facilitar el uso del programa desarrollado.

Capítulo 3

Motivación y antecedentes

En la actualidad, existe un problema a la hora de encontrar un equilibrio entre la oferta de plazas disponibles y la demanda real de pasajeros en el transporte ferroviario. Esto supone un gran reto operativo para las empresas que ofertan servicios ferroviarios, dado que la venta de billetes debe ajustarse a la capacidad de cada tren, intentando evitar tanto una saturación de los servicios, como un bajo aprovechamiento de los mismos. Este problema surge debido a la liberalización y creciente competitividad en los mercados ferroviarios, especialmente en Europa, donde se ha fomentado la competencia entre distintos operadores ferroviarios con la búsqueda de un espacio ferroviario europeo único. Para abordar este problema, investigadores de la Universidad de Castilla-La Mancha, de los grupos ORETO y MAT de la Escuela Superior de Informática han desarrollado Rail mOBIlity simulatioN (ROBIN) [1] que consiste en un simulador destinado a modelar la movilidad de pasajeros en entornos ferroviarios en los que exista una competencia entre los diferentes operadores.

ROBIN adopta un enfoque completamente microscópico, simulando pasajero a pasajero y asiento a asiento. Para cada pasajero potencial, el simulador evalúa todas las alternativas de servicio disponibles (teniendo en cuenta para ello horarios, tipos de asiento y condiciones comerciales) y determina la elección final de cada usuario según reglas de decisión basadas en utilidad y aleatoriedad. De este modo, permite cuantificar con detalle la demanda atendida, los pasajeros que no viajan por falta de plazas y los flujos de ocupación en cada tramo de la red.

La arquitectura de ROBIN está dividida en tres componentes principales: un módulo de oferta que procesa la información de los servicios disponibles (obtenida generalmente mediante *scraping* de fuentes como la web de Renfe), un módulo de demanda que genera perfiles de usuario y distribuciones de viaje, y un núcleo de simulación que coordina ambas entradas para ejecutar el escenario diario. Al finalizar, todos los datos de salida (ocupaciones, rechazos de demanda y métricas de rendimiento) se exportan automáticamente en formato CSV para su posterior análisis.

Gracias a su diseño modular y parametrizable, ROBIN facilita la experimentación con distintos escenarios de mercado: variaciones en los hábitos de viaje, ajustes de precios, incorporación de nuevos operadores o cambios en la infraestructura. Esto convierte al simulador en una herramienta estratégica para evaluar el impacto de políticas de liberalización, estrategias de precios y planes de capacidad antes de su puesta en marcha real.

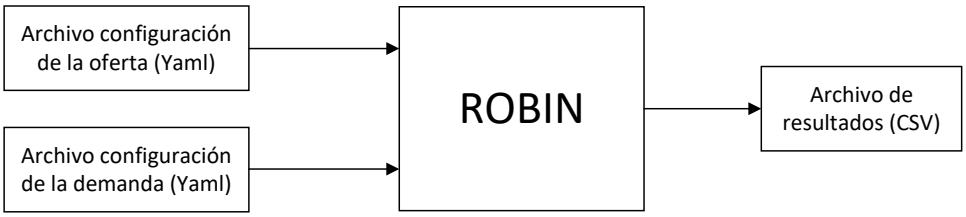


Figura 3.1: Esquema de entrada y salida de archivos actual de ROBIN

ROBIN emplea archivos en formato YAML Ain't Markup Language (Yaml) para cargar los datos de la oferta y la demanda para realizar las simulaciones, y genera como salida ficheros Comma-Separated Values (CSV) con los resultados de las mismas. El manejo de estos archivos puede resultar tedioso debido a la gran cantidad de información que se ha de manejar para realizar la simulación con ROBIN. Esto debido a que han de analizarse, para cada pasajero modelado en el sistema, todos los posibles asientos de cada servicio de forma individual.

Concretamente, el programa emplea dos archivos Yaml como entrada de datos: un archivo con la información de la oferta en el mercado que se va a simular y un archivo con los patrones de usuario y los patrones de demanda esperados para el día que se desea estudiar (Figura 3.1). La información de la oferta, que posteriormente se usará en el simulador, proviene de una herramienta de extracción de datos web (*scraper*) empleada en diferentes páginas, como por ejemplo, la página de venta al público de *Renfe*[1]. De esta forma, se recopilan los datos de todas las estaciones dentro del sistema ferroviario español. Este programa genera diferentes archivos con datos sobre la oferta, dependiendo del corredor español al que pertenecen los servicios y de las fechas de los mismos. Finalmente, al acabar la simulación, se genera un archivo CSV con los resultados obtenidos.

Los archivos Yaml tienen un acceso muy rápido para pequeños volúmenes de datos, pero cuando el volumen de datos aumenta, se vuelve más lento, debido a que hay que cargar y procesar el archivo completo en memoria para poder acceder a los datos. Esto también deriva en un problema de escalabilidad, ya que para grandes volúmenes de datos, habría que procesar un archivo que contiene mucha información, lo que puede derivar en demoras a la hora de cargar el archivo en memoria. Por todo esto, las bases de datos relacionales son una alternativa al aportar una serie de ventajas:

- Son rápidas y eficientes a la hora de manejar grandes volúmenes de datos debido a que la información se procesa por bloques.
- Tienen la información bien organizada y previenen la duplicidad de información.
- Permiten consultas sobre los resultados obtenidos de una forma más sencilla y eficiente que en el archivo CSV.

En este contexto, este TFG se centra en la creación de una herramienta para gestionar los ficheros de entrada de datos de la oferta y de la demanda, en formato Yaml, y los ficheros de resultados, en formato CSV asociados a ROBIN utilizando bases de datos relacionales. De esta forma, la información de los archivos de entrada y salida que emplea ROBIN en la actualidad se almacenará en 3 bases de datos, una para cada tipo de archivo, para un mejor manejo y tratamiento de la información, siendo la herramienta desarrollada la que se encargue de la gestión de dichos archivos. De esta manera, toda la información relacionada con los diferentes archivos, tanto de entrada como de salida, quedará almacenada en un único punto. Además, se pretende diseñar y desarrollar una interfaz para el manejo de los datos de entrada y salida, lo que permitirá realizar consultas sobre los datos manejados y generados en las simulaciones. Esto es de gran utilidad dentro de este entorno. La Figura 3.2 muestra la nueva estructura de trabajo

[1]Web de venta de billetes de Renfe: https://www.renfe.com/es/es

de ROBIN, en la que se muestra cómo se generarán los archivos Yaml desde la información de las bases de datos. Además, los resultados de las simulaciones se almacenarán en una base de datos de salida que permitirá realizar consultas sobre éstos.

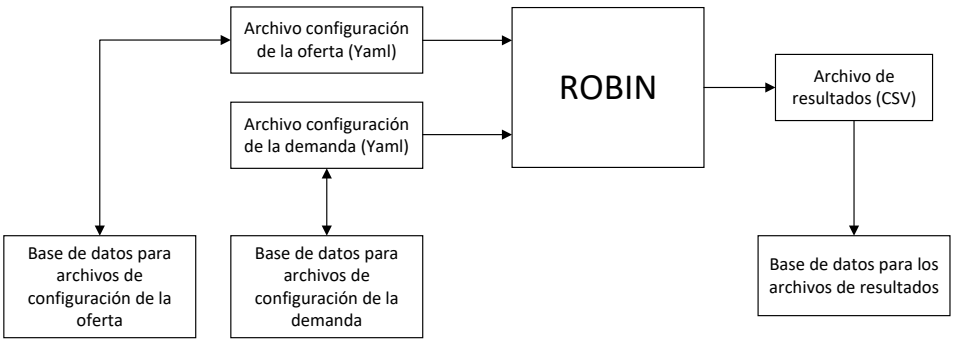


Figura 3.2: Esquema de entrada y salida de archivos de ROBIN tras la realización del TFG

Para la creación de sistemas de gestión de archivos, pueden emplearse diversos lenguajes de programación como C, Java, Python, etc. Uno de los más útiles, en este contexto, para la programación e implementación de este gestor, es el lenguaje Python [2] [3] [4] debido a que se trata de un lenguaje fácil de aprender y cuenta con una extensa cantidad de librerías, lo que permite una gran variedad de opciones para el desarrollo. Dentro de la gran cantidad de librerías de las que dispone Python, cabe recalcar algunas que son especialmente útiles para los objetivos de este TFG, como son:

- **PyYAML [5] [6] [7]:** Esta librería permite la lectura y creación de archivos en formato Yaml. Permite a Python obtener datos de los ficheros con formato Yaml al transformar los datos contenidos en este a estructuras que puedan ser utilizadas en Python como listas o diccionarios. También permite transformar estas estructuras al formato empleado por los ficheros Yaml.
- **sqlite3 [8] [9] [10]:** Se trata de una interfaz DB-API 2.0[2] para trabajar con bases de datos SQLite desde Python. Esto permite la creación y edición de bases de datos, así como, la realización de consultas a estas bases de datos.
- **csv [11] [12]:** Esta librería se trata de un módulo nativo de Python para leer y escribir archivos CSV.
- **Tkinter [13] [14] [15]:** Se trata de una biblioteca estándar de Python para interfaces gráficas de usuario (GUI). Permite construir ventanas, menús, formularios y controles sin necesidad de instalar dependencias externas.

Otra herramienta que cabe mencionar es SQLiteStudio [16] que resulta muy útil para la creación, edición y visualización de bases de datos que empleen SQLite como motor de base de datos. Además, permite interaccionar con las bases de datos conectadas a la aplicación mediante el uso de sentencias con lenguaje Structured Query Languaje (SQL).

[2]Define un conjunto de métodos y convenciones para acceder de forma uniforme a distintas bases de datos desde Python.

Capítulo 4

Desarrollo del TFG

En este TFG se pretende desarrollar un software modular que sirva para manejar los datos de los diferentes archivos que componen la configuración y los resultados del simulador ROBIN mediante un conjunto de bases de datos. Este software maneja los datos de una forma más eficiente que la actual, mediante la utilización de bases de datos relacionales. Esto es debido a que no es necesario almacenar todo el contenido de los archivos de entrada de datos y resultados de forma independiente. El uso de estas bases de datos aprovecha la existencia de relaciones entre la información de los archivos, evitando la duplicidad de información gracias a la utilización de tablas relacionadas entre sí, y permitiendo acceder a toda la información del simulador.

Este capítulo está estructurado de la siguiente forma. Inicialmente se presenta un estudio de las Bases de Datos relacionales para que el lector pueda entender el funcionamiento de las mismas (Sección 4.1). A continuación, se detalla cómo está estructurada la información en los archivos Yaml de ROBIN (Sección 4.2). Una vez explicados estos archivos, en la Sección 4.3, se expondrá el diseño e implementación de las Bases de Datos que contendrán la información de los archivos Yaml. Finalmente, se presentará el diseño y desarrollo del software elaborado en este TFG (Sección 4.4), junto con la interfaz gráfica diseñada.

4.1. Estudio del funcionamiento de bases de datos relacionales

Para el almacenamiento de los datos, tanto de la configuración como de los resultados del simulador, se ha optado por emplear bases de datos relacionales, debido a que es una solución eficiente para el almacenamiento masivo de datos que están relacionados entre sí, como es el caso de ROBIN.

Una base de datos relacional está diseñada como un sistema estructurado donde se almacenan, organizan y gestionan datos en tablas interrelacionadas entre sí, de ahí que a este tipo de bases de datos se les denomine bases de datos relacionales. Estas bases de datos tienen una serie de características que facilitan el almacenamiento, tratamiento y consulta de la información, como por ejemplo, la organización de los datos en tablas, el empleo de comandos SQL para su manipulación y el uso de relaciones lógicas entre tablas, entre otras.

Los datos se organizan en tablas, que se componen de filas y columnas, comúnmente denominadas registros y campos, respectivamente. Cada columna dentro de la tabla tiene asignado un tipo de datos específico, como números, texto, fechas, etc. A su vez, una de estas columnas debe ser una clave primaria que identifica inequívocamente al registro y que se puede emplear en la creación de relaciones con otras tablas de la base de datos. Además, algunas columnas pueden pertenecer a claves primarias de otras tablas, lo que se denomina como una clave externa. Estas claves externas sirven para definir las relaciones entre las tablas. Dichas relaciones

vinculan las tablas dentro de la base de datos para que operen de forma conjunta.

	ID_CLIENTE (**Clave Primaria**)	NOMBRE	EMAIL
1	1	Juan Pérez	juan.perez@email.com
2	2	María López	maria.lopez@email.com
3	3	Carlos García	carlos.garcia@email.com

Tabla 4.1: Tabla de ejemplo: CLIENTES

	ID_PEDIDO (**Clave Primaria**)	FECHA	ID_CLIENTE (**Clave externa**)
1	101	18/03/2025	1
2	102	19/03/2025	2
3	103	20/03/2025	1
4	104	21/03/2025	3

Tabla 4.2: Tabla de ejemplo: PEDIDOS

A modo de ejemplo, se mostrará una base de datos donde se tienen almacenados los datos de los clientes que posee una empresa (Tabla 4.1) y los pedidos realizados a dicha empresa (Tabla 4.2). Los clientes y pedidos tienen la clave primaria ID_CLIENTE e ID_PEDIDO respectivamente. La tabla PEDIDOS utiliza como clave externa la clave primaria de CLIENTES para relacionar los pedidos con el cliente que los realizó. Por ejemplo, si se observa el pedido 101, se puede comprobar que lo hizo el cliente 1; es decir, Juan Pérez con email juan.perez@email.com, junto con el resto de sus datos. Con estas relaciones se evita la replicación de información en diferentes lugares.

Listado 4.1: Seleccion de pedidos del cliente 1

```
-- Pedidos del cliente con id 1
SELECT *
FROM PEDIDOS
WHERE PEDIDOS.ID_CLIENTE = 1
```

Además, se dispone del lenguaje de consultas SQL que permite realizar consultas de alto nivel. En el ejemplo, si se desea comprobar qué pedidos ha realizado el cliente 1 se puede utilizar el comando del Listado 4.1 que selecciona los pedidos del cliente 1 dentro de la tabla PEDIDOS.

	ID_PEDIDO (**Clave Primaria**)	FECHA	ID_CLIENTE (**Clave externa**)
1	101	18/03/2025	1
2	103	20/03/2025	1

Tabla 4.3: Resultado de la sentencia del Listado 4.1

Utilizando la relación entre las tablas de clientes y de pedidos, se pueden cruzar los datos de ambas para, por ejemplo, mostrar en una misma respuesta los datos referentes al cliente junto con los datos del pedido. Esto se realiza utilizando la sentencia SQL mostrada en el Listado 4.2. Esta sentencia devuelve la información relacionada con los pedidos realizados por el cliente 1 añadiendo el nombre y el email, información que se encuentra en la tabla `CLIENTES`. Para lograr esto, se emplea el comando `INNER JOIN` que relaciona `CLIENTES` con `PEDIDOS` mediante la condición `PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE`, combinando los valores de ambas tablas para el cliente que posea el identificador 1 (`CLIENTES.ID_CLIENTE = 1`). La salida de esta sentencia se encuentra en la Tabla 4.4.

Listado 4.2: Seleccion de pedidos del cliente 1 con datos cruzados

```
-- Pedidos del cliente con id 1 con datos del cliente
SELECT
    PEDIDOS.ID_PEDIDO,
    PEDIDOS.FECHA,
    CLIENTES.NOMBRE,
    CLIENTES.EMAIL
FROM
    CLIENTES

INNER JOIN
    PEDIDOS
ON
    PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE

WHERE
    CLIENTES.ID_CLIENTE = 1
```

	ID_PEDIDO	FECHA	NOMBRE	EMAIL
1	101	18/03/2025	Juan Pérez	juan.perez@email.com
2	103	20/03/2025	Juan Pérez	juan.perez@email.com

Tabla 4.4: Resultados de la sentencia del Listado 4.2

Como motor de base de datos se ha elegido SQLite3 [8], ya que es lo suficientemente potente para llevar a cabo este TFG. SQLite3 es una biblioteca multiplataforma escrita en lenguaje C, que implementa un motor de base de datos SQL pequeño, rápido, autónomo, altamente confiable y con todas las funciones de una base de datos SQL. SQLite3 puede operar sin la necesidad de emplear un servidor que aloje la base de datos, dado que los datos se almacenan en archivos dentro del dispositivo que emplee los programas que utilicen SQLite3. Esto, además, brinda la posibilidad de mover estos archivos entre diferentes sistemas sin que suponga un problema, pudiéndose usar así, en cualquier dispositivo con soporte para SQLite3. Dichas algunas de las ventajas, ahora se expondrán algunos de los inconvenientes del empleo de SQLite3. SQLite3 no es ideal para aplicaciones con alta concurrencia; es decir, que múltiples usuarios estén accediendo o modificando la base de datos al mismo tiempo. Esto se debe al modelo de bloqueo que implementa. También presenta una limitación en el tamaño máximo del archivo que puede manejar: unos 140 Tera Byte (TB). A diferencia de otros sistemas de bases de datos SQL como MySQL o PostgreSQL, no posee funcionalidades avanzadas como roles definidos, usuarios, replicación o clustering. Estas desventajas no suponen un gran impedimento para el desarrollo de este TFG, ya que no se espera que haya más de un usuario accediendo a la base de datos, ni que la base de datos supere este límite de almacenamiento, ni tampoco sea necesario el uso de funciones avanzadas para la consecución de este TFG.

4.2. Archivos de entrada y salida del simulador ROBIN

Actualmente, el simulador ROBIN emplea dos archivos Yaml para la entrada de datos y genera un archivo CSV donde se reflejan los resultados de la simulación. Yaml es un formato de serialización de datos diseñado para ser legible por humanos. A diferencia de otros lenguajes como HyperText Markup Language (HTML) o eXtensible Markup Language (XML), Yaml se enfoca en la representación de datos de forma estructurada mediante indentación y asociaciones

clave-valor (diccionarios). Esta simplicidad hace que Yaml sea especialmente adecuado para definir estructuras de datos jerárquicas de forma clara y concisa.

Dentro de un archivo Yaml, la clave raíz (o clave de primer nivel) es el elemento que aparece en el primer nivel de la jerarquía estructurada del documento. Cada clave raíz actúa como un contenedor principal de un bloque de información, del cual dependen las claves secundarias, listas o asociaciones anidadas que componen la estructura de datos. Un mismo archivo puede contener múltiples claves raíz. En el Listado 4.3, se muestra un ejemplo de un archivo Yaml que emplea las claves raíz `oferta` y `demanda`, conteniendo también claves secundarias como `estaciones` y `operadores`.

Listado 4.3: Ejemplo de archivo Yaml con múltiples claves raíz

```yaml
oferta:
  estaciones: [ES1, ES2, ES3]
  operadores:
    - nombre: Renfe
      trenes: [S-114]

demanda:
  mercados:
    - id: 1
      nombre: ES1-ES3
      origen: ES1
      destino: ES3
  frecuencia:
    mercado: 1
    frecuencia: frecuente
```

El primero de los archivos de entrada a ROBIN almacena todos los datos relacionados con la oferta de servicios ferroviarios como, por ejemplo, el corredor que va a emplearse para realizar las simulaciones, las estaciones que aparecen en este corredor, los proveedores de servicios ferroviarios que ofrecen servicios en ese corredor, etc. La estructura de este archivo de configuración de la oferta se detalla en la Sección 4.2.1.

El segundo archivo de entrada a ROBIN contiene los datos relacionados con la demanda de servicios como, por ejemplo, los patrones que modelizan el comportamiento de los usuarios de los servicios ferroviarios, los mercados empleados en la simulación, la demanda que se espera que tengan los mercados, etc. La estructura de este archivo de configuración de la demanda se detalla en la Sección 4.2.2.

Por otro lado, el archivo de resultados generado por el simulador ROBIN contiene los resultados de la simulación, donde cada fila representa un viaje individual con todos los datos asociados a este. La estructura de este archivo de configuración de la demanda se detalla en la Sección 4.2.3.

Las tres siguientes subsecciones detallan la estructura de los dos archivos Yaml de entrada y del CSV de salida. Finalmente, comentar que la estructura de los archivos Yaml se encuentra en el Anexo B, en el que se pueden ver tanto la estructura de los archivos de configuración de la oferta como la de los archivos de configuración de la demanda.

4.2.1. Estructura de los archivos de configuración de la oferta

Los archivos de oferta tienen como propósito almacenar los datos referentes a la oferta ferroviaria. Para representar la oferta ferroviaria se consideran los siguientes elementos:

- Estaciones: Las estaciones son los puntos de la red ferroviaria en las que los trenes paran para embarcar o desembarcar pasajeros y mercancías. Estas forman parte de un corredor ferroviario y pueden ser el inicio o el final de una o varias lineas de tren o una parada intermedia dentro de una línea.
- Asientos: Los asientos, en este caso, hacen referencia al tipo de billete que se encuentra disponible para su compra por usuarios de servicios ferroviarios. Estos pueden emplear asientos físicos con diferentes características, como el tamaño, si cuentan con respaldo reclinable, conexión para la carga de terminales moviles, etc. Además, estos billetes también tienen diferentes tipos de servicios asociados a ellos dependiendo del tipo de billete que se compre, como por ejemplo, reembolso parcial o total del billete en caso de incidencias con el servicio o la posibilidad de usar áreas de descanso reservadas a los poseedores de un cierto tipo de billete.
- Corredor ferroviario: El corredor es un conjunto de vías que conectan dos o más estaciones. Define la infraestructura técnica de la red (ancho de la vía, electrificación) y suele agrupar varias líneas de tren que circulan por el trazado del corredor. Puede haber múltiples líneas en un corredor.
- Líneas de tren: Las líneas de tren hacen referencia a las rutas comerciales que cubren el trayecto entre un origen y un destino pasando por una secuencia de estaciones en las que puede haber una parada programada. Estas suelen llevar un identificador asociado a ellas, horarios fijos, e incluso algunas pueden llegar a tener variaciones estacionales o de servicio.
- Material rodante: Agrupa los diferentes trenes y material rodante en general en función de la cantidad y tipos de asientos de los que disponga cada uno de los trenes.
- Proveedores de servicios ferroviarios: Un proveedor de servicios ferroviarios es una empresa que opera en las líneas y gestiona los trenes que tiene a su disposición. Se encargan de la venta de billetes, de la puesta en marcha de los trenes y su mantenimiento, etc. Pueden ser empresas dedicadas al transporte de viajeros, mercancías o ambas. Además, pueden ser empresas públicas o privadas.
- Franjas de tiempo de los servicios: Las franjas de tiempo de los servicios son periodos horarios en los que se encuentra el tren estacionado en las vías junto al andén.
- Servicios ferroviarios: Los servicios ferroviarios se corresponden con cada uno de los viajes programados de un tren sobre una linea concreta, con una hora de salida desde la estación de origen, un destino, paradas intermedias y diferentes condiciones de viaje ligadas al tipo de billete que se haya adquirido.

A continuación, se presentarán los siguientes ejemplos:

- Estaciones: Un ejemplo para una estación podría ser la estación Madrid Puerta de Atocha - Almudena Grandes ubicada en la ciudad de Madrid o la estación Joaquín Sorolla ubicada en la ciudad de Valencia.
- Asientos: Como ejemplo para un tipo de billete podrían ser los que emplea Renfe en la fecha actual de realización de este TFG, los cuales pueden ser: Básico, Elige, Elige confort y Prémium. Cada uno de estos tiene diferentes tipos de asientos físicos: los billetes Básico y Elige tienen un asiento estándar, mientras que Elige confort y Prémium tienen un asiento confort, el cual es más amplio y cuenta con reposa brazos individuales para cada asiento. Además, cada uno de los billetes cuenta con una serie de servicios asociados, como la posibilidad de obtener un cambio de billete gratuito, en el caso del billete Elige, o la posibilidad de realizar cambios de billete y titular gratuitos de manera ilimitada para el caso de los billetes Prémium.
- Corredor ferroviario: En el caso del corredor ferroviario, un ejemplo podría ser el Co-

rredor Noreste, el cual recorre la Península Ibérica desde Madrid hasta la frontera con Francia.

- Líneas de tren: Una línea que podría servir como ejemplo es la línea AVE Madrid-Barcelona. Se trata de una línea de alta velocidad que se abarca las estaciones comprendidas entre las ciudades de Madrid y Barcelona.
- Proveedores de servicios ferroviarios: Ejemplos de empresas proveedoras de servicios ferroviarios publicas serían Renfe Viajeros, destinada al transporte de pasajeros, o Renfe Mercancías, destinada al transporte de mercancías y ejemplos de empresas privadas podrían ser Ouigo o Iryo, en el transporte de pasajeros o CEFSA (Compañía Europea Ferroviaria) para el transporte de mercancías.
- Material rodante: Un ejemplo de un modelo de tren podría ser el modelo S-114 de Renfe. El S-114 es un tren de alta velocidad, diseñado para trayectos de media distancia, el cual consta de 237 plazas.
- Franjas de tiempo de los servicios: Un ejemplo para franja de tiempo seria la que siguiera un tren cuyo servicio empieza a las 6:32 de la mañana y sale de la estación a las 6:42 de la mañana. Dicho esto, la franja horaria que tendría sería 6:32-6:42.
- Servicios ferroviarios: Un ejemplo de servicio ferroviario sería el AVE 03206, que realiza un trayecto directo con origen en estación de Madrid Puerta de Atocha, y como destino tiene la de Barcelona Sants, en un día determinado, con las capacidades y precios que el operador haya establecido.

En base a esto, los archivos de oferta se componen de 8 claves raíz para recoger esta información. Estas claves raíz son: `stations`, `seat`, `corridor`, `line`, `rollingStock`, `trainServiceProvider`, `timeSlots` y `service` (Listado B.1). Cada una define un bloque concreto de los datos de configuración vinculados a la oferta ferroviaria.

En las siguientes secciones se detalla la estructura de cada uno de estos 8 bloques identificados por su clave raíz.

4.2.1.1. Estaciones

La clave raíz `stations` alberga la información de todas las estaciones que forman parte del corredor ferroviario empleado en la simulación. Cada estación se representa como un elemento dentro de la lista asociada a esta clave raíz, incluyendo los datos necesarios para su identificación y localización.

Listado 4.4: Estructura de la clave raíz `stations`

```yaml
# Lista con los datos de las estaciones:
stations:
  - id: <Identificador de la estación>
    name: <Nombre de la estación>
    city: <Ciudad en la que se encuentra la estación>
    short_name: <Nombre corto de la estación>
    coordinates:
      latitude: <Latitud a la que se encuentra la estación>
      longitude: <Longitud a la que se encuentra la estación>
```

En el Listado 4.4, se muestra la estructura de la clave raíz `stations`. Esta contiene una lista con los datos de las diferentes estaciones, donde cada elemento de esta lista es un diccionario al que se accede con las siguientes claves: `id`, `name`, `city`, `short_name` y `coordinates`. La estructura clave-valor de cada elemento de la lista asociada a `stations` es la siguiente:

- `id`: Esta clave accede al identificador único para una estación en concreto. Por ejemplo, para el caso de la estación de Atocha sería el número "60000".
- `name`: Utilizada para acceder al nombre de la estación. Por ejemplo, para el caso de Atocha, el nombre sería "MADRID PTA. ATOCHA - ALMUDENA GRANDES".
- `city`: Permite el acceso a la ciudad en la que se encuentra la estación.
- `short_name`: Cada estación cuenta con un nombre corto al que se accede con esta clave. Por ejemplo, a Atocha, se le asigna el nombre corto de "MADRI".
- `coordinates`: Las coordenadas de la estación se localizan mediante esta clave. Las coordenadas están almacenadas en un diccionario con las claves `latitude` y `longitude`, que reciben los valores de latitud y longitud respectivamente.

El Listado 4.5 muestra un caso de uso de un ejemplo real de la estructura con datos presentados anteriormente y que define las estaciones de Atocha y de Joaquín Sorolla.

Listado 4.5: Ejemplo con datos reales de la estructura de `stations`

```
stations:
  - id: '60000'
    name: MADRID PTA. ATOCHA - ALMUDENA GRANDES
    city: MADRID
    short_name: MADRI
    coordinates:
      latitude: 40.406442
      longitude: -3.690886
  - id: '03216'
    name: VALENCIA JOAQUÍN SOROLLA
    city: VALENCIA
    short_name: VALEN
    coordinates:
      latitude: 39.459051
      longitude: -0.382923
```

4.2.1.2. Asientos

La clave raíz `seat` contiene la información de los diferentes tipos de asientos ofertados por los proveedores de servicios ferroviarios. Cada tipo de asiento se representa mediante un elemento de la lista asociada a `seat`, donde se encuentra la información necesaria para definir cada tipo de asiento.

Listado 4.6: Estructura de la clave raíz `seat`

```
#Lista con los asientos ofertados
seat:
- id: <Identificador del asiento>
  name: <Nombre del identificador>
  hard_type: <Tipo de asiento físico>
  soft_type: <Tipo de asiento según los servicios incluidos>
```

En el Listado 4.6 se encuentra la plantilla que sigue `seat`. La estructura clave-valor de cada elemento dentro de la lista dependiente de `seat` es la siguiente:

- `id`: Esta clave accede al identificador único que recibe cada tipo de asiento.

- `name`: En esta clave se guarda el nombre del tipo de asiento. Los nombres de los tipos de asiento actualmente son: Básica, Básico, Elige, Elige Confort y Prémium.
- `hard_type`: Esta clave contiene el tipo físico del asiento. Los tipos de asiento físico disponibles son: 1 (asiento básico) y 2 (asiento más grande y con reposa brazos individual).
- `soft_type`: En esta clave se almacena el tipo de asiento según los servicios asociados a este. Los tipos de servicios son actualmente: 1, 2, 3 y 4.

En el Listado 4.7 se muestra un ejemplo de cómo se definiría un tipo de asiento. En este ejemplo se define un asiento básico que recibe el identificador "1", y con los tipos físico y de servicios definidos como "1".

Listado 4.7: Ejemplo con datos reales de la estructura de `seat`

```
seat:
  - id: '1'
    name: Básica
    hard_type: 1
    soft_type: 1
  - id: '5'
    name: Prémium
    hard_type: 2
    soft_type: 4
```

4.2.1.3. Corredores

La clave raíz `corridor` contiene los datos de los corredores ferroviarios que se van a emplear en la simulación. Cada corredor se define mediante un identificador, un nombre y una estructura de datos que define los diferentes ramales del corredor.

Listado 4.8: Estructura de la clave raíz `corridor`

```
#Lista con la información de los corredores ferroviarios
corridor:
- id: <Identificador del corredor>
  name: <Nombre del corredor>

  #Lista de diccionarios que define el corredor
  stations:
  - org: <Estación de origen>
    des: # Si hay más estaciones, esto es un diccionario
    - org: <Estación de origen>
      des: [] # Para indicar el final del corredor
              # se usa una lista vacia
```

El Listado 4.8 contiene la plantilla estructural que sigue `corridor` para construir un corredor. La estructura clave-valor de cada corredor dentro de `corridor` es:

- `id`: Esta clave almacena el identificador único que recibe el corredor ferroviario.
- `name`: Esta clave contiene el nombre que recibe el corredor ferroviario.
- `stations`: En esta clave se definen los diferentes ramales que posee el corredor mediante el uso de diccionarios. Estos diccionarios se agrupan formando cadenas de tramos entre estaciones donde cada tramo se representa mediante un diccionario con 2 campos, uno

para la estación de origen denominado `org`, que alberga el identificador de la estación de origen, y otro para la estación de destino denominado `des`, en el que se almacena una lista que contiene los tramos posteriores. Para indicar la finalización de un ramal, la lista almacenada en `des` estará vacía. Empleando esta estructura recursiva se puede modelar el orden secuencial de las estaciones que componen el corredor. Además, esta estructura permite representar bifurcaciones en los ramales si en la lista dentro de `des` se incluyeran múltiples tramos definidos por diccionarios diferentes.

En el Listado 4.9 se muestra un ejemplo de cómo se definiría un corredor. En este ejemplo, el corredor definido no está completo, debido al tamaño que requeriría el mostrar el corredor español completo, por lo que se ha optado por mostrar únicamente el ramal que parte desde la estación de Madrid-Chamartín-Clara Campoamor (con el identificador 17000), en Madrid y va hasta las estaciones de Castelló (con el identificador 65300), en Castellón de la Plana y de Murcia (con el identificador 61200), en Murcia. La bifurcación se produce en la estación de Cuenca Fernando Zóbel (con identificador 03208), situada en Cuenca y de ahí se bifurca. Una de las estaciones de la bifurcación es la de Requena Utiel (con identificador 03213), situada en Requena y cuyo ramal termina en la estación de Castelló. La otra estación de la bifurcación es la de Albacete-Los Llanos (con identificador 60600), situada en Albacete y su ramal termina en la estación de Murcia.

Listado 4.9: Ejemplo con datos reales de la estructura de `corridor`

```
corridor:
- id: '1'
  name: Spanish Corridor
  stations:
  - org: '17000'
    des:
    - org: 03208
      des:
      - org: '03213'
        des:
        - org: '03216'
          des:
          - org: '65200'
            des:
            - org: '65300'
              des: []
      - org: '60600'
        des:
        - org: 03309
          des:
          - org: '60911'
            des:
            - org: '03410'
              des:
              - org: '62002'
                des:
                - org: '61200'
                  des: []
```

4.2.1.4. Línea

La clave raíz `line` alberga las diferentes líneas de tren empleadas en la simulación. Cada línea viene definida por un identificador único, el nombre que recibe, el identificador del corredor por el que discurre y la lista de paradas que se realizan durante el trayecto.

Listado 4.10: Estructura de la clave raíz `line`

```
#Lista con los datos de las líneas
line:
- id: <Identificador de la línea>
  name: <Nombre de la línea>
  corridor: <Corredor al que pertenece la línea>
  stops: # Lista con las paradas de la línea
    - station: <Identificador de la estación de llegada>
      arrival_time: <Tiempo relativo de llegada a la estación>
      departure_time: <Tiempo relativo de salida de la estación>
```

En el Listado 4.10 se muestra la plantilla estructural que sigue `line` en el archivo Yaml para definir cada línea. La estructura clave-valor de cada elemento dentro de la lista dependiente de `line` está compuesta por las siguientes claves:

- `id`: Almacena el identificador único que recibe cada línea.
- `name`: Contiene el nombre de la línea.
- `corridor`: En esta clave se guarda el corredor sobre el que discurre la línea.
- `stops`: Contiene una lista de diccionarios, en los que cada diccionario almacena una parada en la línea. La estructura de los diccionarios es la siguiente:
  - `station`: Contiene el valor del identificador de la estación en la que se efectúa la parada.
  - `arrival_time`: Almacena el tiempo relativo de la llegada a la estación. Este tiempo se expresa en minutos y cuenta desde la salida de la primera estación.
  - `departure_time`: Guarda el tiempo relativo de la salida de la estación. Este tiempo, al igual que en el caso de `arrival_time`, se expresa en minutos y cuenta a partir del instante de salida desde la primera estación.

En el Listado 4.11 se muestra un ejemplo de cómo se definiría una línea empleando la estructura de la plantilla para `line`. En esta línea aparece el trayecto entre la estación de Valencia Joaquín Sorolla (con identificador 03216), ubicada en Valencia, y la estación Madrid-Chamartín-Clara Campoamor (con el identificador 17000), en Madrid, con una duración de 126 minutos, o lo que es lo mismo, 2 horas y 6 minutos.

Listado 4.11: Ejemplo con datos reales de la estructura de `line`

```
line:
- id: '05065'
  name: Line 05065
  corridor: '1'
  stops:
  - station: '03216'
    arrival_time: 0
    departure_time: 0
  - station: '03213'
    arrival_time: 25
```

```
    departure_time: 27
  - station: 03208
    arrival_time: 61
    departure_time: 63
  - station: '17000'
    arrival_time: 126
    departure_time: 126
```

4.2.1.5. Material rodante - `rollingStock`

La clave raíz `rollingStock` almacena los diferentes trenes disponibles. Cada tren se define con un identificador, el nombre que tiene el modelo de tren y una lista con diccionarios que definen la cantidad de asientos por tipo físico que tiene el tren.

Listado 4.12: Estructura de la clave raíz `rollingStock`

```
#Lista con los trenes en servicio
rollingStock:
- id: <Identificador del tren>
  name: <Nombre del tren>
  seats: # Lista con los tipos de asientos físicos que tiene el tren
  - hard_type: <Tipo de asiento físico>
    quantity: <Cantidad del tipo de asiento>
```

En el Listado 4.12 se encuentra la plantilla estructural que sigue `rollingStock` para definir cada tren o instancia de material rodante en general. La estructura clave-valor de cada elemento dentro de la lista asociada a `rollingStock` es:

- `id`: Esta clave almacena el identificador único que recibe cada tren.
- `name`: En esta clave se guarda el nombre del modelo de tren.
- `seats`: Esta clave contiene una lista de diccionarios, donde cada uno de los diccionarios se compone de las siguientes claves:
  - `hard_type`: Identificador del tipo de asiento físico.
  - `quantity`: Cantidad que posee el tren del tipo de asiento físico definido en `hard_type`.

El Listado 4.13 muestra un ejemplo de cómo se definiría un modelo de tren. En este ejemplo se ha creado una entrada para el modelo de nombre "S-114", cuyo identificador es el "1" y que posee 250 asientos del tipo físico 1 y 50 asientos del tipo físico 2.

Listado 4.13: Ejemplo con datos reales de la estructura de `rollingStock`

```
rollingStock:
- id: '1'
  name: S-114
  seats:
  - hard_type: 1
    quantity: 250
  - hard_type: 2
    quantity: 50
```

4.2.1.6. Proveedores de Servicio

La clave raíz `trainServiceProvider` almacena los datos de los diferentes proveedores de servicios ferroviarios. Cada proveedor de servicios ferroviarios se define mediante un identificador único, el nombre del proveedor y una lista con el material rodante que posee.

Listado 4.14: Estructura de la clave raíz `trainServiceProvider`

```
#Lista con los proveedores de servicios ferroviarios
trainServiceProvider:
- id: <Identificador del proveedor>
  name: <Nombre del proveedor>

  #Lista de los trenes que posee el proveedor de servicios ferroviarios
  rolling_stock:
  - <Identificador del tren 1>
  - <Identificador del tren n>
```

En el Listado 4.14 se encuentra la estructura que sigue `trainServiceProvider`. La estructura clave-valor de cada elemento dentro de `trainServiceProvider` es:

- `id`: Identificador único que recibe el proveedor de servicios ferroviarios.
- `name`: Nombre del proveedor de servicios ferroviarios.
- `rolling_stock`: Lista con el material rodante del que posee el proveedor.

El Listado 4.15 muestra un ejemplo de cómo se definiría un proveedor de servicios ferroviarios basándose en la plantilla estructural del Listado 4.14. En el ejemplo se define el proveedor con un identificador de valor "1", cuyo nombre es "Renfe" y posee un tren con identificador "1", que corresponde al modelo S-114.

Listado 4.15: Ejemplo con datos reales de la estructura de `trainServiceProvider`

```
trainServiceProvider:
- id: '1'
  name: Renfe
  rolling_stock:
  - '1'
```

4.2.1.7. Franjas horarias - `timeSlot`

La clave raíz `timeSlot` contiene una lista de franjas horarias, que se emplearán posteriormente en la definición de los servicios ofertados. Cada franja de tiempo se construye con un identificador único para cada franja, una hora de inicio y una de finalización.

Listado 4.16: Estructura de la clave raíz `timeSlot`

```
#Lista con todas las franjas horarias
timeSlot:
- id: <Identificador de la franja horaria>
  start: <Hora de inicio de la franja>
  end: <Hora de finalización de la franja>
```

A continuación, en el Listado 4.16 se encuentra la plantilla estructural que sigue `timeSlot`. La estructura clave-valor de cada franja de tiempo dentro de la lista asociada a `timeSlot` es:

- `id`: Identificador único de cada franja de tiempo.
- `start`: Hora de comienzo de la franja de tiempo.
- `end`: Hora de finalización de la franja de tiempo.

En el Listado 4.17 se observa un ejemplo de cómo se definiría una franja de tiempo. En este ejemplo, el identificador tomaría el valor "39210", iniciando la franja a las "6:32:00" y finalizando a las "6:42:00".

Listado 4.17: Ejemplo con datos reales de la estructura de `timeSlot`

```
timeSlot:
- id: '39210'
  start: '6:32:00'
  end: '6:42:00'
```

4.2.1.8. Servicio

La clave raíz `service` contiene los datos de los diferentes servicios ofertados por los proveedores de servicios ferroviarios. Cada una de las entradas corresponde a un servicio, el cual se define mediante un identificador único, la fecha en la que se produce el servicio, la línea a la que pertenece, el proveedor que se encarga del servicio, el material rodante empleado, los diferentes trayectos efectuados en el servicio con los precios de los diferentes tipos de asientos y las restricciones que presente el servicio, siendo la restricción de capacidad la única existente actualmente.

Listado 4.18: Estructura de la clave raíz `service`

```
#Lista con la información de los servicios
service:
- id: <Identificador del servicio>
  date: <Fecha en la que se da el servicio>
  line: <Linea a la que pertenece el servicio>
  train_service_provider: <Proveedor encargado del servicio>
  time_slot: <Franja horaria de inicio del servicio>
  rolling_stock: <Tren que va a cumplir el servicio>

  #Lista con la información de los trayectos entre estaciones del servicio
  origin_destination_tuples:
  - origin: <Estación de origen>
    destination: <Estación de destino>

    #Lista con el precio de cada asiento
    seats:
    - seat: <Identificador del asiento>
      price: <Precio del asiento para el trayecto>
  capacity_constraints: <Restricción de capacidad, null en caso de no existir>
```

El Listado 4.18 muestra la plantilla estructural que sigue `service` para cada uno de los servicios. La estructura clave-valor de cada elemento dentro de la lista dependiente de `service` es:

- `id`: Identificador único del servicio ofertado.

- `date`: Fecha en la que se va a dar el servicio.
- `line`: Línea a la que pertenece el servicio.
- `train_service_provider`: Proveedor de servicios ferroviarios que ofrece el servicio.
- `time_slot`: Franja de tiempo en la que comienza el servicio.
- `rolling_stock`: Tren empleado para el servicio.
- `origin_destination_tuples`: Lista que contiene la información de los trayectos que se realizan en el servicio. Cada elemento de la lista es un diccionario que tiene las siguientes claves:
  - `origin`: Estación de origen del trayecto.
  - `destination`: Estación de destino del trayecto.
  - `seats`: Lista con los datos del precio de los asientos por tipo. Cada diccionario se define con las claves `seat`, en la que aparece el identificador del tipo de asiento, y `price`, que almacena el precio, en euros, para el tipo de asiento definido en la clave `seat` del mismo diccionario.
- `capacity_constraints`: Restricción de capacidad para ese servicio; es decir, cuántos asientos han de quedar restringidos (no disponibles para la venta) para evitar que se llene el tren en la primera estación.

En el Listado 4.19 se muestra un ejemplo de cómo se definiría un servicio. En el ejemplo aparece el servicio con el identificador "05065_02-06-2025-06.32", que se efectuará el día 2 de junio del 2025. Este servicio pertenece a la línea "05065", lo ofrece el proveedor con el identificador "1" y el tren que se va a emplear es el modelo con el identificador "1". Cuenta con un único trayecto entre las estaciones "03216" (Valencia Joaquín Sorolla) y "17000" (Madrid-Chamartín-Clara Campoamor) y, para este trayecto, cuenta con un único tipo de asiento, el que posee el identificador "1" y un precio de 45 euros.

Listado 4.19: Ejemplo con datos reales de la estructura de `service`

```yaml
service:
- id: 05065_02-06-2025-06.32
  date: '2025-06-02'
  line: '05065'
  train_service_provider: '1'
  time_slot: '39210'
  rolling_stock: '1'
  origin_destination_tuples:
  - origin: '03216'
    destination: '17000'
    seats:
    - seat: '1'
      price: 45.0
  capacity_constraints: null
```

4.2.2. Estructura de los archivos de configuración de la demanda

Los archivos de demanda recopilan la información referente a la modelización de la demanda de servicios ferroviarios. Esta información viene dada por:

- Mercado: Definen los flujos de pasajeros en los diferentes trayectos que se van a analizar.
- Modelo de patrón de usuario: Define el comportamiento de los diferentes usuarios de los

servicios ferroviarios. Cada uno de los patrones contiene una serie de características que modelan las decisiones que tomarán los usuarios, como qué tipo de billete comprarán, con cuanta antelación lo harán o la posibilidad de que el viaje se cancele, entre otros. ROBIN utiliza modelos difusos para conseguir este modelado de comportamiento.

- Modelos de patrón de demanda: Definen el volumen de pasajeros que cada uno de los mercados tendrá en función de una serie de parámetros. Estos parámetros pueden ser la demanda potencial o la distribución de los diferentes tipos de usuarios modelados.
- Día de la simulación: Corresponde con el día que se pretende simular. Éste debe coincidir con la fecha de los servicios ofertados para que los pasajeros puedan disponer de una oferta acorde a sus necesidades de día de viaje.

A continuación se detallan ejemplos de cada uno de los conceptos anteriores:

- Mercado: el mercado Madrid - Zaragoza, en el que se definirían los diferentes trayectos posibles entre estas dos ciudades.
- Modelo de patrón de usuario: Un hombre de negocios, que podría preferir que las estaciones de origen y destino se encuentren lo más cerca posible de su destino, que el tren sea puntual y que las horas de salida no estén comprendidas entre las 9 de la mañana y las 12 de la noche. También preferirá que su billete sea el billete más prémium disponible.
- Modelos de patrón de demanda: Para el mercado Madrid - Zaragoza se define la distribución de los diferentes modelos de patrón de usuario para ese mercado y la demanda potencial esperada.
- Día de la simulación: El día 2 de julio del 2025 en el que sale el servicio AVE 03206 de la estación Madrid Puerta de Atocha - Almudena Grandes, por lo que el día de la simulación corresponde con la fecha de salida de la estación.

En base a lo anterior, los archivos de demanda se componen de 4 claves raíz: `market`, `userPattern`, `demandPattern` y `day`. Cada una representa un bloque de datos para la configuración de los patrones de demanda y los patrones de usuario empleados en las simulaciones.

En los siguientes apartados se detalla la estructura de cada uno de los bloques definidos por las claves raíz mencionadas en el párrafo anterior.

4.2.2.1. Mercado

En la clave raíz `market` se encuentran los datos sobre los diferentes mercados que se emplearán posteriormente para definir los patrones de demanda que se han de seguir a la hora de realizar la simulación. Los diferentes mercados se representan mediante un identificador único, el nombre de la estación de salida, las coordenadas de la estación de salida, el nombre de la estación de llegada y sus coordenadas.

Listado 4.20: Estructura de la clave raíz `market`

```
#Lista de mercados para la simulación
market:
  - id: <Identificador del mercado>
    departure_station: <Nombre de la estación de salida>
    departure_station_coords: <Coordenadas de la estación de salida>
    arrival_station: <Nombre de la estación de llegada>
    arrival_station_coords: <Coordenadas de la estación de llegada>
```

A continuación, en el Listado 4.20 se muestra la plantilla estructural que sigue `market`. La estructura clave-valor de cada elemento dentro de la lista dependiente de `market` es:

- `id`: Identificador único que posee el mercado.
- `departure_station`: Nombre de la estación de salida.
- `departure_stations_coords`: Coordenadas de la estación de salida.
- `arrival_station`: Nombre de la estación de llegada.
- `arrival_station_coords`: Coordenadas de la estación de llegada.

El Listado 4.21 muestra un ejemplo de cómo se definiría un mercado. En este ejemplo se define un mercado que recibe el identificador "1", con origen en Madrid y destino a Zaragoza.

Listado 4.21: Ejemplo con datos reales de la estructura de `market`

```yaml
market:
  - id: 1
    departure_station: 'Madrid'
    departure_station_coords: [40.416775, -3.703790]
    arrival_station: 'Zaragoza'
    arrival_station_coords: [41.648822, -0.889085]
```

4.2.2.2. Patrón de usuario

La clave raíz `userPattern` alberga los diferentes patrones de usuarios que se emplearán en ROBIN para realizar las simulaciones. Cada patrón de usuario está definido por un identificador, el nombre para el patrón de usuario, una lista de reglas difusas utilizadas para definir la lógica difusa que se empleará en la simulación, una lista de variables, las cuales pueden ser categóricas o difusas, una serie de funciones que modelan parámetros para el patrón, la franja horaria en la que el usuario no va a empezar el trayecto, la utilidad de cada tipo de asiento para ese usuario, los proveedores de servicios ferroviarios que prefiere, la probabilidad de que el usuario cancele el billete y, por último, un umbral de utilidad para ese patrón de usuario.

A modo de curiosidad, simplemente indicar que la lógica difusa es un sistema que permite manejar información imprecisa o ambigua. A diferencia de la lógica clásica, donde las afirmaciones únicamente pueden ser verdaderas o falsas, la lógica difusa asigna un grado de pertenencia que varía entre 0 y 1 a cada afirmación. Esto permite modelar conceptos que en la vida real no tienen límites nítidos como, por ejemplo, cómodo, barato o rápido. Gracias a ello, resulta especialmente útil para representar decisiones humanas en entornos complejos. Dentro de ROBIN se utiliza para modelar los perfiles de pasajero. Esta componente va más allá de los objetivos de este TFG, solamente indicar que en las bases de datos diseñadas se han modelado los conceptos relacionados con la lógica difusa que aparecen en los archivos Yaml. Para el interés del lector se ha realizado un breve resumen de los conceptos utilizados en este TFG (Anexo C).

Listado 4.22: Estructura de la clave raíz `userPattern`

```yaml
#Lista con los patrones de usuario
userPattern:
  - id: <Identificador del patrón de usuario>
    name: <Nombre del patron de usuario>

    #Lista de reglas difusas
    rules:
      R0: <Regla difusa número 0>
      Rn: <Regla difusa número n>
```

```
#Lista de variables lingüísticas
variables:
    #Variable tipo "fuzzy"
  - name: <Nombre de la variable>
    type: fuzzy
    support: <Dominio de la variable lingüística>
    sets: [Conjunto_1, Conjunto_2, Conjunto_n]
    Conjunto_1: <Valores que definen el Conjunto_1>
    Conjunto_n: <Valores que definen el Conjunto_n>

  - name: <Nombre de la variable>
    type: categorical
    labels: [Etiqueta_1,Etiqueta_2,Etiqueta_n]
arrival_time: <Función para generar la distribución del tiempo de llegada>
arrival_time_kwargs: # Lista con los argumentos de la función de arrival_time
  arg_1: <Valor del argumento 1>
  arg_n: <Valor del argumento n>
purchase_day: <Función que genera los días de antelación de la compra del
    billete>
purchase_day_kwargs: # Argumentos para la función purchase_day
  arg_1: <Valor del argumento 1>
  arg_n: <Valor del argumento n>
forbidden_departure_hours: # Franja horaria en la que el usuario prefiere no
    empezar el viaje
  start: <Hora de inicio de la franja>
  end: <Hora de finalización de la franja>

#Lista de diccionarios para representar la utilidad de cada asiento
#para el patrón de usuario
seats:
  - id: 1
    utility: <Valor de utilidad para el asiento con id = 1>
  - id: n
    utility: <Valor de utilidad para el asiento con id = n>

#Lista de diccionarios para representar la utilidad de cada proveedor
#de servicios ferroviarios para el patrón de usuario
train_service_providers:
  - id: 1
    utility: <Valor de utilidad para el proveedor con id = 1>
  - id: 2
    utility: <Valor de utilidad para el proveedor con id = 2>
  - id: n
    utility: <Valor de utilidad para el proveedor con id = n>

early_stop: <Probabilidad de que el usuario compre un billete útil sin
    realizar una búsqueda exhaustiva>
utility_threshold: <Umbral de utilidad para el patron de usuario>
error: <Función para generar la distribución del error>
error_kwargs:
```

```
arg_1: <Valor del argumento 1>
arg_n: <Valor del argumento n>
```

En el Listado 4.22 se encuentra la estructura que sigue `userPattern` para definir cada patrón de usuario. La estructura clave-valor de cada elemento dentro de la lista asociada a `userPattern` es:

- `id`: Identificador que recibe el patrón de usuario.
- `name`: Nombre que se le da al patrón de usuario.
- `rules`: Conjunto de reglas difusas que se emplearán en las simulaciones. Están definidas mediante una clave que comienza con una letra "R" seguida del número de la regla.
- `variables`: Conjunto de variables que se utilizan en la simulación. Pueden ser variables difusas o variables categóricas. Las variables difusas se definen con la siguiente estructura:
  - `name`: Nombre que posee la variable difusa.
  - `type`: Tipo de la variable, en este caso al ser una variable difusa, el tipo será "fuzzy".
  - `support`: Dominio en el que se mueven los conjuntos que definen la variable difusa.
  - `sets`: Lista con los nombres de los conjuntos que definen la variable difusa. Cada uno de ellos se especifica debajo de la clave `sets`, empleando como clave el nombre del conjunto y como valor una lista de 4 elementos con los que se define la pertenencia. Empleando esta estructura, se pueden generar conjuntos triangulares (cuando coinciden los puntos centrales) y trapezoidales.

  En caso de que la variable sea del tipo categórico, tendría esta otra estructura:
  - `name`: Nombre que tiene la variable.
  - `type`: Tipo de la variable, en este caso, "categorical".
  - `labels`: Lista con las etiquetas que definen la variable.
- `arrival_time`: Función que se utiliza para generar la distribución del tiempo de llegada.
- `arrival_time_kwargs`: Argumentos que emplea la función definida en `arrival_time`.
- `purchase_day`: Función encargada de generar con cuántos días de antelación comprará el usuario el billete.
- `purchase_day_kwargs`: Argumentos utilizados por la función definida en `purchase_day`.
- `forbidden_departure_hours`: Franja horaria en la que el usuario prefiere no iniciar el viaje. Se emplean las claves `start` y `end` para definir la hora de inicio y finalización de la franja horaria, respectivamente.
- `seats`: Lista de diccionarios donde se define la utilidad que tiene cada tipo de asiento para el usuario. Cada diccionario se crea con las claves `id` y `utility`, para indicar el identificador del tipo de asiento y la utilidad para el usuario que tiene.
- `train_service_providers`: Lista de diccionarios que establece la utilidad de cada proveedor de servicios ferroviarios para el usuario. Cada diccionario presente en la lista se define mediante las claves `id` y `utility` e indican el identificador del proveedor y la utilidad que tiene este para el usuario, respectivamente.
- `early_stop`: Probabilidad de que el usuario adquiera un billete que le resulta útil sin realizar una búsqueda exhaustiva; es decir, sin buscar el mejor billete posible.
- `utility_threshold`: Umbral de utilidad que tiene el patrón de usuario.
- `error`: Función que se usa para calcular la distribución del error.
- `error_kwargs`: Argumentos que aplica la función de la clave `error` para funcionar.

El Listado 4.23 recoge un ejemplo en el que se ha definido un patrón de usuario basándose en la estructura presentada en el Listado 4.22. Se puede observar que las reglas poseen más variables de las que se encuentran definidas. Estas variables se han removido debido al tamaño que ocupan. Las variables que faltan son: `destination`, `departure_time`,`arrival_time`, `price` y `seat`. Este ejemplo se ha extraído del archivo "demand_data.yml" el cual se encuentra en el repositorio de GitHub. Se puede acceder a este archivo mediante el siguiente enlace[1].

Listado 4.23: Ejemplo con datos reales de la estructura de `userPattern`

```yaml
userPattern:
  - id: 1
    name: Business
    rules:
      R0: IF (seat is Premium) THEN 20.0
      R1: IF (tsp is RU1) | (tsp is RU2) THEN 20.0
      R2: IF (origin is very_near) & (destination is very_near) & (departure_time
          is in_time) & (arrival_time is in_time) THEN 60.0
    variables:
      - name: origin
        type: fuzzy
        support: [0, 100]
        sets: [very_near, mid_range, far, far_away]
        very_near: [0, 0, 10, 20]
        mid_range: [10, 20, 50, 60]
        far: [50, 60, 70, 80]
        far_away:  [70, 80, 100, 100]

      - name: tsp
        type: categorical
        labels: [RU1, RU2, RU3, RU4]
    arrival_time: norm
    arrival_time_kwargs:
      loc: 8
      scale: 1
    purchase_day: randint
    purchase_day_kwargs:
      low: 2
      high: 7
    forbidden_departure_hours:
      start: 9
      end: 24
    seats:
      - id: 1
        utility: 10
      - id: 2
        utility: 15
      - id: 3
        utility: 20
    train_service_providers:
      - id: 1
```

[1] **Enlace a "demand_data.yml":** https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos/blob/master/Archivos%20Yaml%20y%20CSV/Originales/Demanda/demand_data.yml

```
      utility: 2
  early_stop: 0.3
  utility_threshold: 50
  error: norm
  error_kwargs:
    loc: 2
    scale: 1
```

4.2.2.3. Patrón de demanda

La clave raíz `demandPattern` almacena los diferentes patrones de demanda que se emplearán en las simulaciones. Cada uno contiene: un identificador, un nombre y la lista de mercados a los que afecta el patrón de demanda con su demanda potencial y la distribución de usuarios esperada.

Listado 4.24: Estructura de la clave raíz `demandPattern`

```
#Lista de patrones de demanda
demandPattern:
  - id: <Identificador del patrón de demanda>
    name: <Nombre del patrón de demanda>
    markets: # Lista de mercados a los que afecta el patrón de demanda

      - market: <Identificador del mercado>
        potential_demand: <Función para calcular la posible demanda>
        potential_demand_kwargs: # Argumentos para la función de potential_demand
            arg_1: <Valor del argumento 1>
            arg_n: <Valor del argumento n>

        #Lista con la distribución de los patrones de usuario para el
        #patrón de demanda actual
        user_pattern_distribution:
          - id: <Identificador del patrón de usuario 1>
            percentage: <Porcentaje del tipo de usuario esperado>
```

El Listado 4.24 presenta la plantilla estructural en la que está basada `demandPattern`. La estructura clave-valor de cada elemento dentro de la lista dependiente de `demandPattern` es:

- `id`: Identificador único del patrón de demanda.
- `name`: Nombre que se le da al patrón de demanda.
- `markets`: Lista de diccionarios donde se refleja la demanda potencial y la posible distribución de los patrones de usuario. Los diccionarios de esta lista tienen la siguiente estructura:
  - `market`: Identificador del mercado del que se van a definir la demanda potencial y la distribución de los patrones de usuario.
  - `potential_demand`: Función empleada para calcular la demanda potencial del mercado.
  - `potential_demand_kwargs`: Argumentos empleados en la función definida en la clave `potential_demand`.
  - `user_pattern_distribution`: Lista de diccionarios donde se refleja la distribución de los patrones de usuario en el mercado. Cada diccionario contiene la clave. `id`, y

la clave `percentage`, que indica el porcentaje de usuarios de ese tipo esperado para el mercado.

En el Listado 4.25 se muestra un ejemplo de cómo se define un patrón de demanda. En este ejemplo, se modela el patrón de demanda con un identificador de valor "1", con el nombre de "Monday-Thursday" y afecta al mercado con el identificador "1".

Listado 4.25: Ejemplo con datos reales de la estructura de `demandPattern`

```yaml
# Demand Pattern
demandPattern:
  - id: 1
    name: Monday-Thursday
    markets:
      # Madrid - Zaragoza
      - market: 1
        potential_demand: randint
        potential_demand_kwargs:
          low: 1000
          high: 1800
        user_pattern_distribution:
          - id: 1 # Business
            percentage: 0.2
          - id: 2 # Student
            percentage: 0.25
          - id: 3 # Tourist
            percentage: 0.35
          - id: 4 # EventTourist
            percentage: 0.1
          - id: 5 # Adventurer
            percentage: 0.1
```

4.2.2.4. Día

La clave raíz `day` alberga los datos correspondientes al día simulado. Este se define mediante un identificador, la fecha y el patrón de demanda empleado en la simulación.

Listado 4.26: Estructura de la clave raíz `day`

```yaml
day:
  - id: <Identificador del día>
    date: <Fecha de la simulación>
    demandPattern: <Patrón de demanda empleado para el día simulado>
```

En el Listado 4.26 se presenta la estructura seguida para la creación de `day`. La estructura clave-valor de `day` es:

- `id`: Identificador del día.
- `date`: Fecha del día simulado.
- `demandPattern`: Patrón de demanda asociado al día simulado.

El Listado 4.7 muestra un ejemplo de cómo se definiría un día dentro del archivo de configuración de la demanda. Este día tomaría el valor "1" como identificador, la fecha sería el 25

id	42	user_pattern	Tourist
departure_station	60000	arrival_station	04040
arrival_day	2023-09-06	arrival_time	9.7311
purchase_day	14	service	03203_06-09-2023-20.40
service_departure_time	20.6666	service_arrival_time	23.9166
seat	Básico	price	36.8
utility	51.1333	best_service	03203_06-09-2023-20.40
best_seat	Básico	best_utility	51.1.333

Tabla 4.5: Tabla con datos reales de un archivo de resultados

de junio del año 2024 y su patrón de demanda sería el que tiene asignado el identificador con valor "1".

Listado 4.27: Ejemplo con datos reales de la estructura de `day`

```
day:
  - id: 1
    date: 2024-06-25
    demandPattern: 1
```

4.2.3. Estructura de los archivos de resultados

Los archivos donde se almacenan los resultados de la simulación son archivos CSV. Los resultados de la simulación para cada pasajero se almacenan en una fila del archivo CSV que incluye las siguientes columnas:

- id: Identificador único que se le da a cada pasajero.
- user_pattern: Patrón de usuario para el que se ha obtenido el resultado de la fila.
- departure_station: Estación donde se inicia del viaje.
- arrival_station: Estación donde finaliza el trayecto.
- arrival_day: Día de llegada a la estación de destino.
- arrival_time: Hora de llegada al destino.
- purchase_day: Día en el que se compró el billete.
- service: Identificador del servicio adquirido por el pasajero.
- service_departure_time: Hora de salida prevista para el servicio.
- service_arrival_time: Hora de llegada prevista para el servicio.
- seat: Asiento elegido por el usuario.
- price: Precio al que se ha comprado el billete.
- utility: Utilidad, en porcentaje, del billete que adquiere el pasajero.
- best_service: Servicio que mejor se adapta a las exigencias del pasajero, y que podría ser diferente al que finalmente adquiere (debido, por ejemplo, a que éste ya estuviera agotado).
- best_seat: Asiento que mejor se adapta a los requerimientos del pasajero.
- best_utility: utilidad del mejor billete para el pasajero simulado.

La Tabla 4.5 muestra un ejemplo de los valores de una fila del archivo de resultados.

4.3. Diseño e implementación de las bases de datos

En base a la información a modelar se ha decidido utilizar tres bases de datos, una por cada archivo del simulador ROBIN, es decir, una para los datos de los archivos que definen la oferta del día que se va a simular, otra para los datos de la configuración de la demanda esperada para ese día y, por último, una que recoge los resultados de las simulaciones realizadas.

Dado que estas tres bases de datos comparten una estructura similar, se va a proceder a detallar el esquema común utilizando nombre de claves genéricos que indican el nivel dónde se encuentra la clave (raíz 1, raíz 2, raíz 2.2, etc.) (Figura 4.1). Como se puede observar, los datos que referencian la clave raíz 1 se almacenan únicamente en una tabla, mientras que los datos de la clave raíz 2, se almacenan en dos tablas, donde una almacenan los datos que referencian las claves 2.1, 2.2, 2.3, 2.4, 2.5 y 2.6, mientras que en la tabla de datos auxiliar se almacenan los datos que asociados a la clave 2.7.

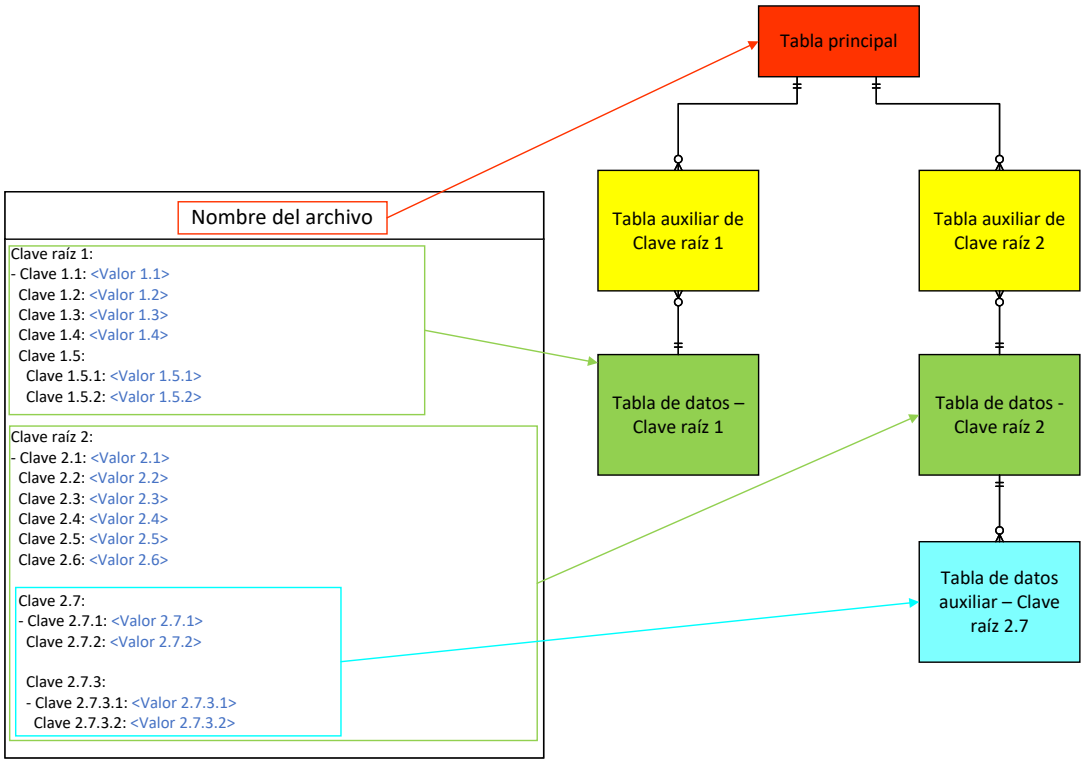


Figura 4.1: Diagrama de la estructura de las bases de datos. En naranja aparece la tabla principal, en amarillo las tablas auxiliares, en verde las tablas de datos y en azul las tablas de datos auxiliares.

La tabla principal (en naranja en la figura) almacena el nombre del archivo y las observaciones pertinentes. El objetivo de estas tablas consiste en tener un punto de partida para la reconstrucción de los archivos empleando los datos almacenados en la base de datos. Esto se lleva a cabo mediante el uso de los identificadores que se asignan a cada uno de los archivos y unas tablas auxiliares que conectan la información contenida en las demás tablas de la base de datos con la tabla principal. La tabla principal tiene las siguientes columnas:

1. La primera es para el identificador que se le asigna a cada fila. Este proceso se realiza de forma automática.
2. Otra que almacena el nombre del archivo sin la extensión del mismo.
3. Finalmente, una columna para guardar las observaciones realizadas para cada archivo, en caso de que existieran.

De esta tabla dependen varias tablas auxiliares, cuyo cometido, como ya se ha mencionado anteriormente, es el de relacionar los archivos con la información perteneciente a dichos archivos, asociando el identificador de la entrada que pertenece a cada archivo con los identificadores de la información perteneciente a dicho archivo.

Por último, se cuenta con las tablas destinadas a albergar los datos. Estas tablas contienen una estructura similar a la que se emplea en los archivos. En el caso de los archivos Yaml, existen tablas que no dependan de otras, que vendrán definidas por las claves raíz dentro de los archivos.

Además, existen unas tablas de datos auxiliares, que dependen de las tablas de datos, en las que se almacenan datos de estructuras más complejas que puedan aparecer en los archivos Yaml, como un diccionario que tenga otro diccionario anidado o una lista de diccionarios. Esta estructura ayuda a que las sentencias SQL sean más sencillas.

En resumen, se ha diseñado como un acceso tipo jerárquico a tres niveles, se puede decir, como "en forma de árbol", contando con una tabla de datos auxiliar final, en caso de ser necesario.

A continuación, se mostrará un ejemplo de algunas tablas de la estructura de la Figura 4.1.

En la Figura 4.2 se pueden ver las columnas de todas las tablas principales. Esta tabla contiene las columnas `ID`, en la que aparece un identificador único, generado automáticamente para cada una de las filas, que actúa como clave primaria, `NAME`, que almacena el nombre del archivo y `OBSERVATIONS`, en el que se guardan las observaciones para el archivo.

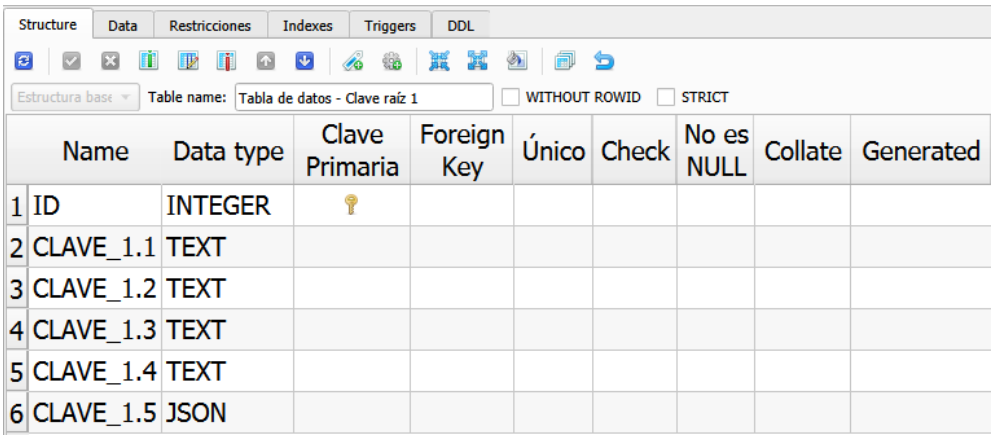
Structure | Data | Restricciones | Indexes | Triggers | DDL

Estructura base | Table name: Tabla principal | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER	🔑						
2	NAME	TEXT							
3	OBSERVATIONS	TEXT							

Figura 4.2: Columnas de la tabla principal.

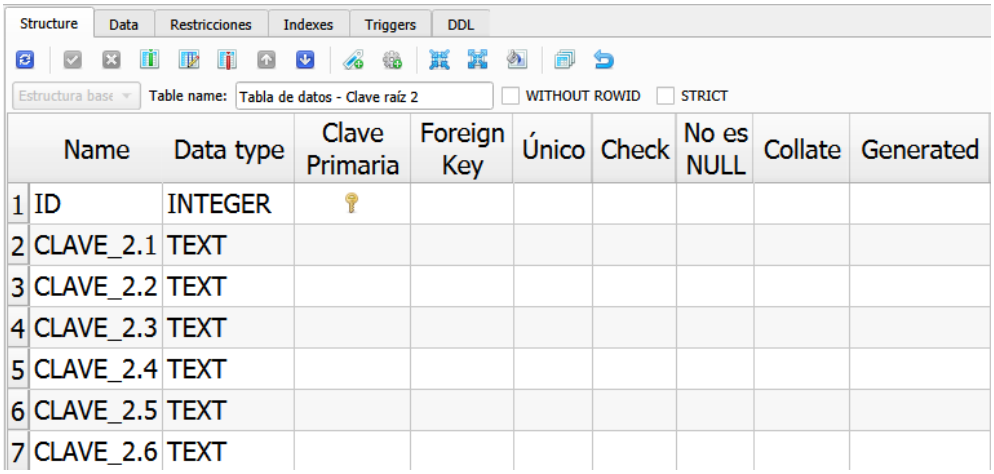
La Figura 4.3 muestra las columnas de la tabla destinada a almacenar los datos a los que se hace referencia en la clave raíz 1 que aparecen en la Figura 4.1. Esta cuenta con una columna `ID` que es una clave primaria con un valor numérico generado de manera autoincremental y las demás columnas contienen los datos asociados a cada una de las claves dentro de la clave raíz 1. Para el caso de la clave 1.5, que hace referencia a un diccionario, se almacena en la columna con el nombre de `CLAVE_1.5` y tipo de datos diccionario (*JSON*).



	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	CLAVE_1.1	TEXT							
3	CLAVE_1.2	TEXT							
4	CLAVE_1.3	TEXT							
5	CLAVE_1.4	TEXT							
6	CLAVE_1.5	JSON							

Figura 4.3: Columnas de la tabla de datos de la clave raíz 1.

La Figura 4.4 muestra los datos asociados a la clave raíz 2 que aparecen en la Figura 4.1. Cuenta con una columna `ID` que almacena el identificador autoincremental asignado a cada fila de forma automática, mientras que las demás columnas contienen los datos asociados a las claves bajo la clave raíz 2, a excepción de los datos referenciados por la clave 2.7 que se almacenan en una tabla distinta, pero relacionada con esta mediante la columna `ID`.



	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	CLAVE_2.1	TEXT							
3	CLAVE_2.2	TEXT							
4	CLAVE_2.3	TEXT							
5	CLAVE_2.4	TEXT							
6	CLAVE_2.5	TEXT							
7	CLAVE_2.6	TEXT							

Figura 4.4: Columnas de la tabla de datos de la clave raíz 2.

En la Figura 4.5 se muestran las columnas de la tabla auxiliar de datos de la clave raíz 2, destinada a contener los datos asociados a la clave 2.7. Esta tabla tiene definida una relación con la tabla de datos de la clave raíz 2 (Figura 4.4) mediante la clave foránea que se almacena en la columna `TABLA_CLAVE_RAIZ_2_ID` y que contiene el identificador de la tabla de datos de la clave raíz 2 con la que los datos de la fila de la tabla de datos auxiliar guardan relación.

Structure | Data | Restricciones | Indexes | Triggers | DDL

Estructura base | Table name: Tabla de datos auxiliar - Clave raíz 2.7 | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	TABLA_CLAVE_RAIZ_2_ID								
3	CLAVE_2.7.1	TEXT							
4	CLAVE_2.7.2	TEXT							
5	CLAVE_2.7.3	JSON							

Figura 4.5: Columnas de la tabla de datos auxiliar de la clave raíz 2 con los datos asociados a la clave 2.7.

En las Figuras 4.6 y 4.7 se pueden ver las tablas auxiliares destinadas a relacionar los datos de las tablas de datos con la tabla principal. Para el caso de la tabla auxiliar de la clave raíz 1 (Figura 4.6), relaciona la tabla principal con la tabla de datos de la clave raíz 1, mediante una relación establecida utilizando los identificadores de ambas tablas. Esta relación se materializa a través de las claves foráneas almacenadas en las columnas `TABLA_PRINCIPAL_ID`, para el identificador de la tabla principal y `TABLA_CLAVE_RAIZ_1_ID`, para el identificador de la tabla de datos de la clave raíz 1.

Structure | Data | Restricciones | Indexes | Triggers | DDL

Estructura base | Table name: Tabla auxiliar de Clave raíz 1 | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	TABLA_PRINCIPAL_ID	INTEGER							
3	TABLA_CLAVE_RAIZ_1_ID	INTEGER							

Figura 4.6: Columnas de la tabla auxiliar de la clave raíz 1.

La tabla auxiliar de la clave raíz 2 (Figura 4.7) funciona de manera similar a la anterior, pero relaciona los datos de la tabla principal con los datos de la tabla de datos de la clave raíz 2, y a su vez, con la tabla auxiliar de datos de la clave raíz 2 debido a que las dos tablas de datos están relacionadas entre sí. La relación entre la tabla principal y la tabla de datos de la clave raíz 2 se lleva a cabo mediante claves foráneas almacenadas en las columnas `TABLA_PRINCIPAL_ID` y `TABLA_CLAVE_RAIZ_1_ID`, respectivamente.

Structure | Data | Restricciones | Indexes | Triggers | DDL

Estructura base | Table name: Tabla auxiliar de Clave raíz 2 | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	TABLA_PRINCIPAL_ID	INTEGER							
3	TABLA_CLAVE_RAIZ_2_ID	INTEGER							

Figura 4.7: Columnas de la tabla auxiliar de la clave raíz 2.

Todas las figuras de las tablas pertenecientes a las bases de datos han sido capturadas del programa SQLiteStudio [16]. Este programa se ha empleado en el diseño, visualización y realización de las pruebas pertinentes en las diferentes bases de datos empleadas en este TFG.

Finalmente, indicar que en el caso de los archivos de resultados, la tabla de datos tendrá las mismas columnas que los archivos CSV que se generan al acabar la simulación con ROBIN.

4.3.1. Base de datos para los archivos de entrada de datos de la oferta

La base de datos que contiene los datos de la configuración de la oferta consta de 21 tablas, cuya organización aparece en el Esquema de diseño relacional (EDR) mostrado en la Figura 4.8). Una versión más extendida de dicho EDR, que contiene también los nombres de las columnas de cada una de las tablas, se encuentra en el Anexo A.1. Esta base de datos almacena los datos relacionados con la oferta de servicios ferroviarios correspondientes a un rango de tiempo determinado, que puede abarcar desde un día hasta varios. Estos datos se emplean en el simulador ROBIN para definir la oferta disponible en base al archivo de configuración de la oferta.

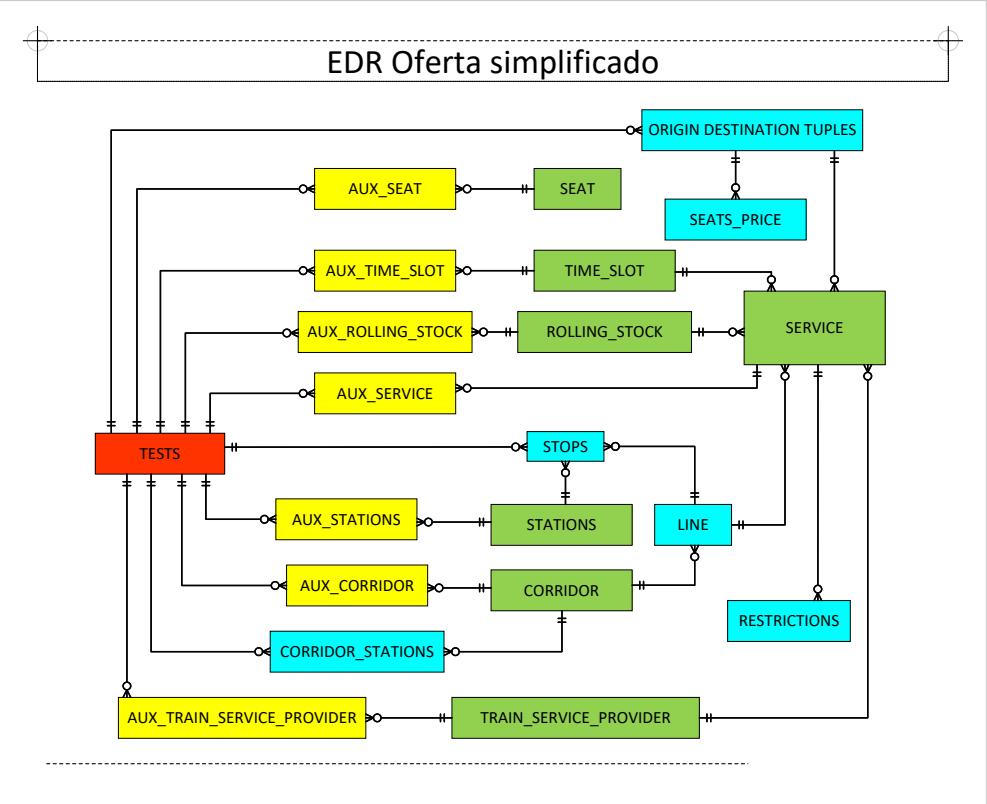


Figura 4.8: Esquema de la base de datos para archivos de entrada de datos de la oferta. En naranja aparece la tabla principal, en amarillo las tablas auxiliares, en verde las tablas de datos y en azul las tablas de datos auxiliares.

La tabla principal de la base de datos aparece representada con el color naranja y tiene la finalidad de almacenar el nombre de los archivos Yaml de entrada de datos de la oferta importados a la base de datos y las observaciones que el usuario estime pertinentes.

Las tablas auxiliares aparecen en color amarillo. Estas tablas relacionan los datos almacenados en las tablas de la base de datos con el archivo de oferta al que pertenecen mediante los identificadores que poseen. Esto evita elementos repetidos en las tablas de datos, optimizando el espacio que ocupan los datos dentro de la base de datos. De esta manera, si una clave raíz tiene asociados los mismos datos en dos archivos diferentes, estos datos solo aparecen reflejados una vez en las tablas de datos y, empleando la tabla auxiliar correspondiente a esta clave raíz, se asocia el dato común a ambos archivos. Por ejemplo, si en dos archivos Yaml diferentes, bajo la clave raíz que referencia a las estaciones aparece la estación de Madrid Puerta de Atocha - Almudena Grandes, en la tabla de datos dedicada a almacenar los datos de las estaciones aparece una única vez, mientras que en la tabla auxiliar aparece el identificador perteneciente a la estación mencionada dos veces, debido a que se encuentra asociado a dos archivos diferentes. Además, estas tablas auxiliares son la base desde la que se reconstruyen los archivos a la hora

de exportarlos, ya que relacionan toda la información de las tablas de datos con el archivo Yaml del que provienen.

Las tablas de datos, de color verde, tienen la función de albergar los datos asociados a las diferentes claves raíz del archivo Yaml de entrada de datos de la oferta, por ejemplo, los datos de las estaciones, las líneas de tren o los servicios ferroviarios, entre otros. Si bajo alguna de estas claves raíz se encuentra referenciada una estructura más compleja (por ejemplo, la clave `origin_destination_tuples` asociada a la clave raíz `service`) se almacena en las tablas de datos auxiliares, que en el diagrama de la Figura 4.8 aparecen de color azul claro. Esto facilita la creación de sentencias SQL, dado que no hay que trabajar con texto en formato JSON. Otras estructuras de datos más simples como listas o diccionarios se han introducido como texto en formato JSON en las tablas de datos, como por ejemplo, en la tabla de datos de las estaciones, en la que aparecen las coordenadas de las estaciones como un diccionario con las claves `latitude` y `longitude`. En este caso, es relativamente sencillo obtener los datos de este diccionario ya que las claves son conocidas y bajo estas claves no se referencia ninguna otra estructura de datos, sino valores numéricos. Un ejemplo de cómo extraer los datos de estas columnas aparecerá más adelante cuando se explique la estructura de la tabla de datos de las estaciones.

A continuación se muestra cada una de las tablas que componen esta base de datos.

4.3.1.1. Tabla principal

La tabla principal de la base de datos es la tabla `TESTS` (Figura 4.9). En dicha tabla, se almacenan el nombre del archivo introducido en la base de datos y las posibles observaciones que el usuario estime pertinentes. Además, a cada archivo introducido se le añade un identificador único, que es la clave primaria de la tabla y que se emplea como clave foránea o externa en otras tablas para establecer las relaciones con la tabla principal.

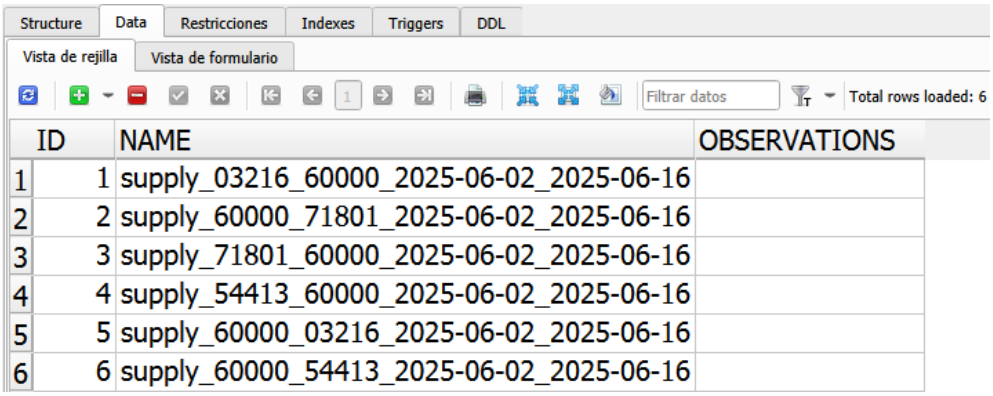
Structure | Data | Restricciones | Indexes | Triggers | DDL

supplyDb - Opt | Table name: TESTS | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	NAME	TEXT							
3	OBSERVATIONS	TEXT							

Figura 4.9: Tabla principal de la base de datos para archivos de oferta.

En la Figura 4.10 se pueden observar cómo están almacenados los datos en la tabla de la Figura 4.9. La información que se puede ver en la Figura 4.10 se corresponde con la de los archivos Yaml de configuración de la oferta, en este caso, el nombre del archivo empleado.


Structure | Data | Restricciones | Indexes | Triggers | DDL

Vista de rejilla | Vista de formulario

Filtrar datos | Total rows loaded: 6

	ID	NAME	OBSERVATIONS
1	1	supply_03216_60000_2025-06-02_2025-06-16	
2	2	supply_60000_71801_2025-06-02_2025-06-16	
3	3	supply_71801_60000_2025-06-02_2025-06-16	
4	4	supply_54413_60000_2025-06-02_2025-06-16	
5	5	supply_60000_03216_2025-06-02_2025-06-16	
6	6	supply_60000_54413_2025-06-02_2025-06-16	

Figura 4.10: Ejemplo con datos reales de la tabla 4.9.

4.3.1.2. Tablas de datos

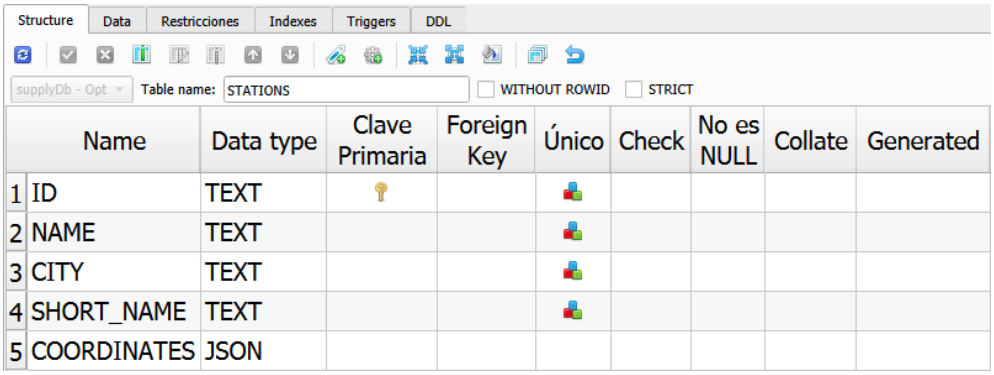
Esta base de datos incluye varias tablas destinadas a almacenar la información proveniente de los archivos de configuración de la oferta. Dichos archivos contienen información relacionada con, por ejemplo, las estaciones que se encuentran en el corredor y las líneas que aparecen en los servicios ofertados por las compañías proveedoras de servicios ferroviarios.

La tabla `STATIONS` (Figura 4.11) contiene los datos relacionados con las estaciones que aparecen en las diferentes líneas de los servicios ferroviarios.

Los campos de esta tabla son:

- `ID`: Identificador único de la estación.
- `NAME`: Nombre de la estación.
- `CITY`: Ciudad donde está ubicada la estación.
- `SHORT_NAME`: Nombre corto de la estación.
- `COORDINATES`: Coordenadas geográficas de la estación.

La mayoría de las columnas de la tabla contienen datos en formato de texto plano, excepto en el caso de la columna `COORDINATES`, que utiliza un formato JavaScript Object Notation (JSON) para almacenar la latitud y la longitud en un único campo.


Structure | Data | Restricciones | Indexes | Triggers | DDL

supplyDb - Opt | Table name: STATIONS | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	TEXT							
2	NAME	TEXT							
3	CITY	TEXT							
4	SHORT_NAME	TEXT							
5	COORDINATES	JSON							

Figura 4.11: Tabla `STATIONS`

A continuación, se muestra cómo extraer los datos de las coordenadas de las estaciones empleando una sentencia SQL. El Listado 4.28 detalla cómo usar el comando `json_extract` para obtener la latitud y la longitud de la columna `COORDINATES` de la tabla `STATIONS` que tiene asignado el tipo JSON como tipo de datos. En este caso, esa columna almacena un diccionario con las claves `latitude` y `longitude` en las que se almacenan los valores de la latitud y longitud,

respectivamente, a las que se encuentra la estación.

Listado 4.28: Sentencia SQL para extraer los datos de las estaciones dentro de la tabla STATIONS

```
SELECT
    STATIONS.ID,
    STATIONS.NAME,
    STATIONS.CITY,
    STATIONS.SHORT_NAME,
    json_extract(STATIONS.COORDINATES,'$.latitude') as LATITUD,
    json_extract(STATIONS.COORDINATES,'$.longitude') as LONGITUD

FROM STATIONS
```

Como el listado de estaciones es extenso, se ha tomado una muestra con las 5 primeras filas del archivo CSV generado a partir de la sentencia del Listado 4.28. Esta muestra se encuentra en la tabla 4.6. El archivo CSV que contiene todas las entradas generadas mediante la sentencia SQL se puede encontrar en el siguiente enlace[2].

	ID	NAME	CITY	SHORT_NAME	LATITUD	LONGITUD
1	60000	MADRID PTA. ATOCHA ALMUDENA GRANDES	MADRID	MADRI	40.406442	3.690886
2	04007	GUADALAJARA YEBES	GUADALAJARA	04007	40.587315	3.124301
3	70600	CALATAYUD	CALATAYUD	70600	41.346692	1.638680
4	04040	ZARAGOZA-DELICIAS	ZARAGOZA	ZARAG	41.658649	0.911615
5	78400	LLEIDA-PIRINEUS	LLEIDA	78400	41.620696	0.632669

Tabla 4.6: Primeras cinco filas de la salida de la sentencia SQL del Listado 4.28

Los datos de los diferentes proveedores de servicios ferroviarios que puedan existir se registran en la Tabla TRAIN_SERVICE_PROVIDER (Figura 4.12). Los campos con los que cuenta esta tabla son:

- ID: Identificador autoincremental único de cada fila.
- ID_ON_FILE: Identificador que aparece en el archivo para cada proveedor de servicios ferroviarios.
- NAME: Nombre del proveedor.
- ROLLING_STOCK: Lista de modelos de tren disponibles para el proveedor.

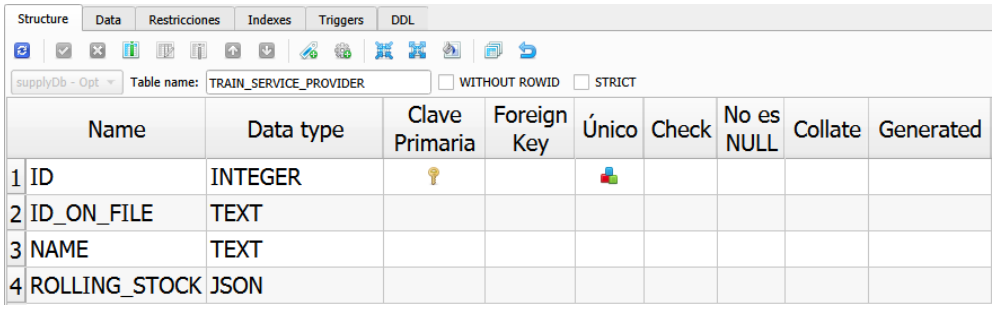
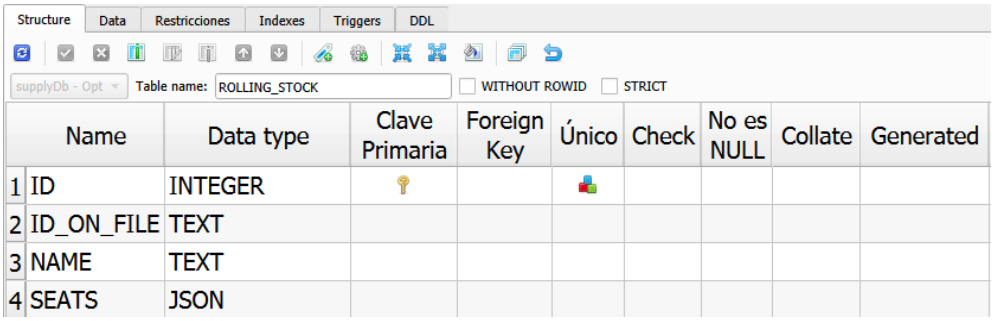


Figura 4.12: Tabla TRAIN_SERVICE_PROVIDER

[2] https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos/blob/master/Archivos%20Yaml%20y%20CSV/Exportados/Vistas%20exportadas/Extraer%20info%20de%20estaciones.csv

Para almacenar las características de los trenes de los que disponen las diferentes compañías de servicios ferroviarios para la realización de los servicios ofertados, se emplea la tabla `ROLLING_STOCK` (Figura 4.13), la cual cuenta con los campos:

- `ID`: Identificador autoincremental único de cada fila.
- `ID_ON_FILE`: Identificador que aparece en el archivo para cada material rodante.
- `NAME`: Nombre asignado al modelo de tren.
- `SEATS`: Diccionario en formato JSON que almacena los diferentes tipos de asientos disponibles en el modelo de tren.
  - La clave del diccionario representa el tipo de asiento.
  - El valor asociado a cada clave indica la cantidad de asientos disponibles de ese tipo.



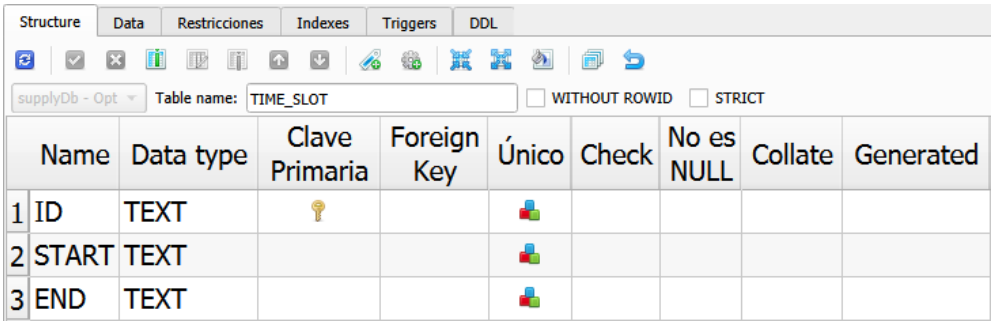
	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	ID_ON_FILE	TEXT							
3	NAME	TEXT							
4	SEATS	JSON							

Figura 4.13: Tabla `ROLLING_STOCK`

En la tabla `TIME_SLOT` (Figura 4.14) se almacenan los intervalos de tiempo de llegada y salida de los trenes de la estación.

Esta tabla cuenta con tres campos:

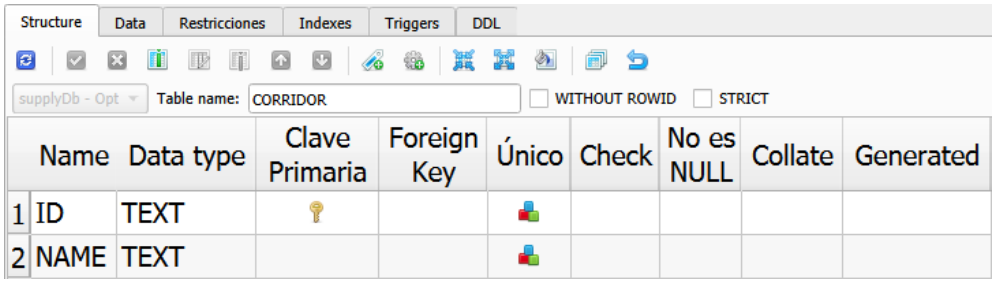
- `ID`: Identificador único del intervalo.
- `START`: Hora de inicio del intervalo.
- `END`: Hora de finalización del intervalo.



	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	TEXT							
2	START	TEXT							
3	END	TEXT							

Figura 4.14: Tabla `TIME_SLOT`

La tabla `CORRIDOR` (Figura 4.15) almacena el identificador del corredor (`ID`) y el nombre que recibe el mismo (`NAME`), siendo los formatos de ambos campos texto plano.

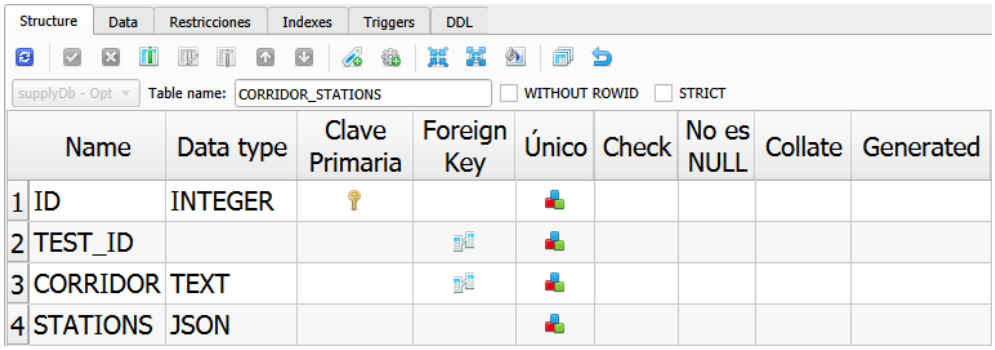


	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	TEXT							
2	NAME	TEXT							

Figura 4.15: Tabla `CORRIDOR`

Esta tabla está relacionada con `CORRIDOR_STATIONS` (Figura 4.16), la cual almacena las estaciones presentes en el ramal del corredor (`STATIONS`), siendo el primer elemento de las listas la estación de inicio del ramal y el último elemento la estación donde finaliza el ramal del corredor. El campo `STATIONS` almacena los datos en formato JSON.

`CORRIDOR_STATIONS` se relaciona con las tablas `TESTS` y `CORRIDOR` mediante el empleo de claves foráneas en las columnas `TEST_ID` y `CORRIDOR_ID` respectivamente, cuyos datos se almacenan en formato de texto plano.



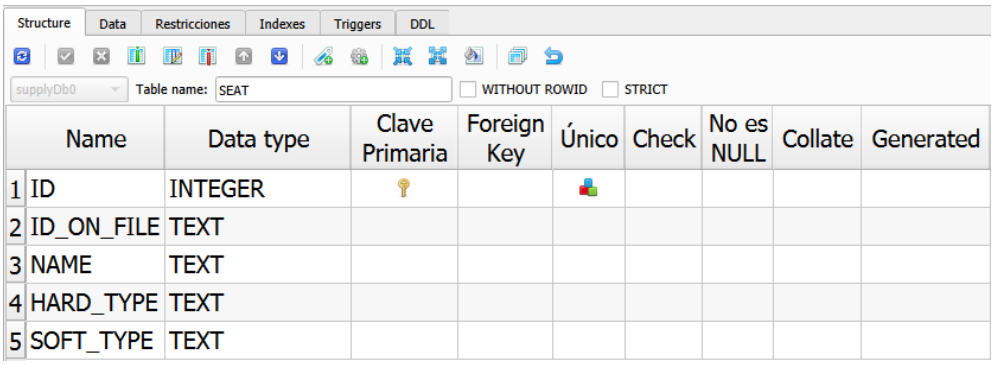
	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	TEST_ID								
3	CORRIDOR	TEXT							
4	STATIONS	JSON							

Figura 4.16: Tabla `CORRIDOR_STATIONS`

Los datos de los asientos que se ofertan por parte de las compañías de servicios ferroviarios se encuentran almacenados en la tabla `SEAT` (Figura 4.17).

Esta tabla está compuesta de los siguientes campos:

- `ID`: Identificador autoincremental único de cada fila.
- `ID_ON_FILE`: Identificador que aparece en el archivo para cada tipo de asiento.
- `NAME`: Nombre asignado al asiento.
- `HARD_TYPE`: Tipo de asiento según sus características físicas.
- `SOFT_TYPE`: Tipo de servicios asociados al asiento.



	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	ID_ON_FILE	TEXT							
3	NAME	TEXT							
4	HARD_TYPE	TEXT							
5	SOFT_TYPE	TEXT							

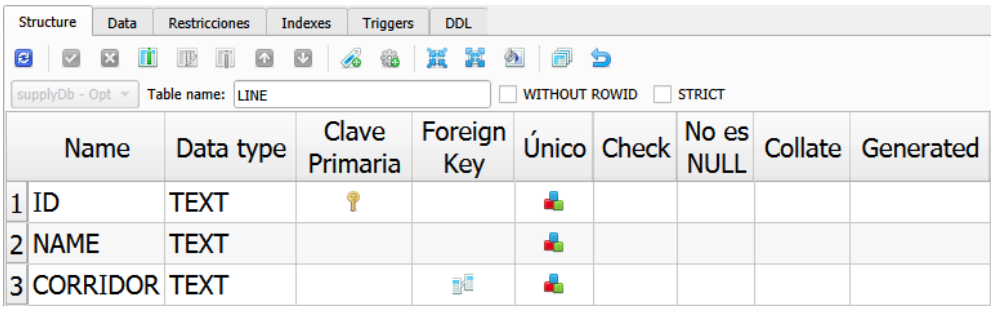
Figura 4.17: Tabla `SEAT`

La tabla `LINE` (Figura 4.18) contiene los datos de las diferentes líneas que componen los corredores por donde circulan los trenes.

Esta tabla contiene las columnas:

- `ID`: Identificador único de la línea.
- `NAME`: Nombre asignado a la línea.
- `CORRIDOR`: Clave foránea que referencia el identificador del corredor al que pertenece la línea.

Esta clave foránea relaciona la tabla `LINE` con la tabla `CORRIDOR` (Figura 4.15).



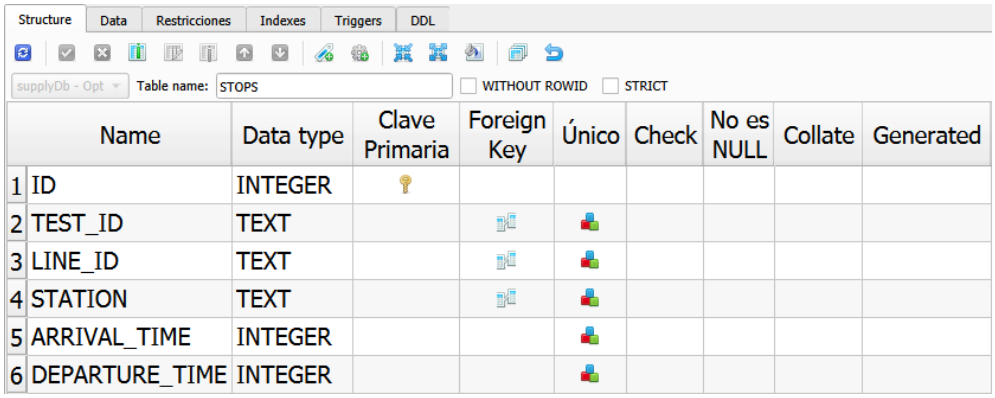
	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	TEXT							
2	NAME	TEXT							
3	CORRIDOR	TEXT							

Figura 4.18: Tabla `LINE`

En la tabla `STOPS` (Figura 4.19) se almacenan las paradas que se realizan en las diferentes líneas ferroviarias.

Esta tabla consta de los siguientes campos:

- `ID`: Identificador único de la parada.
- `TESTS_ID`: Clave foránea que referencia el identificador almacenado en la tabla `TESTS` (Figura 4.9).
- `LINE_ID`: Clave foránea que referencia el identificador de la línea almacenado en la tabla `LINE` (Figura 4.18).
- `STATION`: Clave foránea que referencia el identificador de la estación almacenado en la tabla `STATIONS` (Figura 4.11).
- `ARRIVAL_TIME`: Hora de llegada a la estación.
- `DEPARTURE_TIME`: Hora de salida de la estación.



	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	TEST_ID	TEXT							
3	LINE_ID	TEXT							
4	STATION	TEXT							
5	ARRIVAL_TIME	INTEGER							
6	DEPARTURE_TIME	INTEGER							

Figura 4.19: Tabla `STOPS`

A continuación, se van a usar las tablas `LINE` 4.18 y `STOPS` para realizar un ejemplo de cómo usar las relaciones que existen entre ambas para extraer datos en base a la intersección de ambas tablas usando el comando SQL `INNER JOIN`. En este caso, la sentencia que aparece en el Listado 4.29 seleccionará las paradas que se efectúan en la línea 05065 de entre todas las que existen en la tabla `STOPS`

Listado 4.29: Sentencia para el ejemplo de uso de tablas auxiliares

```
SELECT
    STOPS.ID,
    STOPS.LINE_ID AS LINE,
    STOPS.STATION,
    STOPS.ARRIVAL_TIME,
    STOPS.DEPARTURE_TIME
FROM STOPS

INNER JOIN
    LINE
ON
    LINE.ID = STOPS.LINE_ID

WHERE LINE.ID = '05065'
```

La tabla 4.7 contiene los datos seleccionados por la sentencia SQL del Listado 4.29.

	ID	LINE	STATION	ARRIVAL_TIME	DEPARTURE_TIME
1	1	05065	03216	0	0
2	2	05065	03213	25	27
3	3	05065	03208	61	63
4	4	05065	17000	126	126

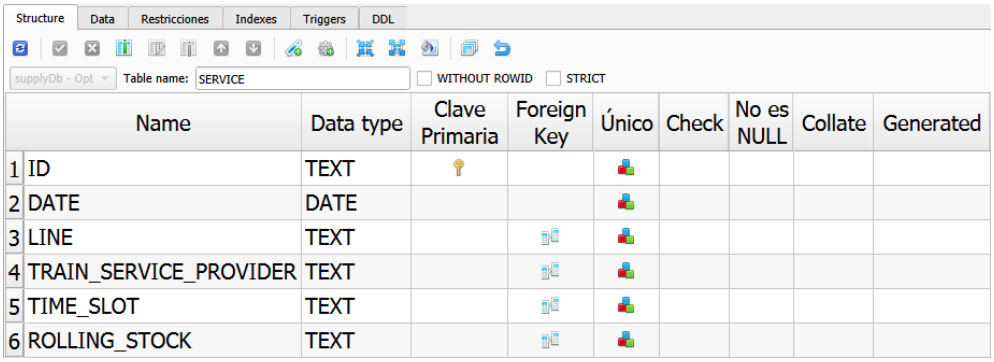
Tabla 4.7: Paradas de la línea 05065

Los datos de los diferentes servicios que se ofertan a lo largo del día se almacenan en la tabla `SERVICE` (Figura 4.20).

Estos datos se componen de:

- `ID`: Identificador único del servicio.
- `DATE`: Fecha en la que se realiza el servicio.

- `LINE`: Identificador de la línea en la que se lleva a cabo el servicio.
- `TIME_SLOT`: Identificador del intervalo de tiempo asignado al servicio.
- `TRAIN_SERVICE_PROVIDER`: Identificador de la compañía que ofrece el servicio.
- `ROLLING_STOCK`: Identificador de la máquina que realiza el servicio.


Structure | Data | Restricciones | Indexes | Triggers | DDL

supplyDb - Opt | Table name: SERVICE | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	TEXT							
2	DATE	DATE							
3	LINE	TEXT							
4	TRAIN_SERVICE_PROVIDER	TEXT							
5	TIME_SLOT	TEXT							
6	ROLLING_STOCK	TEXT							

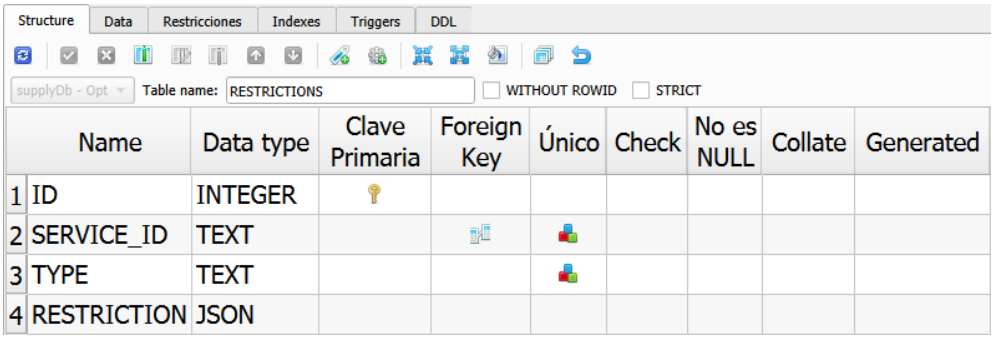
Figura 4.20: Tabla `SERVICE`

La tabla `RESTRICTIONS` (Figura 4.21) contiene las restricciones que tiene cada servicio, en este caso, restricciones de capacidad, ya que el simulador ROBIN de momento solo acepta este tipo de restricciones. Tanto la tabla `RESTRICTIONS`, como el programa desarrollado, están preparados para aceptar otro tipo de restricciones.

Esta tabla cuenta con las columnas:

- `ID`: Identificador auto-incremental de la restricción.
- `SERVICE_ID`: Identificador del servicio al que esta asociada la restricción.
- `TYPE`: Tipo de restricción aplicada.
- `RESTRICTION`: Valor o conjunto de valores que definen la restricción.

Esta tabla está relacionada con la tabla `SERVICE` (Figura 4.20), mediante el uso de la clave foránea `SERVICE_ID`.


Structure | Data | Restricciones | Indexes | Triggers | DDL

supplyDb - Opt | Table name: RESTRICTIONS | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	SERVICE_ID	TEXT							
3	TYPE	TEXT							
4	RESTRICTION	JSON							

Figura 4.21: Tabla `RESTRICTIONS`

En la tabla `ORIGIN_DESTINATION_TUPLES` se almacenan los datos de todos los trayectos ofrecidos en un servicio.

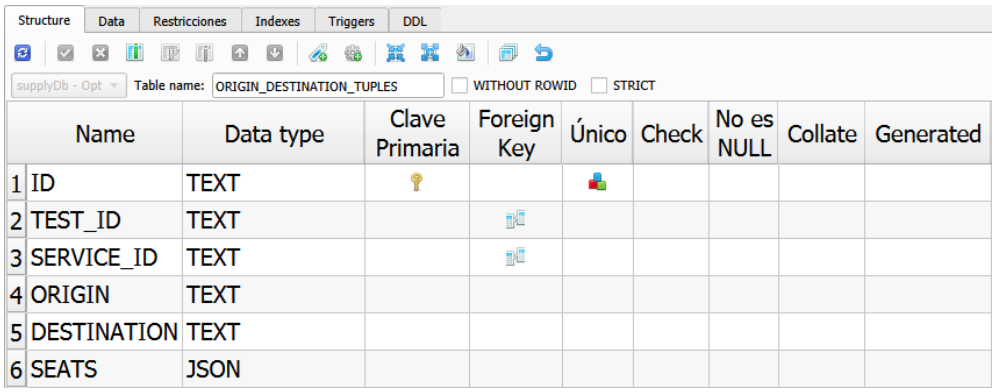
Esta tabla contiene los siguientes campos:

- `ID`: Identificador único de la tupla origen-destino.
- `TEST_ID`: Clave foránea que referencia el identificador del archivo al que pertenece el servicio asociado a la tupla.
- `SERVICE_ID`: Clave foránea que indica el identificador del servicio al que pertenece la

tupla origen-destino.

- `ORIGIN`: Identificador de la estación de origen.
- `DESTINATION`: Identificador de la estación de destino.
- `SEATS`: Lista en formato JSON que almacena los tipos de asientos para el trayecto entre origen y destino.

Las claves foráneas `TEST_ID` y `SERVICE_ID` establecen relaciones con las tablas `TESTS` (Figura 4.9) y `SERVICE` (Figura 4.20), respectivamente.



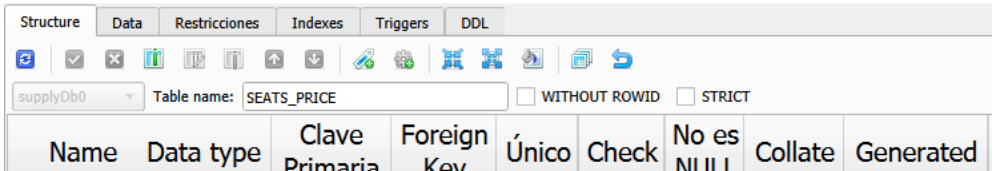
	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	TEXT							
2	TEST_ID	TEXT							
3	SERVICE_ID	TEXT							
4	ORIGIN	TEXT							
5	DESTINATION	TEXT							
6	SEATS	JSON							

Figura 4.22: Tabla `ORIGIN_DESTINATION_TUPLES`

Los precios de los asientos ofertados en los diferentes servicios se almacenan en la tabla `SEATS_PRICE` (Figura 4.23). Esta tabla contiene los siguientes campos:

- `ID`: Identificador único de cada asiento.
- `ODT_ID`: Clave foránea que referencia el identificador de la tupla origen-destino a la que pertenece el asiento.
- `SEAT`: Identificador del tipo de asiento.
- `PRICE`: Precio asignado al asiento para un trayecto determinado por `ODT_ID`.

La tabla `SEATS_PRICE` se relaciona con `ORIGIN_DESTINATION_TUPLES` mediante la clave foránea `ODT_ID`.



	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	ODT_ID	TEXT							
3	SEAT	TEXT							
4	PRICE	REAL							

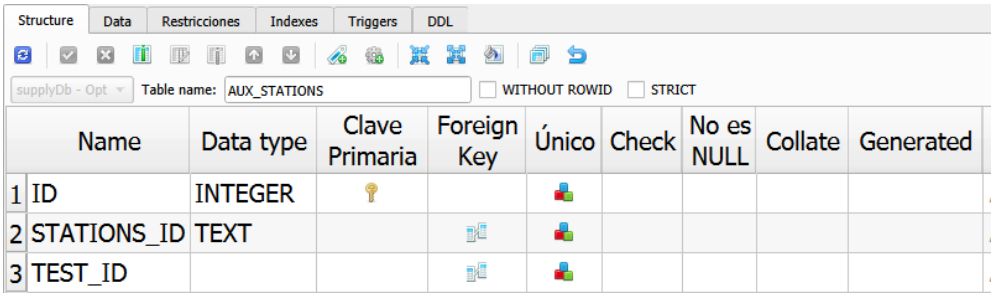
Figura 4.23: Tabla `SEATS_PRICE`

4.3.1.3. Tablas auxiliares

Las tablas auxiliares tienen como cometido relacionar los diferentes archivos con los datos almacenados en las demás tablas de la base de datos y así evitar que haya datos duplicados. Esto permite que en cada archivo se pueda determinar qué datos de las diferentes tablas corres-

ponden a ese archivo.

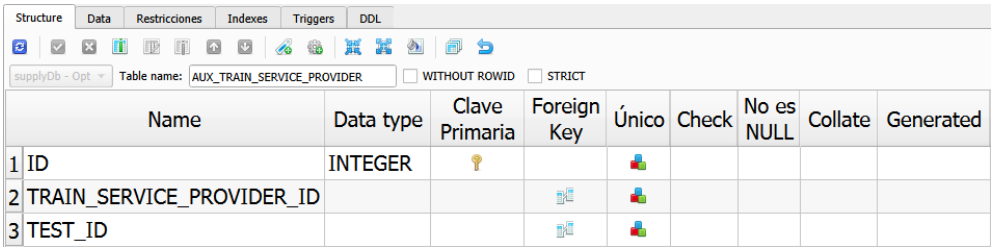
En la tabla AUX_STATIONS (Figura 4.24) se establecen las relaciones entre los archivos y las estaciones incluidas en el conjunto de datos de cada archivo.



	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	STATIONS_ID	TEXT							
3	TEST_ID								

Figura 4.24: Tabla AUX_STATIONS

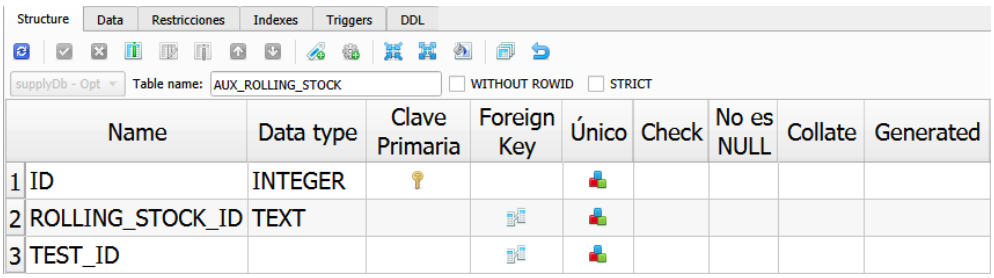
En AUX_TRAIN_SERVICE_PROVIDER (Figura 4.25) se establecen las relaciones entre los archivos y los proveedores de servicio ferroviario. Gracias a ello, se puede definir a qué archivo pertenecen los diferentes proveedores de servicios ferroviarios.



	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	TRAIN_SERVICE_PROVIDER_ID								
3	TEST_ID								

Figura 4.25: Tabla AUX_TRAIN_SERVICE_PROVIDER

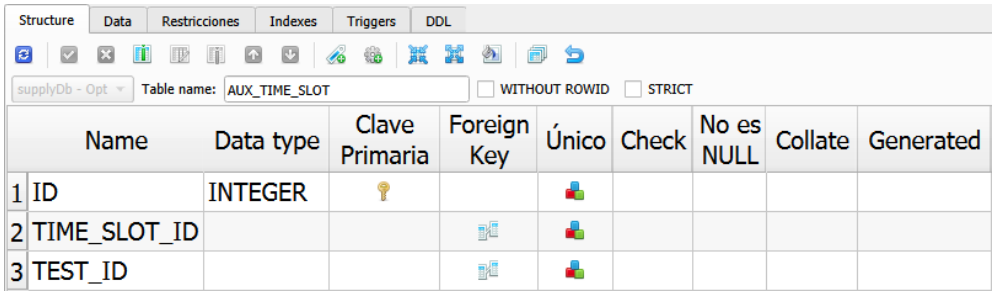
AUX_ROLLING_STOCK (Figura 4.26) vincula los archivos con el material rodante (rolling stock), relacionando así el archivo con el material rodante que aparece en dicho archivo.



	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	ROLLING_STOCK_ID	TEXT							
3	TEST_ID								

Figura 4.26: Tabla AUX_ROLLING_STOCK

La tabla AUX_TIME_SLOT (Figura 4.27) asocia cada archivo con los intervalos de tiempo (time slots) pertenecientes a cada uno de los archivos.

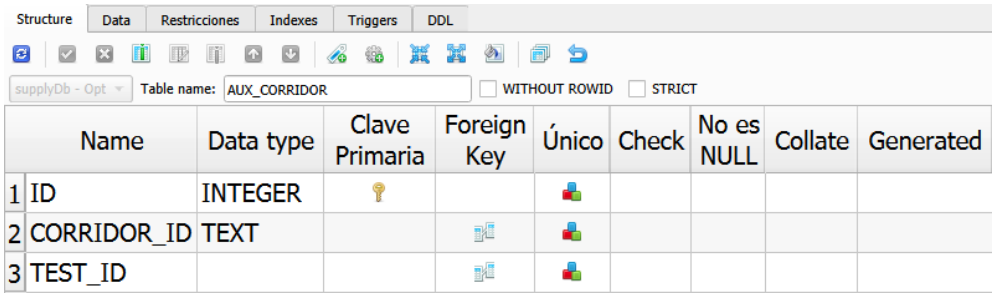

Structure | Data | Restricciones | Indexes | Triggers | DDL

supplyDb - Opt | Table name: AUX_TIME_SLOT | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	TIME_SLOT_ID								
3	TEST_ID								

Figura 4.27: Tabla AUX_TIME_SLOT

En la tabla AUX_CORRIDOR (Figura 4.28) se asocia cada archivo con corredores específicos relacionados con cada uno de los archivos.

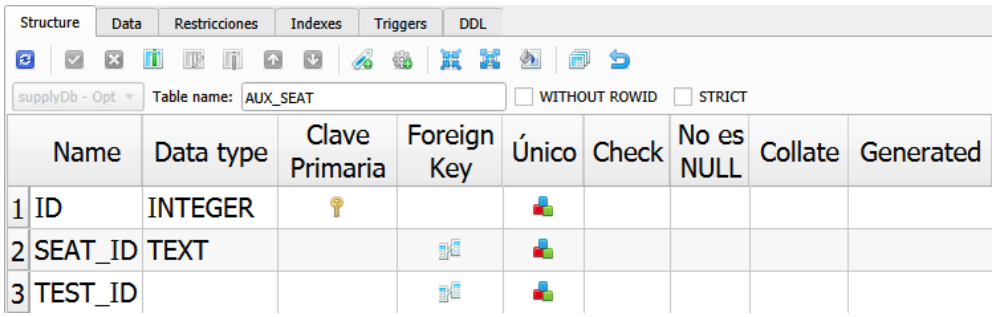

Structure | Data | Restricciones | Indexes | Triggers | DDL

supplyDb - Opt | Table name: AUX_CORRIDOR | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	CORRIDOR_ID	TEXT							
3	TEST_ID								

Figura 4.28: Tabla AUX_CORRIDOR

AUX_SEAT (Figura 4.29) indica qué tipos de asientos se emplean en los diferentes archivos que pueda albergar la base de datos.

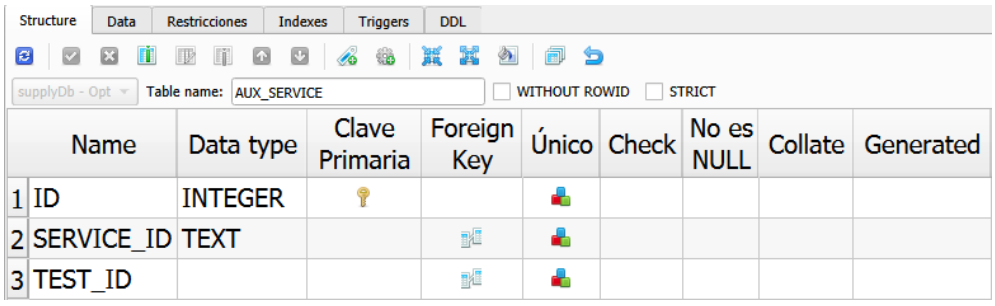

Structure | Data | Restricciones | Indexes | Triggers | DDL

supplyDb - Opt | Table name: AUX_SEAT | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	SEAT_ID	TEXT							
3	TEST_ID								

Figura 4.29: Tabla AUX_SEAT

Por último, la tabla AUX_SERVICE (Figura 4.30) relaciona los diferentes servicios almacenados en la tabla SERVICE (Figura 4.20) con los archivos que emplean los datos de dichos servicios.


Structure | Data | Restricciones | Indexes | Triggers | DDL

supplyDb - Opt | Table name: AUX_SERVICE | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	SERVICE_ID	TEXT							
3	TEST_ID								

Figura 4.30: Tabla AUX_SERVICE

A continuación se va a exponer un ejemplo de uso de las tablas auxiliares para extraer los datos de los servicios del archivo Yaml con el nombre "supply_03216_60000_2025-06-02_2025-06-16"[3]. Este archivo se encuentra subido al repositorio de GitHub. Empleando la sentencia SQL del Listado 4.30 se han obtenido todos los servicios pertenecientes al archivo mencionado.

Listado 4.30: Sentencia para el ejemplo de uso de tablas auxiliares

```
SELECT
    *
FROM
    SERVICE

WHERE SERVICE.ID IN (
SELECT
    AUX_SERVICE.SERVICE_ID
FROM
    AUX_SERVICE
WHERE
    AUX_SERVICE.TEST_ID = (SELECT TESTS.ID
                           FROM TESTS
                           WHERE TESTS.NAME = 'supply_03216_60000_2025-06-02_2025
                               -06-16'))
```

Dado que la lista de servicios seleccionados mediante la sentencia SQL del Listado 4.30 es extensa, se ha cogido una muestra de 5 filas, la cual aparece en la Tabla 4.8. El archivo que contiene todas las entradas generadas mediante la sentencia SQL del Listado 4.30 se encuentra en el siguiente enlace[4] del repositorio de GitHub.

	ID	DATE	LINE	TRAIN_SERVICE_PROVIDER	TIME_SLOT	ROLLING_STOCK
1	05065_02-06-2025-06.32	2025-06-02	05065	1	39210	1
2	05065_03-06-2025-06.32	2025-06-03	05065	1	39210	1
3	05065_04-06-2025-06.32	2025-06-04	05065	1	39210	1
4	05065_05-06-2025-06.32	2025-06-05	05065	1	39210	1
5	05065_06-06-2025-06.32	2025-06-06	05065	1	39210	1

Tabla 4.8: Muestra de datos obtenidos de la sentencia del Listado 4.30

4.3.2. Base de datos para los archivos de entrada de datos de la demanda

La base de datos que almacena la información de la demanda consta de 12 tablas. La organización de estas se puede encontrar en el EDR (Figura 4.31). Una versión más extendida de dicho EDR, que contiene también los nombres de las columnas de cada una de las tablas, puede encontrarse en el Anexo A.2. En esta base de datos se guardan los datos relacionados con la modelización de los patrones de demanda de los diferentes tipos de usuarios previstos para emplearlos en el simulador ROBIN.

[3] **Archivo "supply_03216_60000_2025-06-02_2025-06-16.yaml":** https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos/blob/master/Archivos%20Yaml%20y%20CSV/Originales/Oferta/supply_03216_60000_2025-06-02_2025-06-16.yaml

[4] **Archivo CSV completo:** https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos/blob/master/Archivos%20Yaml%20y%20CSV/Exportados/Vistas%20exportadas/Ejemplo%20de%20uso%20de%20tablas%20auxiliares%20largo.csv

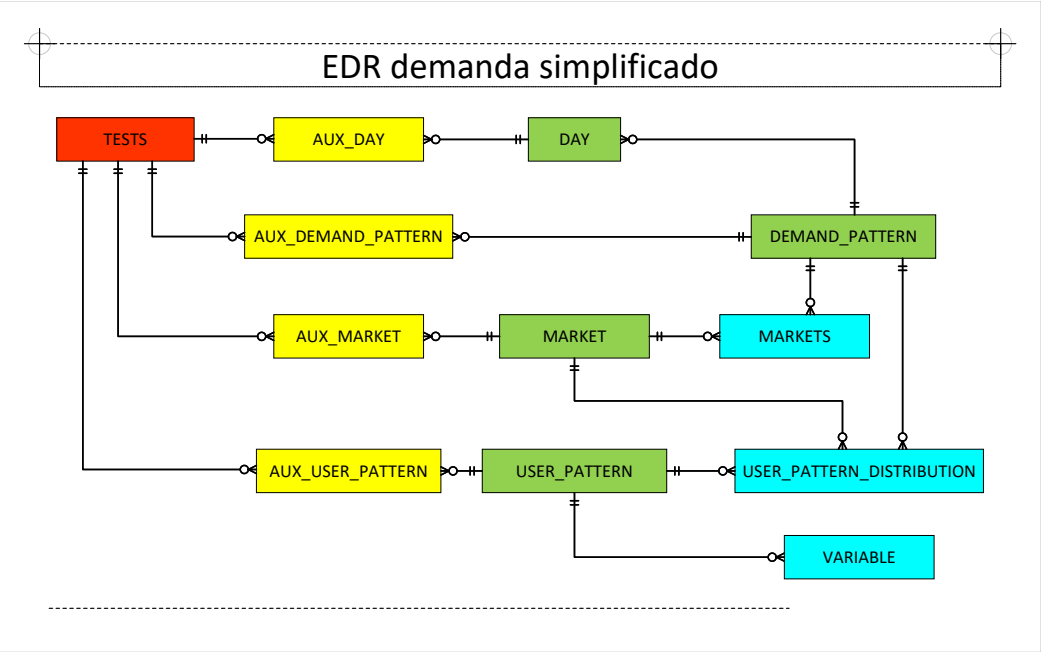


Figura 4.31: Esquema de la base de datos de la demanda. En naranja aparece la tabla principal, en amarillo las tablas auxiliares, en verde las tablas de datos y en azul las tablas de datos auxiliares.

La tabla principal de la base de datos aparece representada con el color naranja y tiene la finalidad de almacenar el nombre de los archivos Yaml de entrada de datos de la demanda y las observaciones que el usuario considere relevantes.

Las tablas auxiliares aparecen en color amarillo y relacionan los datos pertenecientes al archivo de entrada de datos de la demanda, almacenados en la base de datos, con el archivo en sí. Esto se realiza mediante el uso del identificador que posee la fila de la tabla principal en la que se almacena el nombre del archivo y los identificadores de las filas con los datos pertenecientes a ese archivo. Como ya se ha comentado en la base de datos de los archivos de entrada de datos de la oferta, esto se ha diseñado así para evitar elementos repetidos en las tablas de datos, optimizando así el espacio ocupado por los datos dentro de la base de datos.

Las tablas de datos, en color verde, están destinadas a almacenar los datos referenciados por las diferentes claves raíz dentro del archivo Yaml de entrada de datos de la demanda, por ejemplo los patrones de usuario o los patrones de demanda. Al igual que sucede en la base de datos de los datos de la oferta, si alguna clave dentro del archivo Yaml tiene asociada una estructura compleja (como una lista de listas o una lista de diccionarios, entre otras) los datos que se encuentren dentro de estas estructuras se almacenan en las tablas auxiliares de datos, en color azul claro. Estructuras más simples como una lista o un diccionario, si que se almacenan dentro de las tablas de datos como texto en formato JSON, como por ejemplo, la lista de reglas difusas que definen el comportamiento del patrón de usuario bajo la clave `rules` definida en la clave raíz `userPattern` o los diferentes conjuntos que componen las variables lingüísticas.

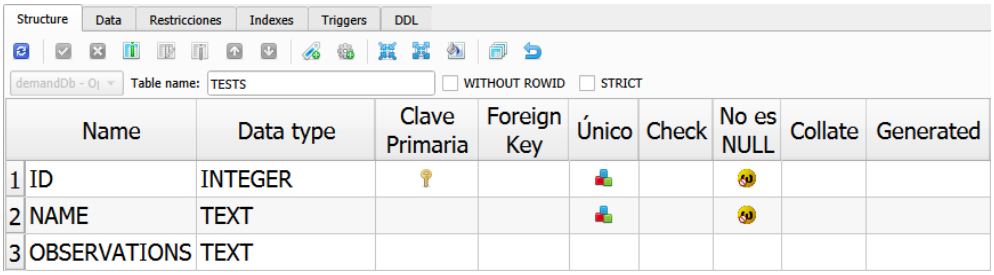
A continuación se muestra cada una de las tablas que componen la base de datos.

4.3.2.1. Tabla principal

Al igual que ocurría con la base de datos para los archivos de configuración de la oferta, la tabla principal será la de `TESTS`, la cual albergará los campos:

- `ID`: Identificador único que se le asigna a cada archivo.
- `NAME`: Nombre del archivo de configuración introducido a la base de datos.

- `OBSERVATIONS`: Observaciones proporcionadas por el usuario.

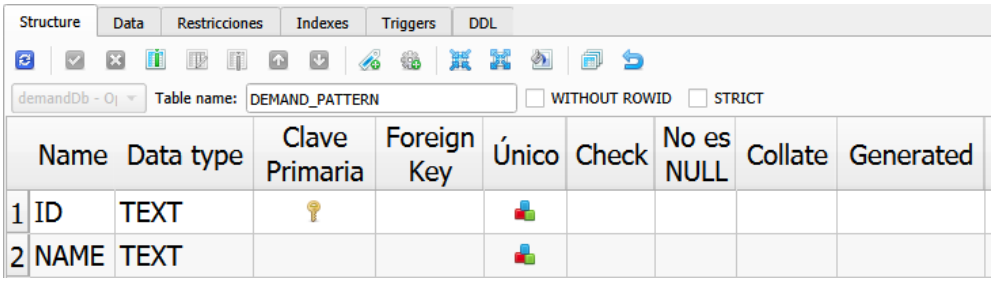


	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	NAME	TEXT							
3	OBSERVATIONS	TEXT							

Figura 4.32: Tabla `TESTS`

4.3.2.2. Tablas de datos

La tabla `DEMAND_PATTERN` (Figura 4.33) contiene el identificador del patrón de demanda (`ID`), así como el nombre que recibe el patrón de demanda (`NAME`).

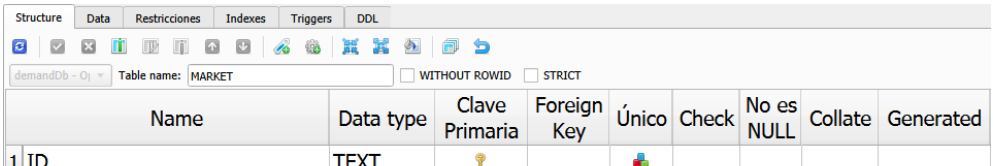


	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	TEXT							
2	NAME	TEXT							

Figura 4.33: Tabla `DEMAND_PATTERN`

`MARKET` (Figura 4.34) contiene los datos que definen los diferentes mercados posibles, es decir, los diferentes trayectos posibles entre las estaciones que se van a estudiar. La tabla `MARKET` cuenta con los siguientes campos:

- `ID`: Identificador único que recibe cada mercado.
- `DEPARTURE_STATION`: Nombre de la estación de salida.
- `DEPARTURE_STATION_COORDS`: Coordenadas de la estación de salida almacenadas como una lista en formato JSON.
- `ARRIVAL_STATION`: Nombre de la estación de destino.
- `ARRIVAL_STATION_COORDS`: Coordenadas de la estación de destino almacenadas como una lista en formato JSON.



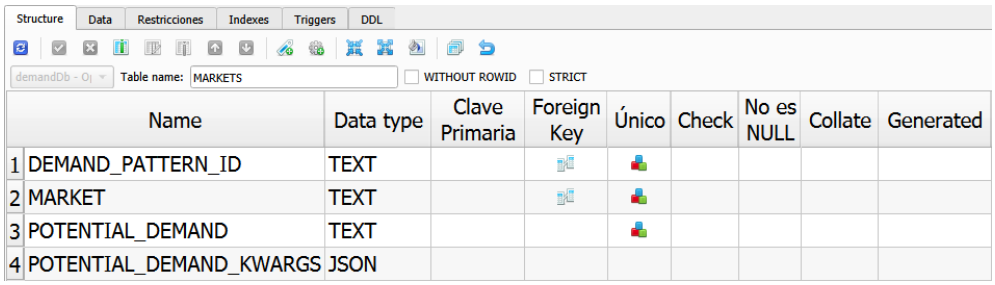
	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	TEXT							
2	DEPARTURE_STATION	TEXT							
3	DEPARTURE_STATION_COORDS	JSON							
4	ARRIVAL_STATION	TEXT							
5	ARRIVAL_STATION_COORDS	JSON							

Figura 4.34: Tabla `MARKET`

En MARKETS (Figura 4.35) se almacenan los diferentes mercados que pertenecen a los patrones de demanda de DEMAND_PATTERN. Esta tabla contiene las siguientes columnas:

- DEMAND_PATTERN_ID: Clave foránea perteneciente al identificador de un patrón de demanda.
- MARKET: Clave foránea que hace referencia a un mercado en específico.
- POTENTIAL_DEMAND: Función empleada en el simulador para el cálculo de la posible demanda.
- POTENTIAL_DEMAND_KWARGS: Argumentos para la función del calculo de la posible demanda.

Esta tabla se relaciona con las tablas DEMAND_PATTERN (Figura 4.33) y MARKET (Figura 4.34), mediante el uso de las claves foráneas DEMAND_PATTERN_ID y MARKET respectivamente.



	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	DEMAND_PATTERN_ID	TEXT							
2	MARKET	TEXT							
3	POTENTIAL_DEMAND	TEXT							
4	POTENTIAL_DEMAND_KWARGS	JSON							

Figura 4.35: Tabla MARKETS

Los datos de los patrones que pueden seguir los usuarios se almacenan en la tabla USER_PATTERN (Figura 4.36). Esta tabla consta de las siguientes columnas:

- ID: Identificador que recibe el patrón de usuario.
- NAME: Nombre del patrón de usuario.
- RULES: Diccionario en formato JSON que almacena las reglas difusas que después se emplearan en la simulación.
- ARRIVAL_TIME: Función para generar una distribución de los diferentes tiempos de llegada que prefiere un patrón de usuario determinado.
- ARRIVAL_TIME_KWARGS: Parámetros para la función definida en ARRIVAL_TIME.
- PURCHASE_DAY: Función que genera el número de días de antelación con los que un tipo de usuario realizará la compra de sus billetes.
- PURCHASE_DAY_KWARGS: Parámetros para la función definida en PURCHASE_DAY.
- FORBIDDEN_DEPARTURE_HOURS: Franja horaria durante la cual el usuario prefiere no iniciar su viaje.
- SEATS: Diccionario en JSON que contiene los datos de utilidad de cada tipo de asiento para un tipo de usuario.
- TRAIN_SERVICE_PROVIDERS: Diccionario que almacena los datos de utilidad de los diferentes proveedores de servicios ferroviarios para cada tipo de usuario. Probabilidad de que el usuario adquiera un billete que le resulta útil sin realizar una búsqueda exhaustiva.
- UTILITY_THRESHOLD: Umbral de utilidad para el patrón de usuario.
- ERROR: Función para generar una distribución del error cometido.
- ERROR_KWARGS: Parámetros para la función definida en ERROR.

Structure | Data | Restricciones | Indexes | Triggers | DDL

demandDb - Oj | Table name: USER_PATTERN | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	TEXT							
2	NAME	TEXT							
3	RULES	JSON							
4	ARRIVAL_TIME	TEXT							
5	ARRIVAL_TIME_KWARGS	JSON							
6	PURCHASE_DAY	TEXT							
7	PURCHASE_DAY_KWARGS	JSON							
8	FORBIDDEN_DEPARTURE_HOURS	JSON							
9	SEATS	JSON							
10	TRAIN_SERVICE_PROVIDERS	JSON							
11	EARLY_STOP	REAL							
12	UTILITY_THRESHOLD	INTEGER							
13	ERROR	TEXT							
14	ERROR_KWARGS	JSON							

Figura 4.36: Tabla `USER_PATTERN`

La tabla `VARIABLE` (Figura 4.37) contiene las diferentes variables difusas que se emplean en la simulación para cada tipo de patrón de usuario. Las columnas que esta tabla posee son las siguientes:

- `USER_PATTERN_ID`: Clave foránea que referencia al identificador del patrón de usuario (`USER_PATTERN`).
- `NAME`: Nombre que recibe la variable difusa.
- `TYPE`: Tipo de variable difusa, que puede ser de dos categorías: `fuzzy` o `categorical`.
- `SUPPORT`: Lista con dos valores en formato JSON que define el intervalo de la variable difusa. Este campo solo se completa cuando la variable es de tipo `fuzzy`.
- `SETS`: Diccionario que almacena los diferentes conjuntos difusos. La clave del diccionario es el nombre del conjunto, mientras que el valor asociado es una lista que representa el dominio del conjunto difuso.
- `LABELS`: Lista de etiquetas asociadas a las variables de tipo `categorical`. Este campo contendrá datos únicamente cuando el tipo de la variable sea `categorical`.

La tabla `VARIABLE` tiene una única relación con la tabla `USER_PATTERN` (Figura 4.36) mediante la clave foránea `USER_PATTERN_ID`.

Structure | Data | Restricciones | Indexes | Triggers | DDL

demandDb - Oj | Table name: VARIABLE | WITHOUT ROWID | STRICT

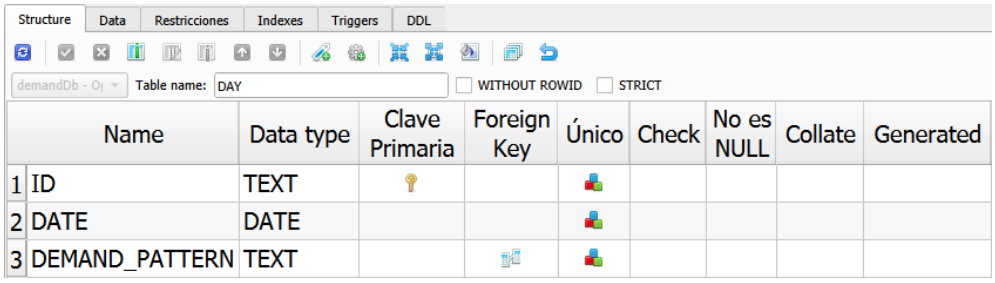
	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	USER_PATTERN_ID	TEXT							
2	NAME	TEXT							
3	TYPE	TEXT							
4	SUPPORT	JSON							
5	SETS	JSON							
6	LABELS	JSON							

Figura 4.37: Tabla `VARIABLE`

En DAY se relaciona un patrón de demanda con el día que se ofrecen los servicios ferroviarios. Esta tabla cuenta con:

- ID: Identificador único para el día.
- DATE: Fecha en la que se aplica el patrón de demanda.
- DEMMAND_PATTERN: Clave foránea que relaciona el día con un patrón de demanda mediante el ID de la tabla DEMAND_PATTERN.

DAY tiene una relación con la tabla DEMAND_PATTERN (Figura 4.33) mediante el empleo de la clave foránea DEMMAND_PATTERN.



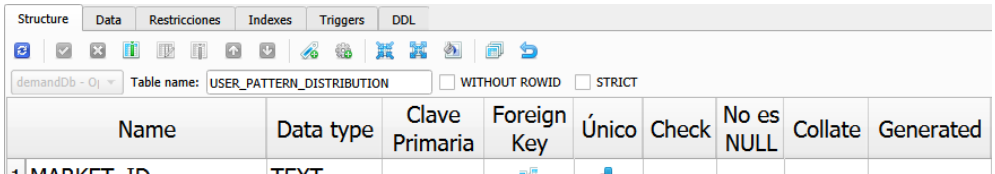
	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	TEXT							
2	DATE	DATE							
3	DEMAND_PATTERN	TEXT							

Figura 4.38: Tabla DAY

USER_PATTERN_DISTRIBUTION almacena los datos de la distribución de los diferentes patrones de usuario para determinados patrones de demanda. USER_PATTERN_DISTRIBUTION consta de las siguientes columnas:

- MARKET_ID: Clave foránea que relaciona el ID de un mercado con USER_PATTERN_DISTRIBUTION.
- DEMAND_PATTERN_ID: Clave foránea que relaciona un patrón de demanda usando el identificador de este.
- USER_PATTERN_ID: Clave foránea que relaciona un patrón de usuario mediante el ID de USER_PATTERN.
- PERCENTAGE: Porcentaje de un tipo de usuario que se espera para un determinado patrón de demanda.

La tabla USER_PATTERN_DISTRIBUTION se relaciona con las tablas MARKET (Figura 4.34), DEMAND_PATTERN (Figura 4.33) y USER_PATTERN (Figura 4.36) mediante las claves foráneas MARKET_ID, DEMAND_PATTERN_ID y USER_PATTERN_ID.



	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	MARKET_ID	TEXT							
2	DEMAND_PATTERN_ID	TEXT							
3	USER_PATTERN_ID	TEXT							
4	PERCENTAGE	REAL							

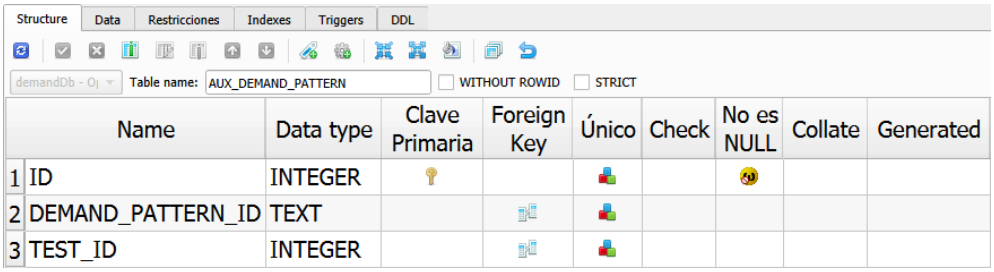
Figura 4.39: Tabla USER_PATTERN_DISTRIBUTION

4.3.2.3. Tablas auxiliares

Las tablas auxiliares están diseñadas para vincular cada archivo con los datos almacenados en las demás tablas de la base de datos, evitando la duplicación de información. Gracias a esta

estructura, es posible determinar con precisión qué registros corresponden a cada archivo sin necesidad de replicar los datos en múltiples instancias.

La tabla AUX_DEMAND_PATTERN (Figura 4.40) vincula los archivos con los distintos patrones de demanda que puedan estar almacenados en la tabla DEMAND_PATTERN (Figura 4.33).

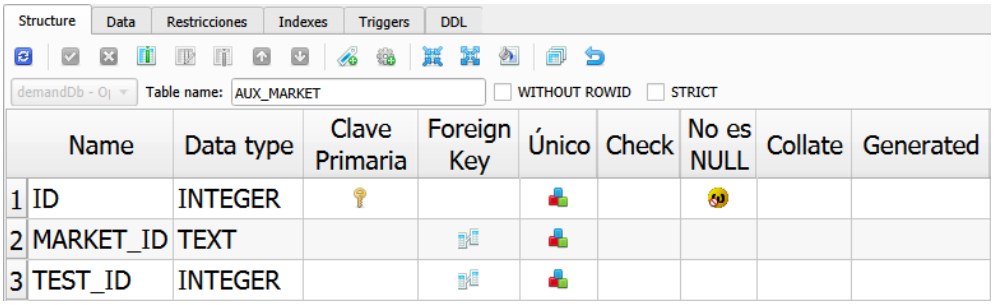

Structure | Data | Restricciones | Indexes | Triggers | DDL

demandDb - O | Table name: AUX_DEMAND_PATTERN | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	DEMAND_PATTERN_ID	TEXT							
3	TEST_ID	INTEGER							

Figura 4.40: Tabla AUX_DEMAND_PATTERN

AUX_MARKET (Figura 4.41) establece la relación entre los archivos y los mercados que forman parte de la tabla MARKET (Figura 4.34).

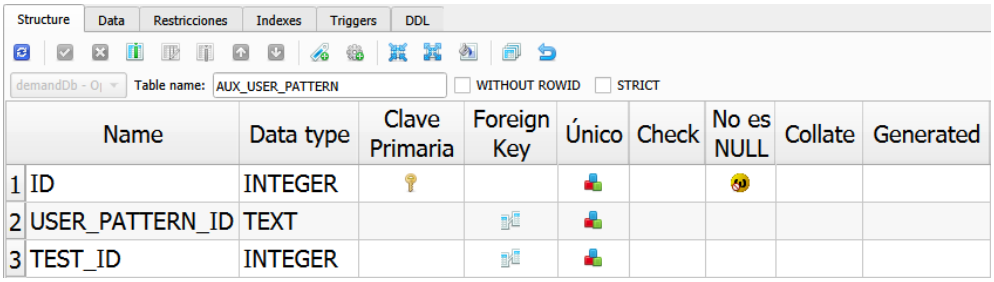

Structure | Data | Restricciones | Indexes | Triggers | DDL

demandDb - O | Table name: AUX_MARKET | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	MARKET_ID	TEXT							
3	TEST_ID	INTEGER							

Figura 4.41: Tabla AUX_MARKET

En AUX_USER_PATTERN (Figura 4.42) se gestionan las relaciones entre los archivos y los distintos patrones de usuario contenidos en USER_PATTERN (Figura 4.36).


Structure | Data | Restricciones | Indexes | Triggers | DDL

demandDb - O | Table name: AUX_USER_PATTERN | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	USER_PATTERN_ID	TEXT							
3	TEST_ID	INTEGER							

Figura 4.42: Tabla AUX_USER_PATTERN

AUX_DAY (Figura 4.43) relaciona los archivos con los días específicos en los que se ejecutan las pruebas que están almacenados en la tabla DAY (Figura 4.38).

Structure | Data | Restricciones | Indexes | Triggers | DDL

Table name: AUX_DAY | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	DAY_ID	TEXT							
3	TEST_ID	INTEGER							

Figura 4.43: Tabla AUX_DAY

4.3.3. Base de datos para la configuración de los resultados

La base de datos destinada a almacenar los resultados consta de tres tablas, organizadas conforme al EDR (Figura 4.44). Una versión más extendida de dicho EDR, que contiene también los nombres de las columnas de cada una de las tablas, puede encontrarse en el Anexo A.3. Su propósito es registrar los datos generados por el simulador ROBIN.

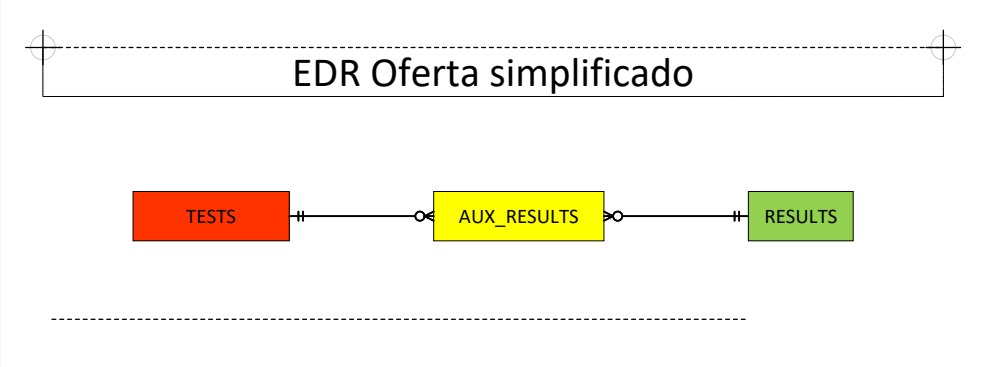


Figura 4.44: Esquema de la base de datos de los resultados. En naranja aparece la tabla principal, en amarillo la tabla auxiliar y en verde la tabla de datos.

La tabla principal de la base de datos, en naranja, tiene como objetivo almacenar el nombre del archivo CSV de resultados importado a la base de datos y las observaciones que un usuario realice sobre el mismo.

La tabla auxiliar, en color amarillo, relaciona los datos almacenados en la tabla de datos con los archivos que posean dichos datos. Esto se realiza mediante los identificadores de las filas que contienen los datos y el identificador asignado a cada uno de los archivos durante la importación de la información.

La tabla de datos almacena los datos provenientes del archivo CSV importado. Debido a que los archivos CSV son tablas, no poseen ninguna estructura compleja, por lo que en esta base de datos no hay ninguna tabla auxiliar de datos.

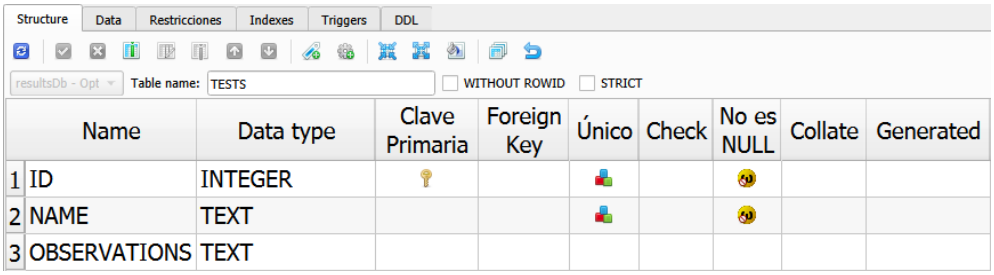
A continuación se muestra cada una de las tablas que componen esta base de datos.

4.3.3.1. Tabla principal

Al igual que ocurre con la base de datos para los archivos de entrada de la oferta y de la demanda, la tabla principal será la de `TESTS` y tiene los siguientes campos:

- `ID`: Identificador único que se le asigna a cada archivo.
- `NAME`: Nombre del archivo de configuración introducido a la base de datos.

- `OBSERVATIONS`: Observaciones proporcionadas por el usuario.



	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	NAME	TEXT							
3	OBSERVATIONS	TEXT							

Figura 4.45: Tabla `TESTS`

4.3.3.2. Tablas de datos

La tabla `RESULTS` (Figura 4.46) contendrá los datos de los resultados de las simulaciones, realizadas con el simulador ROBIN y consta de las siguientes columnas:

- `ID`: Identificador de cada uno de los resultados.
- `USER_PATTERN`: Patrón de usuario empleado.
- `DEPARTURE_STATION`: Estación de salida de la simulación.
- `ARRIVAL_STATION`: Estación de llegada empleado en la simulación.
- `ARRIVAL_DAY`: Día de llegada en la simulación.
- `ARRIVAL_TIME`: Tiempo relativo en el que el tren llega a la estación.
- `PURCHASE_DAY`: Días de antelación de la compra de los billetes.
- `SERVICE`: Identificador del servicio que se ha simulado.
- `SERVICE_DEPARTURE_TIME`: Hora prevista de salida del servicio.
- `SERVICE_ARRIVAL_TIME`: Hora prevista de llegada del servicio.
- `SEAT`: Tipo de asiento comprado por el usuario.
- `PRICE`: Precio pagado por el usuario.
- `UTILITY`: Porcentage que describe como de útil es el resultado para el tipo de usuario.
- `BEST_SERVICE`: Servicio que mejor se adapta a los patrones de usuario y demanda.
- `BEST_SEAT`: Asiento que mejor se adapta a los patrones de usuario y demanda.
- `BEST_UTILITY`: Valor más alto de utilidad alcanzado durante la simulación.

Structure | Data | Restricciones | Indexes | Triggers | DDL

resultsDb - Opt | Table name: RESULTS | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	TEXT							
2	USER_PATTERN	TEXT							
3	DEPARTURE_STATION	TEXT							
4	ARRIVAL_STATION	TEXT							
5	ARRIVAL_DAY	TEXT							
6	ARRIVAL_TIME	NUMERIC							
7	PURCHASE_DAY	NUMERIC							
8	SERVICE	TEXT							
9	SERVICE_DEPARTURE_TIME	NUMERIC							
10	SERVICE_ARRIVAL_TIME	NUMERIC							
11	SEAT	TEXT							
12	PRICE	NUMERIC							
13	UTILITY	NUMERIC							
14	BEST_SERVICE	TEXT							
15	BEST_SEAT	TEXT							
16	BEST_UTILITY	NUMERIC							

Figura 4.46: Tabla RESULTS

4.3.3.3. Tablas auxiliares

La tabla AUX_RESULTS (Figura 4.47) vincula los archivos con los distintos resultados de los tests simulados que se encuentran en la tabla RESULTS (Figura 4.46).

Structure | Data | Restricciones | Indexes | Triggers | DDL

resultsDb - Opt | Table name: AUX_RESULTS | WITHOUT ROWID | STRICT

	Name	Data type	Clave Primaria	Foreign Key	Único	Check	No es NULL	Collate	Generated
1	ID	INTEGER							
2	RESULTS_ID	TEXT							
3	TEST_ID	INTEGER							

Figura 4.47: Tabla AUX_RESULTS

4.4. Desarrollo del software

El software desarrollado en este TFG se ha realizado empleando el lenguaje de programación Python [2] en su versión 3.13, apoyándose en los paquetes Tkinter [13] para hacer la interfaz, sqlite3 [9] para realizar el manejo de las bases de datos, PyYaml [5] para leer los archivos de configuración de la oferta y la demanda, y por último, csv [11] para leer los archivos de resultados que salen del simulador. Para la realización de los scripts *.py* se ha utilizado el programa PyCharm Community Edition [17].

4.4.1. Estructura funcional del software

El programa se ha dividido en módulos para facilitar su desarrollo y su legibilidad. La Figura 4.48 muestra los módulos del programa, junto con la conexión y funciones de estos. Cada bloque del diagrama representa un módulo encargado de una serie de tareas específicas,

como por ejemplo, el bloque "Lectura de YAML" corresponde al módulo que se encarga de la lectura y procesamiento de los datos dentro de los archivos Yaml. Las flechas representan la relación funcional existente entre los distintos módulos. Por ejemplo, el módulo de lectura de Yaml, mencionado antes, extrae y procesa los datos para que el módulo de introducción de datos, haciendo uso de las funciones del módulo de manejo de las bases de datos y de los datos procesados por el módulo de lectura de Yaml, inserte esos datos dentro de la base de datos correspondiente.

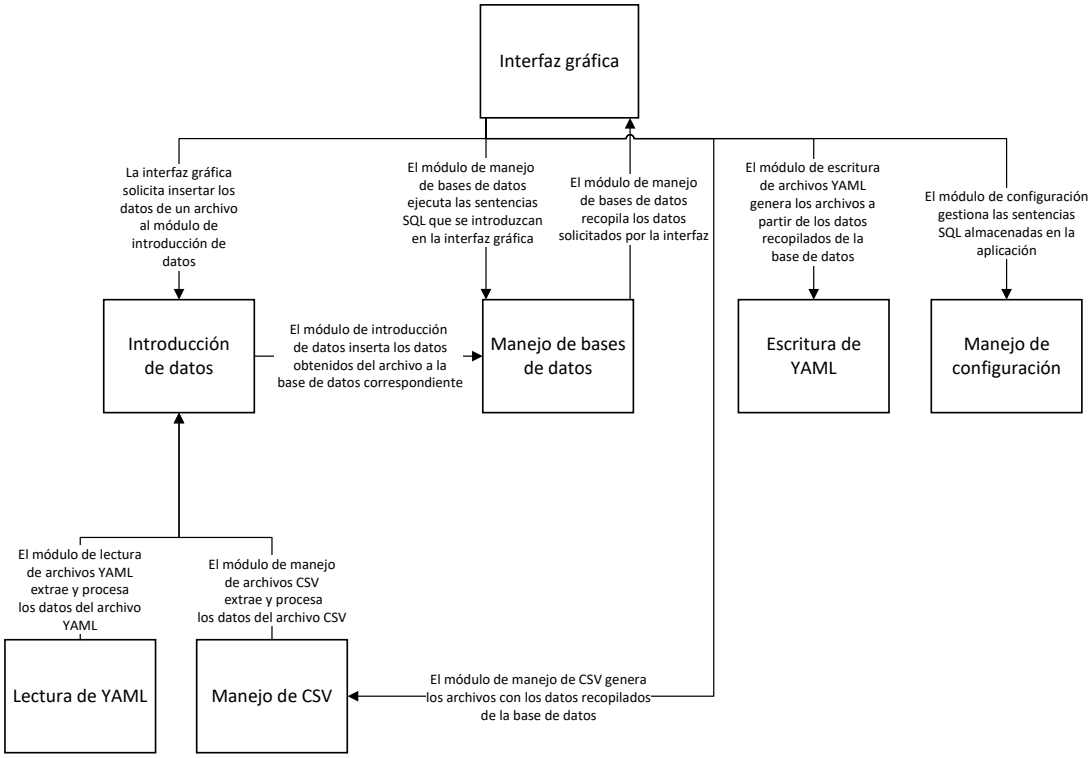


Figura 4.48: Diagrama que muestra las relaciones entre los módulos en los que se ha dividido el programa

A continuación se detalla la función de cada uno de los módulos, así como el archivo Python que contiene el código asociado al mismo:

- `Interfaz Gráfica (UI.py)`: Es el programa principal que maneja la interfaz y todos los demás procesos que hacen funcionar la aplicación.
- `Lectura de Yaml (yamlParser.py)`: Módulo encargado de la lectura de los datos contenidos en los archivos Yaml de la configuración de la oferta y la demanda.
- `Escritura de Yaml (yamlWriter.py)`: Módulo encargado de la escritura de los archivos Yaml exportados desde las bases de datos.
- `Manejo de bases de datos (SQLHandler.py)`: Módulo encargado de toda la comunicación con las bases de datos.
- `Introducción de datos (dataLoger.py)`: Módulo encargado de introducir los datos empleando los módulos `yamlParser.py`, `csvHandler.py` y `SQLHandler.py` en la base de datos correspondiente.
- `Manejo de CSV (csvHandler.py)`: Módulo encargado de leer y formatear los datos obtenidos de la simulación para introducirlos en la base de datos de los resultados.
- `Configuración (configManager.py)`: Módulo encargado de gestionar la configuración de la aplicación.

Todos los módulos mencionados y de los que se va a hablar en las siguientes secciones, se encuentran en la carpeta Modules[5] dentro del repositorio de GitHub. A continuación, se detalla el funcionamiento de cada uno de ellos.

4.4.1.1. Módulo de lectura de Yaml

El módulo `yamlParser.py` está diseñado para extraer los datos de los archivos Yaml, para después introducirlos en la base de datos empleando el módulo `dataLoger.py`, que a su vez emplea el módulo `SQLHandler.py` para introducir la información extraída a la base de datos.

Los archivos Yaml de entrada de datos de la oferta de los que han de extraerse la información siguen una estructura como la que se ha explicado en la Sección 4.2. La plantilla estructural que siguen estos archivos puede verse en el anexo B.1. A continuación, se muestra un archivo de ejemplo (Listado 4.31) que sigue la estructura de los archivos Yaml de la configuración de la oferta (Anexo B.1).

Listado 4.31: Ejemplo de archivo Yaml de entrada de datos de la oferta

```
stations:
- id: '00001'
  name: Ejemplo1
  city: Ciudad1
  short_name: EJ1
  coordinates:
    latitude: 1.00000000
    longitude: 1.00000000

- id: '00002'
  name: Ejemplo2
  city: Ciudad2
  short_name: EJ2
  coordinates:
    latitude: 2.00000000
    longitude: 2.00000000

seat:
- id: '0'
  name: asientoEjemplo
  hard_type: 0
  soft_type: 0

corridor:
- id: '0'
  name: corredorEjemplo
  stations:
  - org: '00001'
    des:
    - org: '00002'
      des: []

line:
```

[5] https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos/tree/master/Modules

```
- id: '00001'
  name: Line 00001
  corridor: '0'
  stops:
    - station: '00001'
      arrival_time: 0
      departure_time: 0
    - station: '00002'
      arrival_time: 55
      departure_time: 56

rollingStock:
- id: '0'
  name: TrenEjemplo
  seats:
  - hard_type: 0
    quantity: 2

trainServiceProvider:
- id: '0'
  name: ProveedorEjemplo
  rolling_stock:
  - '0'

timeSlot:
- id: '00001'
  start: '00:00:00'
  end: '00:10:00'

service:
- id: 00001_06-09-2023-00.00
  date: '2023-09-06'
  line: '00001'
  train_service_provider: '0'
  time_slot: '00001'
  rolling_stock: '0'
  origin_destination_tuples:
  - origin: '00001'
    destination: '00002'
    seats:
    - seat: '0'
      price: 0
  capacity_constraints: null
```

En el archivo de ejemplo, se puede ver que hay 8 claves raíz, que son:

- `stations`: En esta clave raíz se encuentran los datos de todas las estaciones que se vayan a emplear en las simulaciones de los servicios descritos en este archivo.
- `seat`: Aquí se almacenan los datos de los diferentes tipos de asiento, como su nombre, el tipo de asiento físico y los beneficios asociados a este.
- `corridor`: Información sobre los corredores ferroviarios que serán utilizados en la simulación.

- `line`: Datos sobre las diferentes lineas que se empleen en las simulaciones.
- `rollingStock`: Caracteristicas sobre los diferentes trenes que van a circular por los corredores para satisfacer los servicios definidos para la simulación.
- `trainServiceProvider`: Información sobre los diferentes proveedores de servicios ferroviarios, que van a prestar servicios en la simulación.
- `timeSlot`: Aquí se encuentran los diferentes intervalos de tiempo en los que los trenes se encuentran estacionados en el anden.
- `service`: Información sobre los diferentes servicios que van a ofrecerse a lo largo del día.

Para obtener los datos de este archivo, como se ha comentado previamente, se ha empleado la librería PyYaml. Esta librería transforma los datos contenidos en el archivo Yaml a tipos de variables existentes en Python (diccionarios, listas, etc.). El siguiente diccionario es un ejemplo de la salida que tendría el archivo Yaml de ejemplo anterior (Listado 4.31) para la clave de `stations`:

Listado 4.32: Diccionario de salida de `stations`

```
{
    "stations": [
        {
            "id": "00001",
            "name": "Ejemplo1",
            "city": "Ciudad1",
            "short_name": "EJ1",
            "coordinates": {
                "latitude": 1.00000000,
                "longitude": 1.00000000
            }
        },
        {
            "id": "00002",
            "name": "Ejemplo2",
            "city": "Ciudad2",
            "short_name": "EJ2",
            "coordinates": {
                "latitude": 2.00000000,
                "longitude": 2.00000000
            }
        }
    ]
}
```

Para poder acceder a los datos de los diferentes archivos que el usuario quiera introducir a la base de datos, primero se ha de abrir el archivo específico mediante el uso de la función `loadSupplyFile` (Listado 4.33), en caso de que sea un archivo de configuración de oferta, y `loadDemandFile`, en caso de que el archivo de configuración sea para la demanda. Estas dos funciones se encuentran dentro de la clase `Parser` en el módulo `yamlParser.py`.

El comportamiento de la función `loadSupplyFile` (Listado 4.33) se muestra en el diagrama de flujo de la Figura 4.49. Su proceso es el siguiente:

1. *Selección de archivo*: Abre una ventana de selección de archivo usando la función de Tkinter `filedialog.askopenfilename` que devuelve la ruta completa al archivo seleccio-

nado por el usuario.

2. *Comprobación ruta*: Después, se comprueba que esta ruta no esté vacía para, posteriormente, asignarla como valor de la variable de la clase `Parser`: `self.supplyFilePath`. Si la ruta está vacía o contiene el valor `None`, la función interrumpe su ejecución y lanza el error personalizado `SupplyFileNotFound` que hace que la función devuelva un valor de -2.

3. *Comprobación errores*: En caso de que ocurra otro error distinto al mencionado durante la ejecución, se devuelve el valor -1.

4. *Carga de archivo*: En caso de que ningún error suceda, se extrae el nombre del archivo de la ruta y se almacena en la variable `self.supplyFileName` y se ejecuta la función `self.getRawDataFromSupply` que carga los datos de todo el archivo Yaml a la variable `self.supplyData`.

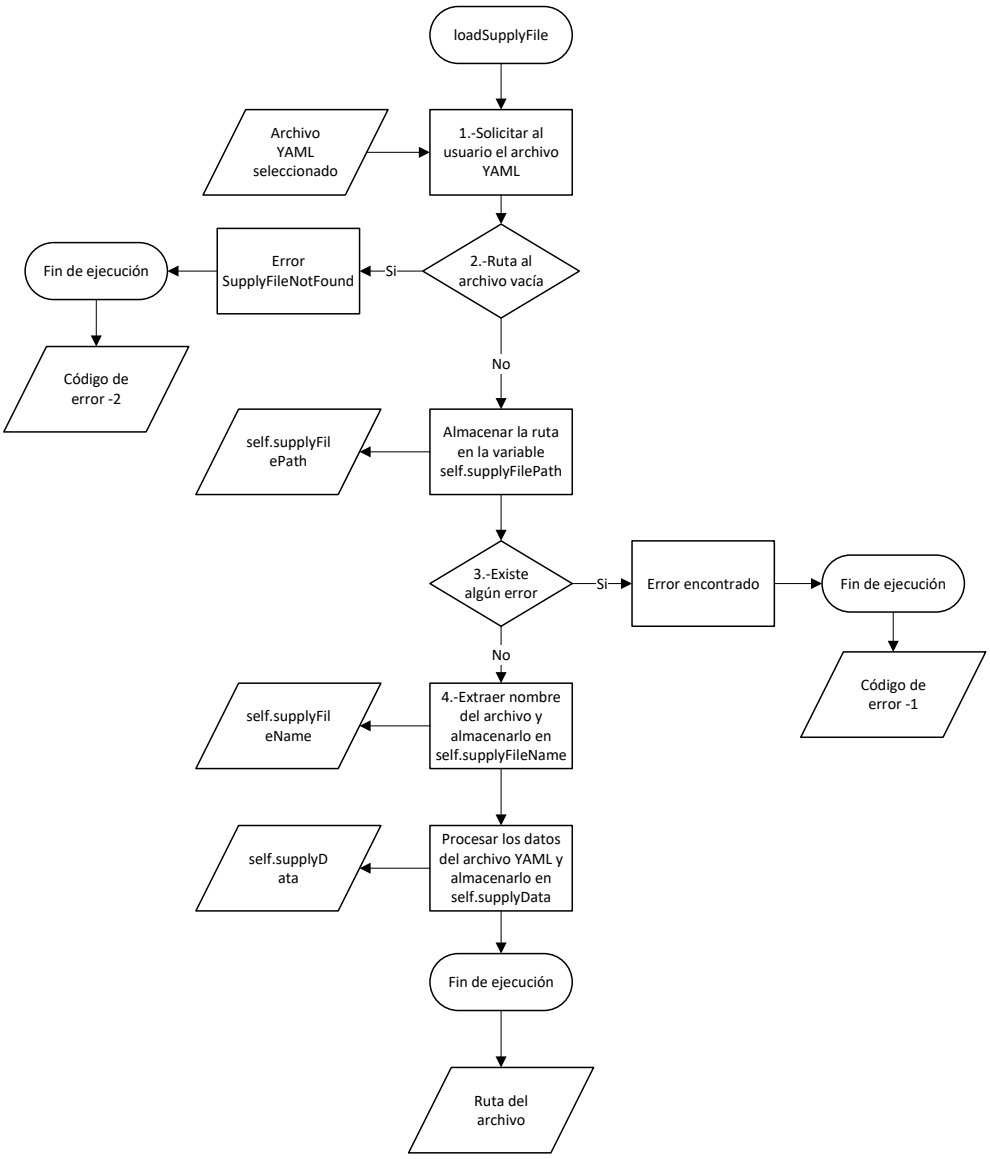


Figura 4.49: Diagrama de flujo que muestra el proceso de lectura de un archivo Yaml mediante `loadSupplyFile`.

Listado 4.33: Función `loadSupplyFile`

```
def loadSupplyFile(self):
    try:
        f = filedialog.askopenfilename(
            title="Seleccionar archivo de oferta", initialdir=self.
            defaultInputDataFolder,
            filetypes=[("Archivos YAML", "*.yml"), ("Todos los archivos", "*.*")],
             defaultextension=".yml")
        if f != "":
            self.supplyFilePath = f.replace("/", "/")
        else:
            self.supplyFilePath = ""
            self.supplyFileName = ""
        if self.supplyFilePath is None or self.supplyFilePath == '':
            #print("Archivo de oferta no seleccionado")
            raise SupplyFileNotFound

        self.supplyFileName = self.supplyFilePath.split("/")[-1].split(".")[0]
        self.getRawDataFromSupply()
        return f

    except SupplyFileNotFound:
        return -2

    except Exception as e:
        print(e)
        return -1
```

Una vez obtenidos todos los datos almacenados del archivo de configuración de la oferta, se extraen los datos de las diferentes claves raíz. Continuando con el ejemplo de la clave raíz `stations`, para la extracción de los datos de las diferentes estaciones que aparecen en el archivo de entrada de datos de la oferta, se emplea la función `getStationsData` (Listado 4.34), cuyo comportamiento se refleja en el diagrama de flujo de la Figura 4.50. El proceso que sigue la función `getStationsData` es el siguiente:

1. *Obtención de datos:* Se obtienen los datos de todas las estaciones del archivo de entrada de datos de la oferta que se ha cargado previamente. Estos datos están almacenados como una lista de diccionarios.

2. *Procesado de datos:* Se construye una lista de listas, a partir de los datos de las estaciones, donde cada sublista contiene el identificador de la estación, el nombre, la ciudad en la que está ubicada, el nombre corto que tenga asignado y, por último, un diccionario con las coordenadas donde se encuentra emplazada la estación, en ese orden.

3. *Datos de retorno:* Por último, se devuelve la lista de listas construida.

Listado 4.34: Función `getStationsData`

```
def getStationsData(self):
    """
    Funcion que devuelve todos los campos de stations en la posicion index
    ordenados en una lista como en el archivo Yaml.\n
    Salida: [<id>,<name>,<city>,<shortName>,{"latitude":>lat>,"longitude":<lon>}]
    :return: Devuelve el value de stations en la posicion index.
```

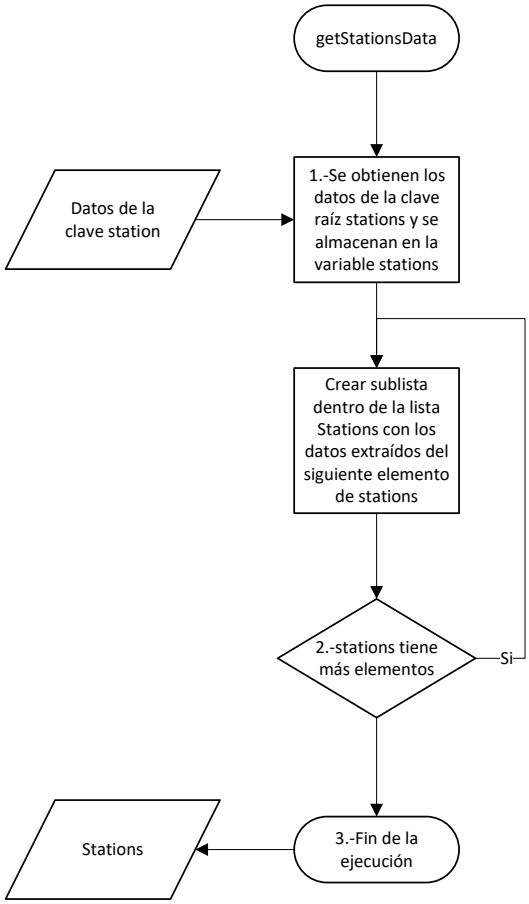


Figura 4.50: Diagrama de flujo del proceso de extracción y procesado de los datos de la clave raíz `stations` con `getStationsData`.

```
"""
Stations = []
try:
    if self.supplyData is None: raise EmptySupplyData
    stations = self.getStations()
    Stations = [[
                data.get("id"),
                data.get("name"),
                data.get("city"),
                data.get("short_name"),
                json.dumps(data.get("coordinates"))]
                for data in stations
                ]
    return Stations
except IndexError:
    print("Se ha sobrepasado el índice máximo")
    return "indexError"
except Exception as e:
    print(e)
    return -1
```

Para el resto de claves raíz, las diferentes funciones que extraen sus datos funcionan de manera similar. La diferencia entre las funciones que extraen datos de las claves raíz son las claves que se emplean para recuperar los datos de los diccionarios donde se encuentra la información, salvo unas excepciones que necesitan emplear funciones auxiliares para el acondicionamiento de los datos.

Como se ha mencionado, existen funciones que deben emplear una función auxiliar para adecuar los datos a la estructura de la base de datos o darles un formato que, mediante un procesado simple, permita la inserción de los datos en la tabla correspondiente dentro de la base de datos. Estas funciones son: `getCorridorData`, `getLineData`, `getRollingStockData` y `getServiceData`.

Para el caso de la función `getCorridorData`, que extrae los datos de los corredores ferroviarios bajo la clave raíz `corridor`, los datos que tienen una estructura que no permite insertarlos directamente a la base de datos son las diferentes estaciones que componen el corredor. Como se puede apreciar en el ejemplo del archivo de configuración de la oferta (Listado 4.31), su estructura se compone de un diccionario donde la clave `org` contiene el identificador de la estación de origen, mientras que la clave `des` contiene otro diccionario con la misma estructura; es decir una nueva clave `org`, con el identificador de la estación de origen y una clave `des`. La clave `des` puede contener sucesivamente otro diccionario con la misma estructura, hasta que la estación referenciada en `org` sea la última del corredor, en cuyo caso aparece una lista vacía; es decir, `[]`.

La función auxiliar encargada de extraer los ramales del corredor y darles un formato válido para su posterior inserción a la base de datos es `extractStationsFromCorridor` (Listado 4.35). Esta función genera una lista de listas donde cada sublista es un ramal del corredor. Estas sublistas contienen los identificadores de las estaciones de los ramales ordenados de manera que el primer elemento de cada una de las sublistas sea la estación donde comienza el ramal y el último elemento corresponde con la estación donde termina. El comportamiento de esta función aparece reflejado en el diagrama de flujo de la Figura 4.51. El proceso de funcionamiento es:

1. *Comprobar variable `stations`:* Se comprueba que la variable `stations` que se pasa como parámetro no tenga el valor `None`. Si esto ocurre, se le asigna una lista vacía a esta

variable, y en caso contrario, se mantiene igual.

2. *Copia de datos:* Se copian los datos de la variable pasada como parámetro `stationsData` a otra variable denominada `data`.

3. *Comprobar tipo de `data`:* Se comprueba si la variable data es una lista o un diccionario. Dependiendo del tipo, se distinguen tres comportamientos:

   a) *`data` es una lista:* Si la variable `data` es una lista, se itera por los diccionario dentro de la lista de tal manera que, se agrega el valor asociado a la clave `org` a la lista `newStations`. Se genera una lista nueva denominada `l2` y se le da el valor asociado a la clave `des`.
   - Si `l2` esta vacía, significa que se ha llegado al final de un ramal, por lo que la lista `newStations` se agrega a la lista de listas `allStations`. Tras esto, el programa finaliza y se devuelve el valor de `allStations` que contiene todos los ramales que existen en el corredor.
   - En caso contrario, se comprueba el tipo de `l2`, si no es una lista o un diccionario, se agrega la lista `newStations` a `allStations`. Después, el programa finaliza y se devuelve el valor de `allStations` que contiene todos los ramales que existen en el corredor.
   - Si por el contrario, `l2` es un diccionario o una lista, se llama a la función `extractStationsFromCorridor` pasando como parámetros la variable `l2`, como el parámetro `stationsData`, y `newStations`, como el parámetro `stations` y el valor que se devuelva de la ejecución se añade a `allStations`.

   Este proceso se repite para cada uno de los diccionarios dentro de la lista dentro de `data`.

   b) *`data` es un diccionario:* Si la variable `data` es un diccionario, se extrae el valor asociado a la clave `org` y se añade a `newStations`. Después, se agrega a `l2` el valor de la clave `des`.
   - Si `l2` esta vacía, se agrega el valor de `newStations` a `allStations`, se devuelve el valor de `allStations` y se finaliza la ejecución de la función.
   - Si por el contrario, `l2`, no esta vacía y se trata de una lista o un diccionario, se llama a la función `extractStationsFromCorridor` pasando como parámetros la variable `l2`, como el parámetro `stationsData`, y `newStations`, como el parámetro `stations` y el valor que se retorne de la ejecución se añade a `allStations`.
   - Si `l2` no es una lista ni un diccionario, se añade el valor de `newStations` a `allStations`, se devuelve `allStations` y se finaliza la ejecución.

   c) *`data` no es ni lista ni diccionario:* Si `data` no es ni una lista ni un diccionario, se agrega el valor de `newStations` a `allStations`, se devuelve el valor de `allStations` y se finaliza la ejecución de la función.

Listado 4.35: Función `extractStationsFromCorridor`

```
def extractStationsFromCorridor(self, stationsData: list, stations=None):
"""
Extrae los ramales del corredor. Esta función devuelve una lista
ordenada con todos los ramales. Cada ramal será una lista ordenada
siendo el primer elemento la estación de inicio del ramal y el último
elemento será la última estación del ramal.
:param stationsData: Lista en las que están contenidos los IDs de
```

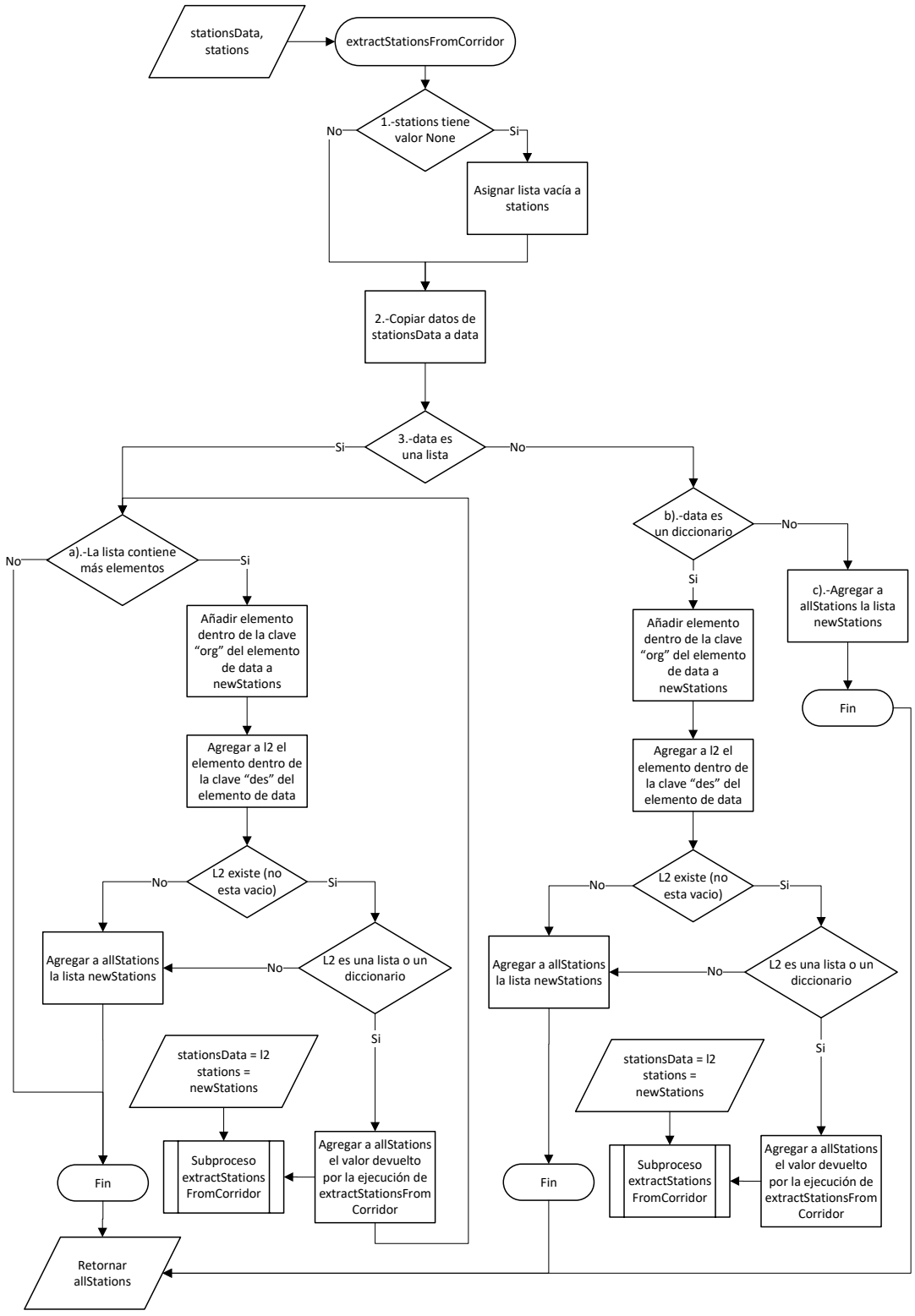


Figura 4.51: Diagrama de flujo de la función `extractStationsFromCorridor`

```
        todas las estaciones por las que va a pasar el
        corredor
        :param stations: Ruta actual acumulada (se utiliza internamente al
        llamar recursivamente).
        :return: Devuelve una lista de listas con todos los ramales del
        corredor.
        """
        if stations is None:
            stations = []

        # Se hace una copia de stationsData para evitar modificar el original
        data = deepcopy(stationsData)
        allStations = []

        if isinstance(data, list):
            # Se recorre cada elemento de la lista (cada elemento representa
            # un inicio o ramal)
            for item in data:
                newStations = stations.copy()
                newStations.append(item.get("org"))
                l2 = item.get("des")
                if l2:
                    # Si existen destinos, se llama recursivamente (ya sea
                    # que l2 sea lista o diccionario)
                    if isinstance(l2, (list, dict)):
                        allStations.extend(
                            self.extractStationsFromCorridor(l2, newStations))
                    else:
                        allStations.append(newStations)
                else:
                    # No hay más destinos: se agrega la ruta completa
                    allStations.append(newStations)
        elif isinstance(data, dict):
            # Caso en el que data es un único diccionario
            newStations = stations.copy()
            newStations.append(data.get("org"))
            l2 = data.get("des")
            if l2:
                if isinstance(l2, (list, dict)):
                    allStations.extend(
                        self.extractStationsFromCorridor(l2, newStations))
                else:
                    allStations.append(newStations)
            else:
                allStations.append(newStations)
        else:
            # En caso de que data no tenga el formato esperado, se devuelve la
            # ruta actual
            allStations.append(stations)

        return allStations
```

Otra de las funciones auxiliares que dan un formato válido a los datos para su inserción a la base de datos, sería la función `extractStopsFromLine` (Listado 4.36). Esta función se encarga de extraer la información de las diferentes paradas que se realizan a lo largo de una línea de tren. Esta información está conformada por el identificador de la estación en la que el tren ha de parar, el tiempo de llegada a esa estación y el tiempo de salida de esa estación. La función devuelve una lista de listas, donde cada sublista contiene los datos ya mencionados para que de esta manera, puedan ser introducidos a la tabla `STOPS` de la base de datos (Figura 4.19).

Listado 4.36: Función `extractStopsFromLine`

```
def extractStopsFromLine(stopsData: list):
    """
    Esta funcion extrae todas las paradas con sus tiempos de llegada y salida de
    una linea.
    Devuelve una lista de listas donde cada lista se compone de la parada, tiempo
    de llegada y tiepo de salida,
    por lo que respeta el orden del archivo Yaml.
    :param stopsData: Lista de diccionarios con los datos de todas las paradas en
    la linea.
    :return: lista de listas con los datos de todas las paradas de la linea.
    """
    stops = []
    data = deepcopy(stopsData)
    while data:
        stops.append([str(data[0].get("station")), data[0].get("arrival_time"),
        data[0].get("departure_time")])
        data.pop(0)
    return stops
```

La función auxiliar `extractSeatsFromRollingStock` (Listado 4.37) tiene como cometido generar un diccionario donde cada una de las claves del diccionario son los diferentes tipos de asientos físicos del tren. Cada una de estas claves toma como valor el número de asientos del tipo almacenado como clave que posee el tren. Por ejemplo, si un tren tiene 200 asientos del tipo 1 y 50 asientos del tipo 2, el diccionario resultante de ejecutar esta función será: `{"1":200,"2":50}`

Listado 4.37: Función `extractSeatsFromRollingStock`

```
def extractSeatsFromRollingStock(seatsData: list):
    """
    Esta funcion extrae los datos de los asientos de uno de los trenes de
    rollingStock pasado como parametro.
    Devuelve un diccionario que emplea como key el hard_type y como value para esa
     key el numero de asientos de
    ese tipo de los que dispone dicho tren.
    :param seatsData: Datos extraidos de seats en rollingStock.
    :return: Devuelve un diccionario con los tipos de asiento y la cantidad de
    estos.
    """
    seats = {}
    data = deepcopy(seatsData)
```

```
        while data:
            seats.update({str(data[0].get("hard_type")): data[0].get("quantity")})
            data.pop(0)
        return seats
```

Por último, la función auxiliar `extractSeatsPriceFromServiceOdt` (Listado 4.38) se encarga de extraer los datos de los orígenes, destinos y precios por asiento que están asociados a la clave `origin_destination_tuples` de un servicio. Estos datos se almacenan en una lista de listas, donde cada lista tiene como primer elemento la estación de origen, seguida de la estación de destino y, por último, un diccionario donde la clave es el tipo de asiento, definido previamente en la clave raíz `seat`, a la que se le asigna como valor el precio que recibe ese tipo de asiento para ese trayecto entre la estación de origen y la estación de destino.

Listado 4.38: Función `extractSeatsPriceFromServiceOdt`

```
def extractSeatsPriceFromServiceOdt(odtData: list):
    data = deepcopy(odtData)
    outputData = []
    while data:
        price = {}
        seats = data[0].get("seats")
        odt = data[0]
        while seats:
            price.update({str(seats[0].get("seat")): seats[0].get("price")})
            seats.pop(0)
        output = [odt.get("origin"), odt.get("destination"), deepcopy(price)]
        outputData.append(output)
        data.pop(0)
    return outputData
```

Para cargar el archivo de entrada de datos de la demanda, el procedimiento es similar al empleado por la función `loadSupplyFile` (Listado 4.33). La función encargada de esta tarea es `loadDemandFile` y su comportamiento aparece reflejado en el diagrama de flujo de la Figura 4.52. El procedimiento que sigue la función es el siguiente:

1. *Selección de archivo:* Abre una ventana de selección de archivo usando una función de Tkinter denominada `filedialog.askopenfilename`, que devuelve la ruta completa al archivo seleccionado.

2. *Comprobación de ruta:* Después, se comprueba que la ruta no este vacía para asignarla como valor de la variable `self.demandFilePath`. En caso de que la ruta esté vacía o contenga el valor `None`, la función interrumpe su ejecución normal lanzando el error personalizado `DemandFileNotFound`, para acto seguido, devolver el valor -2.

3. *Comprobación de errores:* En caso de que ocurra otro error diferente al mencionado durante la ejecución, se detiene la ejecución normal de la función se devuelve el valor -1.

4. *Carga de archivo:* Si ningún error sucede, se extrae el nombre del archivo de la ruta y se almacena en la variable `self.demandFileName` y se ejecuta la función `getRawDataFromDemand` que carga los datos de todo el archivo Yaml a la variable `self.demandData`.

Al igual que ocurre con los archivos de la oferta, las funciones que extraen los datos del archivo de la demanda son muy similares entre sí, todas obtienen los datos asociados a las

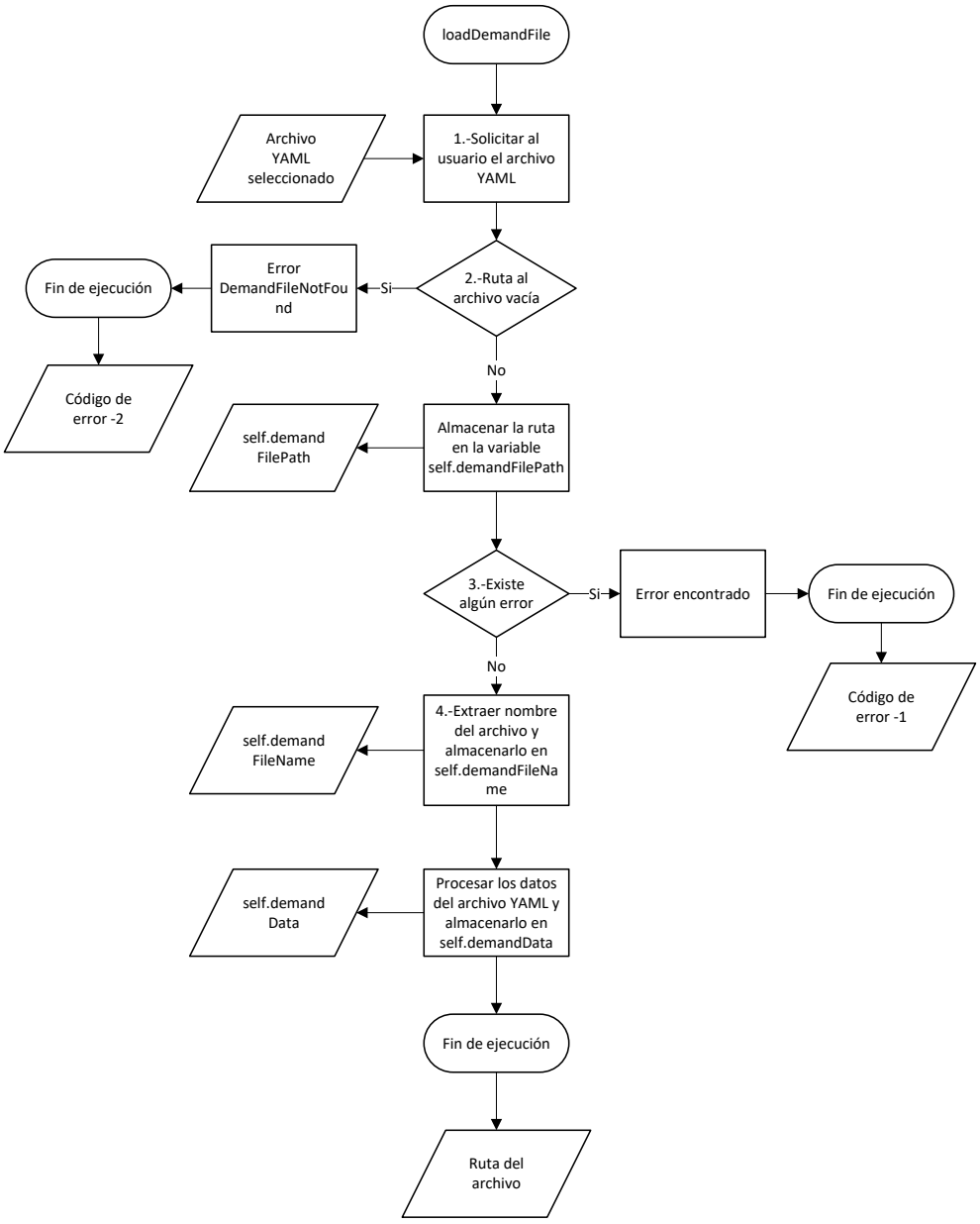


Figura 4.52: Diagrama de flujo que muestra el proceso de lectura de un archivo Yaml mediante `loadDemandFile`.

claves raíz empleando las claves bajo la clave raíz. Un ejemplo de una función de la demanda es la función `getMarktetData` (Listado 4.39)que devuelve los datos extraídos de la clave raíz `market` del archivo de configuración de la demanda.

Pero al igual que ocurría con algunas de las funciones destinadas a obtener los datos de los archivos de configuración de la oferta, algunas de las funciones destinadas a obtener los datos del archivo de la demanda necesitan apoyarse en funciones auxiliares para reformar los datos y hacerlos coincidir con lo que espera la base de datos. Estas funciones auxiliares son las siguientes: `extractMarketsFromDemmandPattern`, `extractUserPatternDistributionFromMarkets`, `extractSeatsFromUserPattern`, `extractTrainServiceProvidersFromUserPattern` y `reformatVariableSets`

Listado 4.39: Función `getMarktetData`

```
def getMarketData(self):
    """
    Funcion que devuelve todos los campos de market en la posicion index ordenados
     en una lista como en el archivo Yaml.\n
    Salida: [<id>,<departure_station>,<[<departure_station_coords>]>,
    <arrival_station>,<timeSlot>,<[<arrival_station_coords>]>]
    :return: Devuelve el value de market en la posicion index.
    """
    Market = []
    try:
        if self.demandData is None: raise EmptyDemandData
        market = self.getMarket()
        Market = [
                    [
                        market[index].get("id"),
                        market[index].get(
                            "departure_station"),
                        json.dumps(market[index].get(
                            "departure_station_coords")),
                        market[index].get(
                            "arrival_station"),
                        json.dumps(market[index].get(
                            "arrival_station_coords"))]

                    for index in range(0,len(market))
                ]

        return Market
    except Exception as e:
        print(e)
        return -1
```

La función `extractMarketsFromDemmandPattern` (Listado 4.40) extrae los mercados potenciales de la clave `markets` dentro de la clave raíz `demandPattern` y los introduce en una lista de listas donde cada lista tiene como primer elemento el identificador que recibe el mercado, seguido de la función que va a utilizar el simulador para calcular la demanda potencial, los parámetros de dicha función y la distribución de los patrones que siguen los usuarios. Este último elemento, a su vez, usa la función auxiliar `extractUserPatternDistributionFromMarkets`

(Listado 4.41) para reformular la distribución de los patrones de usuario.

Estos patrones de usuario, tras usar la función `extractUserPatternDistributionFromMarkets`, se reformulan en una lista de diccionarios, donde la clave de cada diccionario es el identificador del patrón de usuario, mientras que el valor asociado a esta clave es el porcentaje de usuarios de ese patrón en concreto.

Listado 4.40: Función `extractMarketsFromDemmandPattern`

```
def extractMarketsFromDemmandPattern(self,marketsData):
    markets = []
    data = deepcopy(marketsData)
    while data:
        markets.append([data[0].get("market"),
                        data[0].get("potential_demand"),
                        json.dumps(data[0].get("potential_demand_kwargs")),
                        self.extractUserPatternDistributionFromMarkets(data[0].get
                        ("user_pattern_distribution"))])
        data.pop(0)
    return markets
```

Listado 4.41: Función `extractUserPatternDistributionFromMarkets`

```
def extractUserPatternDistributionFromMarkets(UpdData):
    percentage = {}
    data = deepcopy(UpdData)
    while data:
        percentage.update({str(data[0].get("id")): data[0].get("percentage")})
        data.pop(0)
    return percentage
```

Las funciones auxiliares `extractSeatsFromUserPattern` y `extractTrainServiceProvidersFromUserPattern` funcionan de manera similar a la función `extractUserPatternDistributionFromMarkets`, forman una lista de diccionarios, donde la clave de cada diccionario es el identificador que recibe el asiento, si la función es `extractSeatsFromUserPattern` o el identificador que recibe el proveedor de servicios ferroviarios, en caso de que la función sea `extractTrainServiceProvidersFromUserPattern` y, como valor referenciado por dicha clave, se toma o bien el valor de utilidad del asiento, si la función es `extractSeatsFromUserPattern` o bien el valor de utilidad del proveedor de servicios ferroviarios si la función es `extractTrainServiceProvidersFromUserPattern`.

Por último, la función auxiliar `reformatVariableSets` es la encargada de reformular los campos de `variables` en una lista de diccionarios donde cada diccionario tiene almacenada una variable. Cada diccionario contiene unos datos en función del tipo de variable. Si la variable es del tipo "fuzzy", este diccionario contiene el nombre, el tipo, en este caso "fuzzy", el dominio de la variable lingüística y, por último, los conjuntos difusos. Si la variable es "categorical", el diccionario contiene el nombre, el tipo, en este caso "categorical" y la lista de etiquetas de la variable.

Listado 4.42: Función `reformatVariableSets`

```
def reformatVariableSets(variableData:dict):
    outputData = []
    for varData in variableData:
```

```
            if "labels" in varData.keys(): outputData.append(varData)
            else:
                data = deepcopy(varData)
                variables = {"name":data.get("name"),"type":data.get("type"),"support"
                :data.get("support")}
                setsData = data.get("sets")
                sets = {}
                while setsData:
                    sets.update({str(setsData[0]):data.get(setsData[0])})
                    setsData.pop(0)
                variables.update({"sets":sets})
                outputData.append(variables)
        return outputData
```

4.4.1.2. Módulo de escritura de Yaml

El módulo `yamlWriter.py` está diseñado para reconstruir los archivos Yaml empleando los datos provenientes de las bases de datos. Estos archivos reconstruidos se guardan en la ubicación que el usuario designe mediante una ventana de selección de carpeta. La ubicación que abre la ventana de selección de carpeta, por defecto, se encuentra ubicada en el directorio de trabajo de la aplicación, normalmente, es la misma carpeta que el ejecutable, dentro de la carpeta "outputData". Esta carpeta se genera la primera vez que se ejecuta el programa o cuando la carpeta no existe dentro del directorio de trabajo.

La función `saveFile` de la clase `Writer` del módulo se emplea para generar archivos Yaml a partir de los datos que se van a insertar en él y del nombre que va a recibir el archivo, ambos pasados como parámetros a la función. El comportamiento de la función puede observarse en el diagrama de flujo de la Figura 4.53. El proceso seguido por la función es:

1. *Seleccionar ubicación de guardado:* Abre una ventana de selección de carpeta empleando la función de Tkinter `filedialog.askdirectory`, que devuelve la ruta a la carpeta seleccionada.

2. *Comprobar ruta:* Se comprueba que la ruta no esté vacía. Si la ruta se encuentra vacía, la función detiene su ejecución.

3. *Generar archivo:* Se genera un archivo en blanco con el nombre que se ha introducido antes como parámetro.

4. *Insertar datos:* Se insertan los datos pasados como parámetro en el archivo recién creado.

4.4.1.3. Módulo de manejo de CSV

`CsvHandler.py` es el módulo encargado de leer los archivos CSV que arroja el simulador con los resultados de las simulaciones realizadas y generar los archivos CSV con la información de la base de datos. Este módulo se compone de dos clases: `csvReader` y `csvWriter`

La clase `csvReader` contiene las funciones necesarias para obtener los datos de los resultados de los archivos CSV. Para obtener dichos datos, se usa la función `loadCsvFile` (Listado 4.43). El comportamiento de esta función se encuentra representado en el diagrama de flujo de la Figura 4.54. Su proceso es el siguiente:

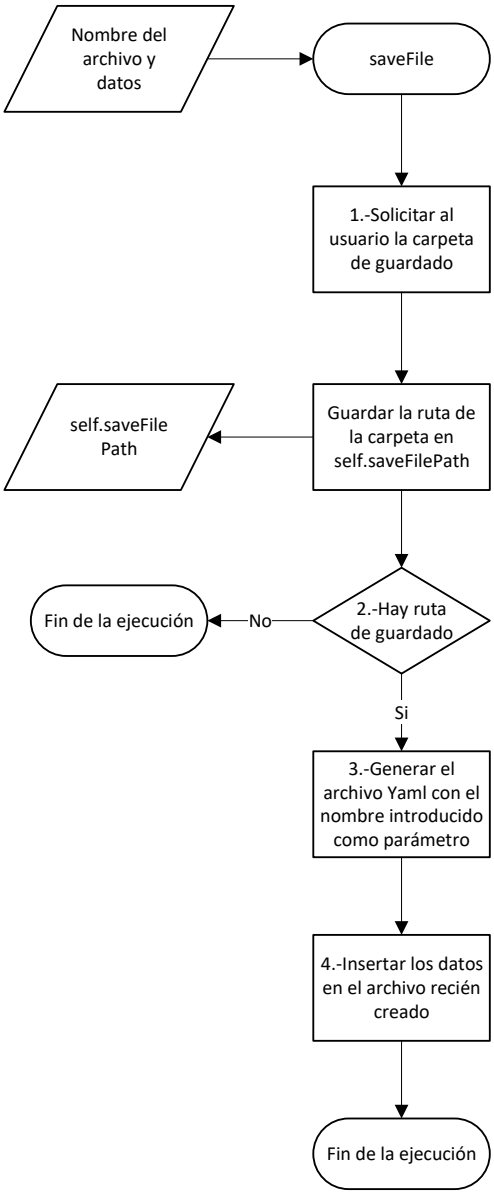


Figura 4.53: Diagrama de flujo que muestra el proceso de guardado de un archivo Yaml mediante `saveFile`.

1. *Selección del archivo:* Se utiliza una ventana de selección de archivo para obtener la ruta del archivo seleccionado por el usuario. Esta ventana se genera con la función `filedialog.askopenfilename` de Tkinter.

2. *Comprobación de ruta:* Se comprueba que la ruta obtenida no este vacía para asignarla como valor de la variable `self.csvFilePath`. Si la ruta está vacía, o posee el valor `None`, la función detiene su ejecución, lanza el error personalizado `CsvFileNotFound` y devuelve el valor -2.

3. *Comprobación de errores:* Si llega a darse un error durante la ejecución diferente al mencionado, se detiene la ejecución de la función y se devuelve el valor -1.

4. *Carga de archivo:* En caso de que no se haya producido ningún error, se extrae el nombre del archivo de la ruta almacenada y se almacena en la variable `self.csvFileName`. Después, se ejecuta la función `getRawDataFromCsv`, que carga los datos a la variable `self.csvData`.

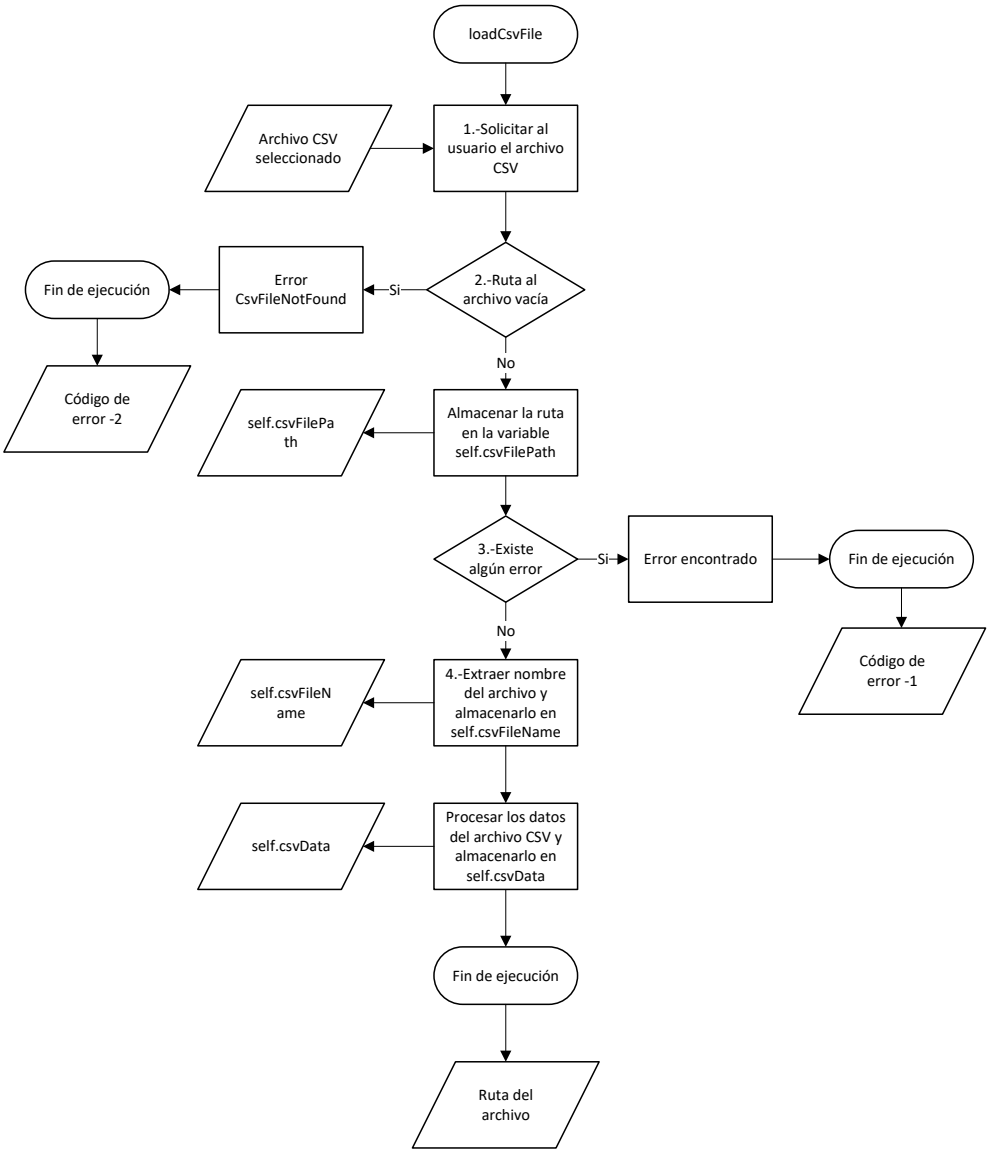


Figura 4.54: Diagrama de flujo que muestra el proceso de lectura de un archivo CSV mediante `loadCsvFile`.

Listado 4.43: Función `loadCsvFile`

```
def loadCsvFile(self):
    try:
        f = filedialog.askopenfilename(title="Seleccionar archivo de resultados",
                                       initialdir=self.defaultInputDataFolder,
                                       filetypes=[("Archivos CSV", "*.csv"), ("
                                       Todos los archivos", "*.*")],
                                       defaultextension=".csv")

        if f != "":
            self.csvFilePath = f.replace("/", "/")
        if self.csvFilePath is None or self.csvFilePath == '':
            print("Archivo de oferta no seleccionado")
            raise CsvFileNotFound

        self.csvFileName = self.csvFilePath.split("/")[-1].split(".")[0]
        self.getRawDataFromCsv()
        return f

    except CsvFileNotFound:
        return -2

    except Exception as e:
        print(e)
        return -1
```

La función `getRawDataFromCsv` (Listado 4.44) primero determina qué delimitador emplea el archivo CSV para separar la información. Aunque lo habitual es que los archivos CSV empleen la coma para separar los datos, también se puede emplear el punto y coma como separador. Haciendo esta comprobación, se evitan futuros errores a la hora de leer la información del archivo. Una vez hecho esto, se procede a la lectura de los datos contenidos en el archivo y se almacenan en la variable `self.csvData`

Listado 4.44: Función `getRawDataFromCsv`

```
def getRawDataFromCsv(self):
    try:
        with open(self.csvFilePath,mode="r", encoding='utf-8') as file:
            sample = file.read(100)
            sniffer = csv.Sniffer()
            delimiter = sniffer.sniff(sample).delimiter
            file.seek(0)
            for row in self.csvReader(file, delimiter=delimiter):
                self.csvData.append((row.get("id"),
                                     row.get("user_pattern"),
                                     row.get("departure_station"),
                                     row.get("arrival_station"),
                                     row.get("arrival_day"),
                                     row.get("arrival_time"),
                                     row.get("purchase_day"),
                                     row.get("service"),
                                     row.get("service_departure_time"),
```

```
                                        row.get("service_arrival_time"),
                                        row.get("seat"),
                                        row.get("price"),
                                        row.get("utility"),
                                        row.get("best_service"),
                                        row.get("best_seat"),
                                        row.get("best_utility")))
            print(self.csvData[:][0])
            print(self.csvData)
        except Exception as e:
            print(e)
            return -1
```

La clase `csvWriter` es la encargada de la escritura de los archivos CSV que contienen los datos provenientes de la base de datos. Estos archivos pueden ser los archivos de resultados almacenados en la base de datos o archivos generados a partir de una sentencia SQL ejecutada por el usuario en el programa en una de las bases de datos. Esta clase toma como parámetros el nombre del archivo a crear, los datos que componen ese archivo y los nombres de las columnas dentro del archivo CSV. Si los nombres de las columnas no se pasan a la función como argumentos, se usan los nombres de las columnas que aparecen en el archivo CSV de los resultados del simulador. Estos parámetros son almacenados en variables de la clase para que se puedan usar en cualquier función perteneciente a la clase.

Al igual que en el módulo `yamlWriter`, los archivos CSV que se generen se almacenan en una carpeta que el usuario escoja a través de un selector de carpeta. La carpeta que se selecciona, por defecto, es la carpeta "outputData" que se encuentra ubicada en el directorio de trabajo de la aplicación. Esta carpeta se genera siempre que se inicie el programa y esta no exista en el directorio de trabajo.

Para generar estos archivos CSV se emplea la función `saveFile` de la clase `Writer` del módulo. Esta función requiere que se pasen como parámetros el nombre que va a recibir el archivo y los datos que se van a insertar en él. El comportamiento de la función puede observarse en el diagrama de flujo de la Figura 4.55. El proceso seguido por la función es:

1. *Seleccionar ubicación de guardado:* Mediante una ventana de selección de carpeta generada por la función de Tkinter `filedialog.askdirectory`, se obtiene la ruta a la carpeta seleccionada por el usuario.

2. *Comprobar ruta:* Se comprueba que la ruta no esté vacía. Si la ruta se encuentra vacía, la función detiene su ejecución.

3. *Generar archivo:* Una vez comrpobado que la ruta no se encuentra vacía, se genera un archivo en blanco con el nombre que se ha introducido antes como parámetro en la ubicación seleccionada.

4. *Insertar datos:* Se insertan los datos que se han pasado previamente como parámetro de la función, en el archivo recién creado.

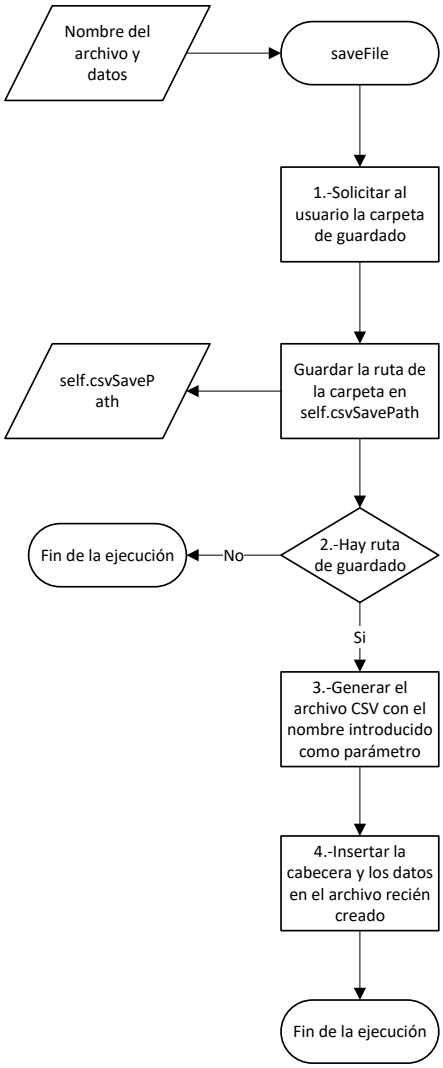


Figura 4.55: Diagrama de flujo que muestra el proceso de generación de un archivo CSV mediante `saveFile`.

Listado 4.45: Función `saveFile`

```
def saveFile(self):
    try:
        self.getSavePath()
        # print(self.saveFilePath)
        if self.csvSavePath != "":
            with open(self.csvSavePath, 'w', encoding='utf-8') as file:
                writer = self.csvWriter(file,delimiter=",",lineterminator="\n"
                )
                writer.writerow(self.columnsName)
                writer.writerows(self.csvData)
    except Exception as e:
        print("Error: " + str(e))
    finally: del self
```

4.4.1.4. Módulo de manejo de bases de datos

El módulo `SQLHandler` se encarga de gestionar la comunicación con las tres bases de datos utilizadas por el software. Estas bases de datos almacenan los datos de la oferta y la demanda para las simulaciones, así como los resultados generados por el simulador.

El módulo consta de cuatro clases: tres destinadas a la gestión individual de cada una de las bases de datos (`SqlSupply`, `SqlDemand` y `SqlResults`), y una última enfocada a la verificación de las sentencias SQL realizadas por los usuarios (`SqlTools`).

La clase `SqlSupply` es la encargada de gestionar la comunicación con la base de datos que almacena la información de los diferentes archivos de configuración de la oferta. Al utilizar esta clase, se genera una carpeta en el directorio de trabajo para almacenar la base de datos llamada "Database", si no existiera dicha carpeta, y dentro de esta, la base de datos con el nombre "supplyDb.db". Acto seguido, se genera una conexión con la base de datos para comenzar a construir la estructura de la misma. Esta estructura es la que se ha explicado en la Sección 4.3.1 y que aparece de manera simplificada en la Figura 4.8 y de manera más detallada en el Anexo A.1.

La creación de la base de datos comienza con la función `generateDataBaseTables`, que se encarga de ejecutar las subfunciones responsables de la creación de cada una de las tablas de la base de datos en un orden específico. El orden al generar las tablas es fundamental debido a que la creación de tablas con claves foráneas que referencien a otras requiere que las tablas referenciadas existan previamente. En caso contrario, la base de datos arrojará un error y esas tablas no se generarán. Debido a esto, primero se generan tablas que no contengan ninguna clave foránea y, después, se generan las tablas que contengan claves foráneas que apunten a tablas creadas con anterioridad. De esta forma se evitan errores durante la creación de la base de datos.

Las tablas se generan mediante una sentencia SQL, en la que se definen todas las columnas que ha de tener la tabla, así como el tipo de datos que almacena cada una de estas columnas y las diferentes restricciones que se le pueden aplicar, como, por ejemplo, que los valores que contenga cada columna sean únicos, es decir, que esa columna no puede contener dos filas que tengan el mismo valor en esa columna. Un ejemplo de una de estas subfunciones podría ser `createServiceTable` (Listado 4.46), que genera la tabla `SERVICE` (Figura 4.20). El diagrama de flujo de la Figura 4.56 refleja el comportamiento de la función `createServiceTable`. La función `createServiceTable` sigue el siguiente proceso:

1. *Inicializar cursor:* Se inicializa el cursor encargado de realizar las tareas de ejecución y recuperación de resultados.

2. *Ejecutar sentencia SQL:* Se comprueba que la tabla `SERVICE` no exista. Si la tabla no existe se ejecuta la sentencia SQL que genera la tabla `SERVICE`, aplica las restricciones que tenga la tabla, en este caso, que cada fila sea única y se añaden las referencias a las distintas claves foráneas que existen en la tabla.

3. *Guardar los cambios:* Una vez ejecutada la sentencia SQL, se guardan los cambios en la base de datos. Si la tabla existe se omite la ejecución de la sentencia SQL.

4. *Cerrar el cursor:* Por último se cierra el cursor de la base de datos.

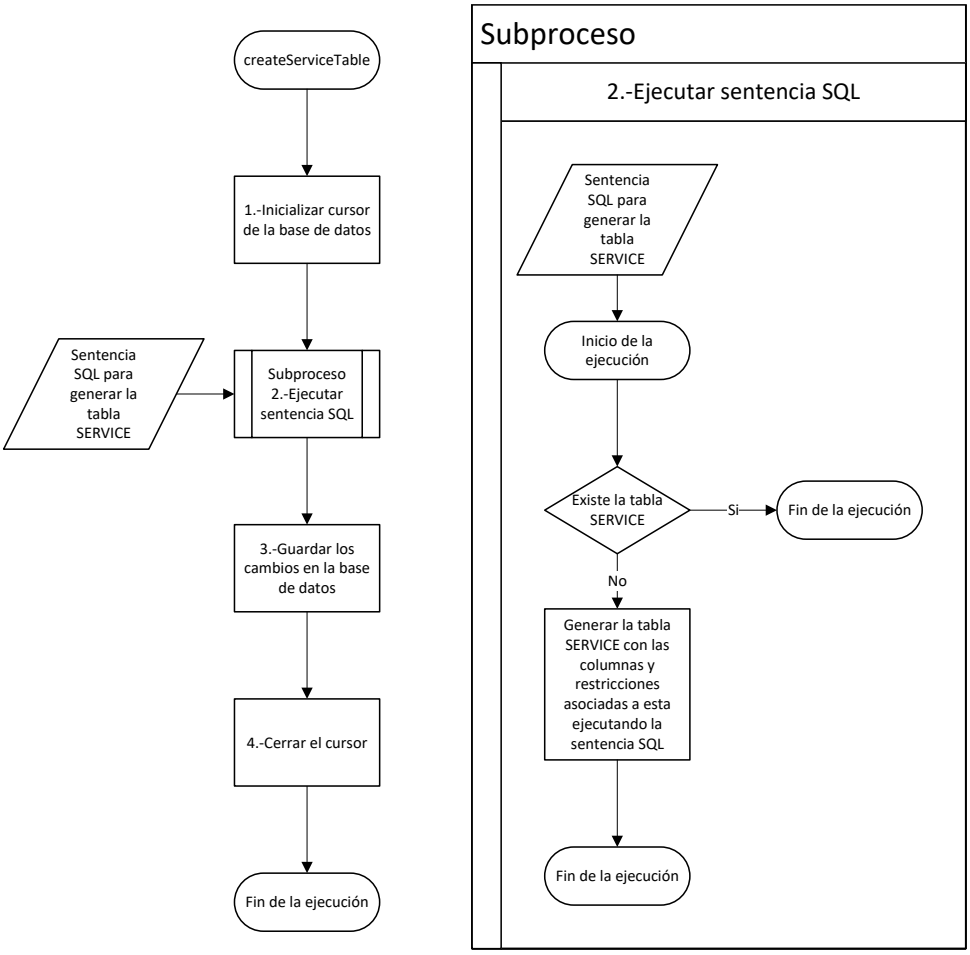


Figura 4.56: Diagrama de flujo para la creación de la tabla `SERVICE`.

Listado 4.46: Función `createServiceTable`

```
def createServiceTable(self):
    """
    Genera la tabla SERVICE en la base de datos, si no existe
    :return:
    """
    self.initCursor()
    try:
        query = """CREATE TABLE IF NOT EXISTS SERVICE (
                    ID                      TEXT PRIMARY KEY,

                    DATE                    DATE,

                    LINE                    TEXT REFERENCES LINE (ID)
                                            ON DELETE CASCADE ON UPDATE CASCADE,

                    TRAIN_SERVICE_PROVIDER  TEXT REFERENCES
                    TRAIN_SERVICE_PROVIDER (ID)
                                            ON DELETE CASCADE ON UPDATE CASCADE,

                    TIME_SLOT               TEXT REFERENCES TIME_SLOT (ID)
```

```
                                        ON DELETE CASCADE ON UPDATE CASCADE,

                    ROLLING_STOCK       TEXT REFERENCES ROLLING_STOCK (ID)
                                        ON DELETE CASCADE ON UPDATE CASCADE,

                    UNIQUE(ID,DATE,LINE,TRAIN_SERVICE_PROVIDER,TIME_SLOT,
                    ROLLING_STOCK)
                    );"""
            self.cursor.execute(query)
            self.conector.commit()
        except sqlite3.Error as e:
            print(e)
        except Exception as e:
            print(e)
        except sqlite3.ProgrammingError as e:
            print(e)
        finally:
            self.cursor.close()
```

Las demás subfunciones tienen un comportamiento similar, variando en cada caso la sentencia SQL para generar cada una de las tablas que componen la base de datos para los archivos de configuración de la oferta.

Dentro de la clase `SqlSupply` también se encuentran las funciones encargadas de insertar los datos en cada una de las tablas. Cada una de estas funciones ejecuta una sentencia SQL para introducir los datos, pasados como parámetro a la función, a la tabla para la que esté diseñada dicha función. Estos datos deben ser una lista de listas, donde cada una de las listas corresponde a una fila dentro de la tabla, por lo que la sentencia se ejecuta tantas veces como elementos tenga la lista de listas en una misma transacción con la base de datos. Esto acelera el proceso de introducir los datos, ya que se realiza una única transacción, evitando que las sentencias se ejecuten de una en una. En caso de que exista un problema a la hora de procesar la transacción, se deshacen todos los cambios realizados en la tabla producidos durante la transacción. Un ejemplo de una de estas funciones podría ser la función que introduce los datos a la tabla `SERVICE`.

La función `insertServiceData` (Listado 4.47) es la encargada de insertar los datos, que se le pasan como parámetro, a la tabla `SERVICE`. El funcionamiento de esta función se puede observar en el diagrama de flujo de la Figura 4.57. El procedimiento seguido por la función es:

1. *Inicializar el cursor:* Se inicializa el cursor de la base de datos. Este permite ejecutar las sentencias SQL.

2. *Generar la entrada:* Se genera una lista de listas donde cada una de las sublistas contiene los 6 primeros elementos de los datos introducidos como parámetro. Estos datos corresponden con las claves `id`, `date`, `line`, `train_service_provider`, `time_slot` y `rolling_stock` bajo la clave raíz `service`. Esta lista de listas está almacenada en la variable `inputData`.

3. *Insertar datos:* Se ejecuta la sentencia SQL para introducir los datos de `inputData` en la tabla `SERVICE` de la base de datos de los archivos de entrada de datos de la oferta. La sentencia SQL se ejecuta tantas veces como sublistas existan dentro de la lista de listas almacenada en `inputData`. Esto inserta una fila en la tabla cada vez que se ejecuta la sentencia SQL dentro de la transacción.

4. *Confirmar cambios:* Se confirman los cambios realizados en la tabla `SERVICE`, en este caso, la inserción de los datos pasados como parámetros a la función.

5. *Cerrar el cursor:* Por último se cierra el cursor de la base de datos.

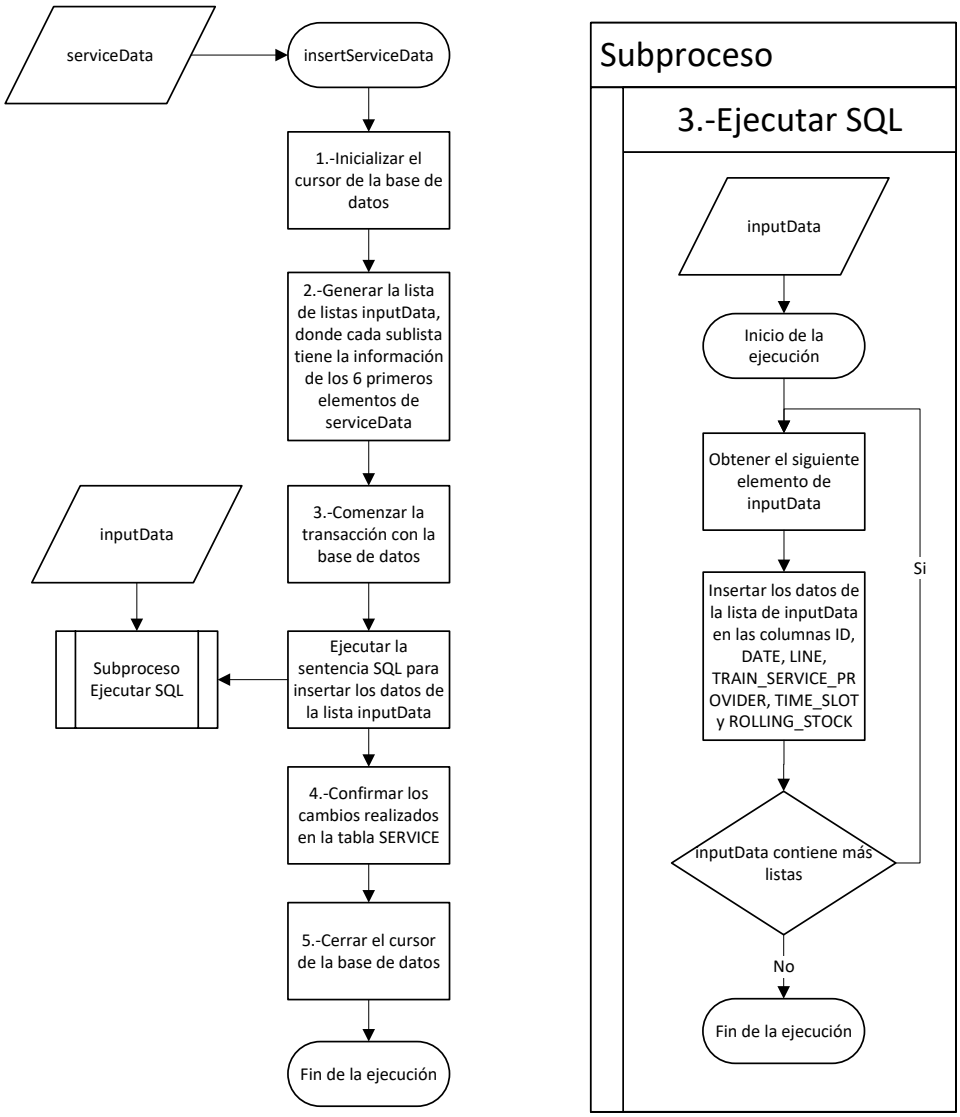


Figura 4.57: Diagrama de flujo de la función `insertServiceData`

Listado 4.47: Función `insertServiceData`

```
def insertServiceData(self, serviceData):
    """
    Inserta los datos de los servicios en la tabla SERVICE
    :param serviceData: Datos de los servicios
    :return:
    """
    self.initCursor()
    try:
        inputData = [service[:6]
                     for service in serviceData]
        query = "INSERT OR IGNORE INTO SERVICE (ID,DATE,LINE,
        TRAIN_SERVICE_PROVIDER,TIME_SLOT,ROLLING_STOCK) VALUES (?,?,?,?,?,?)"
        self.cursor.execute("BEGIN TRANSACTION")
```

```
            self.cursor.executemany(query, inputData)
            self.conector.commit()
        except sqlite3.Error as e:
            print("Service Sqlite Error SERVICE: "+str(e))
            self.conector.rollback()
        except Exception as e:
            print("Service Error: "+str(e))
            self.conector.rollback()
        except sqlite3.ProgrammingError as e:
            print("Service Sqlite programming Error: "+str(e))
            self.conector.rollback()
        finally:
            self.cursor.close()
```

Las otras funciones encargadas de insertar la información en las tablas de la base de datos siguen la misma dinámica que `insertServiceData`, pero cada una tiene una sentencia SQL similar, variando en cada caso la tabla en la que se quiere introducir la información y las columnas que van a almacenar dicha información.

En la clase `SqlSupply`, también se pueden encontrar las funciones encargadas de borrar los archivos y los datos asociados a estos. Empleando la función `deleteTestEntry` (Listado 4.48) se eliminan los archivos de una lista pasada como parámetro a la función. Además, esta función ejecuta una serie de subfunciones para eliminar la información que pueda permanecer en las tablas de la base de datos y que ya no tenga ningún test asociado, apoyándose en las tablas auxiliares para tal fin.

Una de las funciones encargadas de eliminar elementos que ya no tengan ningún archivo asociado es, por ejemplo, `deleteUnusedServiceData` (Listado 4.49). Esta función ejecuta una sentencia SQL que busca y elimina elementos huérfanos dentro de la base de datos destinada a archivos de entrada de datos de la oferta. El comportamiento de la función puede verse en el diagrama de flujo de la Figura 4.58. El proceso que sigue esta función es el siguiente:

1. *Ejecución de la sentencia SQL:* La función comienza la ejecución de la sentencia SQL.

2. *Selección de los servicios:* La base de datos busca todos los identificadores pertenecientes a los servicios dentro de la tabla `SERVICE`.

3. *Comprobar pertenencia:* Se comprueba que los identificadores recopilados en el paso anterior tengan asociados uno o más archivos. Esto se hace buscando en la tabla auxiliar `AUX_SERVICE` que todos los identificadores de los servicios recopilados anteriormente se encuentren presentes, ya que si se encuentran en dicha tabla, significa que tienen asociado mínimo un archivo. En caso de que existan elementos dentro de la tabla `SERVICE` que no tengan asociado un archivo, es decir, que no aparezcan en la tabla `AUX_SERVICE`, estos se eliminan de la tabla `SERVICE`. En el caso contrario, el programa para la ejecución sin realizar ningún cambio en la tabla `SERVICE`.

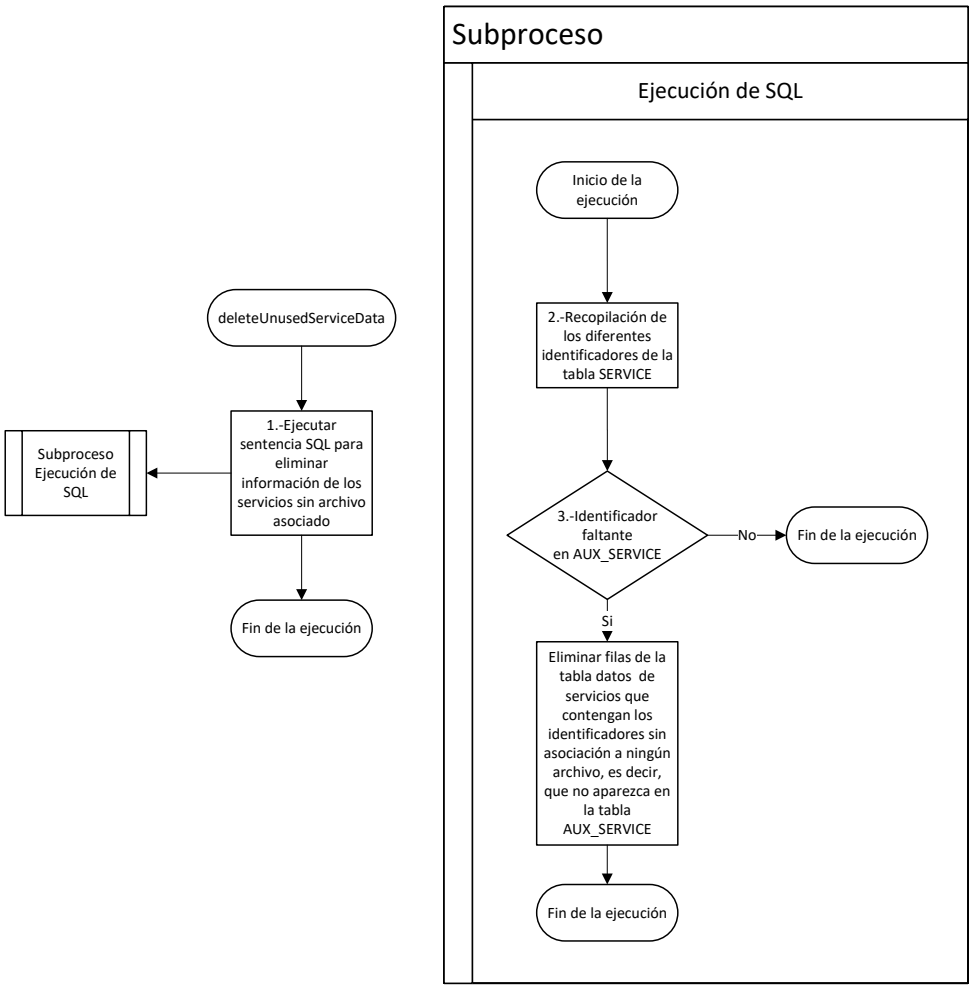


Figura 4.58: Diagrama de flujo de la función `deleteUnusedServiceData`

Listado 4.48: Función `deleteTestEntry`

```
def deleteTestEntry(self, testList):
    """
    Borra un test elegido por el usuario y todos los datos relacionados con este.
    Si los datos son comunes a otros
    tests, estos datos no se eliminarán.
    :return:
    """
    try:
        self.initCursor()
        for testName in testList:
            query = f"""DELETE FROM TESTS WHERE TESTS.NAME = '{testName}'"""
            self.cursor.execute(query)

        self.conector.commit()

        self.deleteUnusedSeatData()
        self.deleteUnusedServiceData()
        self.deleteUnusedCorridorData()
        self.deleteUnusedStationsData()
```

```
            self.deleteUnusedTrainServiceProviderData()
            self.deleteUnusedTimeSlotData()
            self.deleteUnusedRollingStockData()

        except sqlite3.Error as e:
            print("Sqlite Error: " + str(e))
        except EmptyTestData as e:
            print(e)
        except Exception as e:
            print(e)
        except sqlite3.ProgrammingError as e:
            print(e)
        finally:
            self.cursor.close()
```

Listado 4.49: Función `deleteUnusedServiceData`

```
def deleteUnusedServiceData(self):
    """
    Borra todos los datos que no se estén utilizando por ningún test en la tabla
    SERVICE
    :return:
    """
    try:
        query = f"""DELETE FROM SERVICE
                    WHERE SERVICE.ID NOT IN (
                    SELECT AUX_SERVICE.SERVICE_ID FROM AUX_SERVICE)"""
        self.cursor.execute(query)
        self.conector.commit()

    except sqlite3.Error as e:
        print("Sqlite Error: " + str(e))
    except Exception as e:
        print(e)
    except sqlite3.ProgrammingError as e:
        print(e)
```

La clase `SqlSupply` también posee funciones que extraen los datos de la base de datos para, posteriormente, reconstruir el archivo de entrada de datos de la oferta. Estas funciones convierten los datos extraídos al mismo formato que adoptan los archivos Yaml al ser cargados en Python mediante la librería PyYaml [5], es decir, un diccionario.

La función principal para la extracción de los datos y posterior construcción del diccionario para recrear el archivo Yaml es `getDataForDumpYaml` (Listado 4.50), que toma como parámetro el nombre del archivo que se va a reconstruir. Este nombre se emplea posteriormente por las subfunciones encargadas de reconstruir la información de cada una de las claves raíz que aparecen en los archivos de configuración de la oferta para extraer los datos que correspondan a ese test.

Listado 4.50: Función `getDataForDumpYaml`

```
def getDataForDumpYaml(self,testName):
    """
```

```
        Genera la estructra necesaria para reconstruir el archivo de oferta mediante
        subfunciones para cada
        uno de los apartados dentro del archivo yaml.
        :return: testName, data
        """
        data = {}
        try:
            self.initCursor()

            data.update(self.getDataForStations(testName))
            data.update(self.getDataForSeat(testName))
            data.update(self.getDataForCorridor(testName))
            data.update(self.getDataForLine(testName))
            data.update(self.getDataForRollingStock(testName))
            data.update(self.getDataForTrainServiceProvider(testName))
            data.update(self.getDataForTimeSlot(testName))
            data.update(self.getDataForService(testName))

        except sqlite3.Error as e:
            print("Sqlite Error: " + str(e))
        except Exception as e:
            print(e)
        except sqlite3.ProgrammingError as e:
            print(e)
        finally:
            self.cursor.close()
            return data
```

Una de las subfunciones encargadas de reconstruir la información de las claves raíz es la función `getDataForTimeSlot` (Listado 4.51). El comportamiento de esta función se muestra en el diagrama de flujo de la Figura 4.59. El procedimiento seguido por la función es:

1. *Ejecución de sentencia SQL:* Se ejecuta la sentencia que, teniendo el nombre del archivo del archivo que se quiere regenerar, obtiene todos los datos asociados a este mediante la tabla auxiliar `AUX_TIME_SLOT`. Con esta tabla, se obtienen los identificadores de los datos de la tabla `TIME_SLOT` que pertenecen al archivo. Se extraen las filas que contengan dichos identificadores y se ordenan de manera ascendente por el valor del identificador.

2. *Recolección de datos:* Se recogen los datos seleccionados por la base de datos en el paso anterior. Estos datos están almacenados en una lista de tuplas, donde cada tupla corresponde con una fila de los datos seleccionados con la sentencia SQL.

3. *Construcción de salida:* Con los datos obtenidos, se construye una lista de diccionarios donde cada diccionario posee las claves `id`, `start` y `end`, corresponden con los elementos 0, 1 y 2 de cada una de las tuplas que conforman la lista de tuplas con los datos extraidos de la base de datos. Posteriormente, esta lista de diccionarios se añade a un diccionario bajo la clave `timeSlot`, que es una de las claves raíz del archivo de entrada de datos de la oferta.

4. *Retorno de salida con formato:* Una vez construido el diccionario con los datos solicitados a la base de datos, este se devuelve y, por ende, finaliza la ejecución de la función.

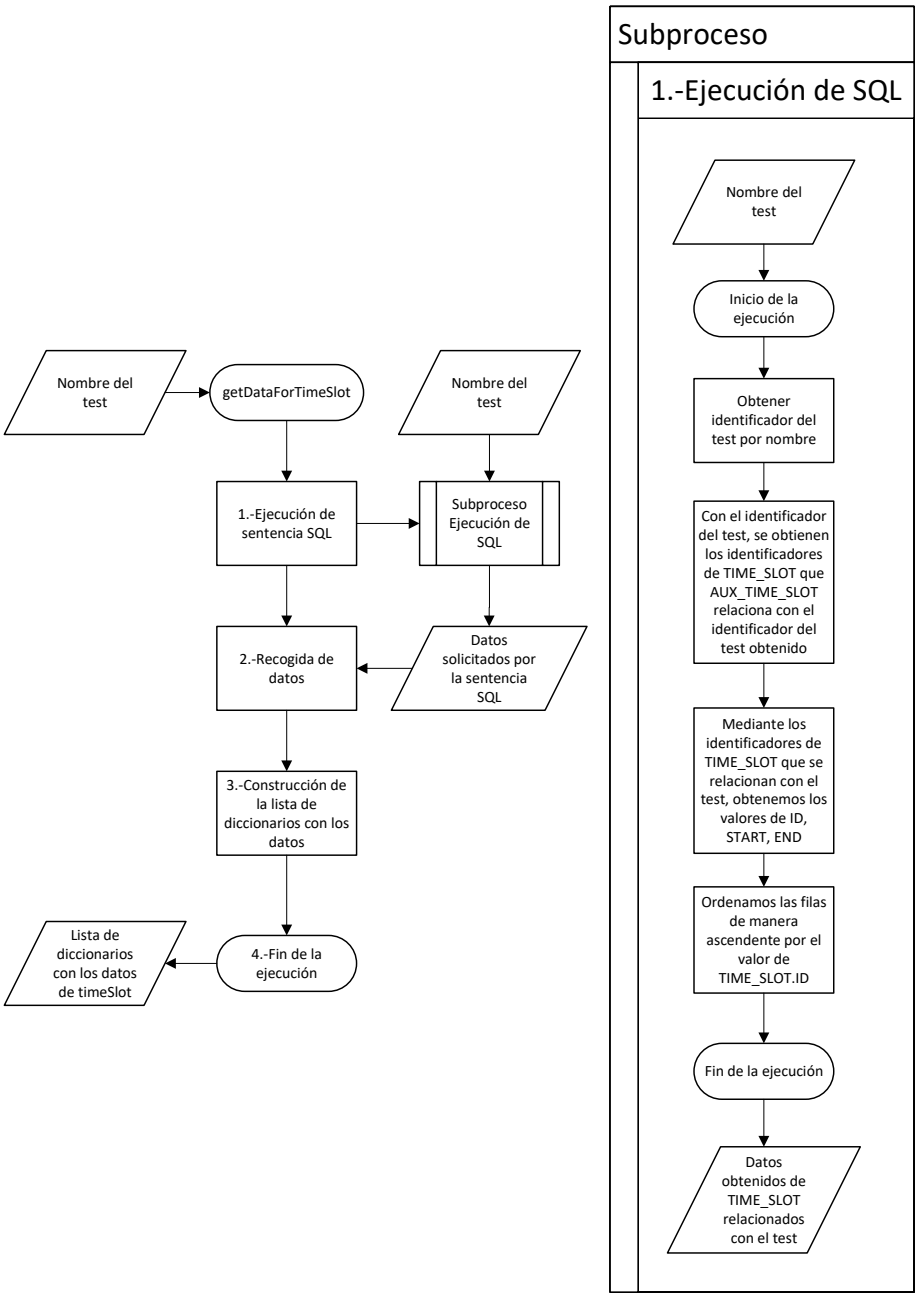


Figura 4.59: Diagrama de flujo de la función `getDataForTimeSlot`

Listado 4.51: Función `getDataForTimeSlot`

```
def getDataForTimeSlot(self, testName):
    """
    Obtiene los datos de los slots de tiempo y los formatea para que sigan la
    estructra de timeSlot dentro del archivo
    yaml de la oferta
    :param testName: Nombre del test que se está extrayendo
    :return:
    """
    try:
        query = f"""SELECT
```

```
                    TIME_SLOT.ID,
                    TIME_SLOT.START,
                    TIME_SLOT.END
                FROM
                    TIME_SLOT
                WHERE TIME_SLOT.ID IN (
                    SELECT AUX_TIME_SLOT.TIME_SLOT_ID FROM AUX_TIME_SLOT
                    INNER JOIN TESTS ON TESTS.ID = AUX_TIME_SLOT.TEST_ID
                    WHERE TESTS.NAME = '{testName}')
                ORDER BY TIME_SLOT.START ASC"""

            self.cursor.execute(query)
            data = self.cursor.fetchall()
            formatedData = [{"id": element[0],"start": element[1],"end": element[2]}
                            for element in data]
            outputData = {"timeSlot": formatedData}
            return outputData
        except sqlite3.Error as e:
            print("Sqlite Error: " + str(e))
        except Exception as e:
            print(e)
        except sqlite3.ProgrammingError as e:
            print(e)
```

En esta clase `SqlSupply` también se encuentra la función `executeQuery` (Listado 4.52) encargada de ejecutar las sentencias, que se pasan como parámetro, en la base de datos para los archivos de la oferta. Esta función puede ejecutar cualquier sentencia válida en la base de datos. Si esta sentencia selecciona datos empleando el comando `SELECT`, la función devuelve los nombres de las columnas que han sido seleccionadas, así como la información contenida que cumpla con las especificaciones introducidas en la sentencia. Un ejemplo de esto puede ser el uso de la sentencia `SELECT ID, NAME FROM STATIONS`. Esta sentencia devuelve el identificador y el nombre de cada una de las estaciones que se encuentren en la tabla `STATIONS` (Figura 4.11) por lo que el valor de la variable `cols` es una lista con los nombres `ID` y `NAME`, que corresponden a los nombres de las columnas de las que se quieren obtener datos, mientras que la variable `result` contiene una lista de tuplas donde cada una de las tuplas sería una fila de la base de datos con dos elementos: el identificador de la estación y el nombre que recibe la misma.

Listado 4.52: Función `executeQuery`

```
def executeQuery(self,query):
    try:
        self.initCursor()
        self.cursor.execute(query)
        self.conector.commit()

        strippedQuery = query.strip().upper()

        if strippedQuery.startswith("SELECT"):
            cols = [item[0] for item in self.cursor.description]
            result = self.cursor.fetchall()
            return cols,result
```

```
        except sqlite3.Error as e:
            print(e)
            return -1
        except Exception as e:
            print(e)
            return -1
        except sqlite3.ProgrammingError as e:
            print(e)
            return -1
        finally:
            self.cursor.close()
```

En cuanto a las clases `SqlDemand` y `SqlResults`, estas contienen funciones que realizan las mismas tareas que las descritas en la clase `SqlSupply`, es decir, crear las tablas que conforman las bases de datos para los archivos de entrada de datos de la demanda y para los archivos de resultados de las simulaciones, insertar datos en dichas tablas, eliminar archivos de las bases de datos y ejecutar sentencias SQL en sus respectivas bases de datos. Dichas funciones están adaptadas a cada base de datos, en el caso de las funciones dentro de la clase `SqlDemand`, están adaptadas para funcionar con la base de datos que almacena los datos de los archivos de entrada de datos de la demanda y en el caso de las funciones dentro de la clase `SqlResults`, están adaptadas para funcionar con la base de datos cuyo propósito es el de almacenar los resultados de las diferentes simulaciones que se realicen con el simulador.

Por último, la clase `SqlTools` (Listado 4.53) posee funciones cuyo cometido es el de validar las sentencias antes de que se ejecuten en las bases de datos, evitando la ejecución de aquellas que contengan errores. Esta clase contiene dos funciones:

- La función `validateSyntaxQuery` está destinada a validar la sintaxis de la sentencia utilizando el comando SQL `EXPLAIN`, el cual permite obtener el plan de ejecución sin ejecutar realmente los comandos, en una base de datos temporal en memoria que se encuentra vacía.
- La función `validateQueryOnDb` se encarga de validar la sentencia en la base de datos donde se va a ejecutar, con el fin de evitar errores que no sean de sintaxis, como por ejemplo, que una de las columnas mencionadas en la sentencia no existe. Esta función también emplea el comando `EXPLAIN` para llevar a cabo la validación.

Listado 4.53: Clase `SqlTools`

```
class SqlTools:
    def __init__(self):
        self.connector = sqlite3.connect(":memory:")

    def validateSyntaxQuery(self,query):
        try:
            self.connector.execute(f"EXPLAIN QUERY PLAN {query}")
            return True

        except sqlite3.OperationalError as e:
            if "no such table" in str(e):
                return True
            else:
                print(e)
                return False
```

```
            except sqlite3.Error as e:
                print(f"Query invalida: {e}")
                return False

        def validateQueryOnDb(self,query,conn):
            try:
                conn.execute(f"EXPLAIN QUERY PLAN {query}")
                return True, ""

            except sqlite3.OperationalError as e:
                print(f"Query invalida: {e}")
                return False, str(e)
            except sqlite3.Error as e:
                print(f"Query invalida: {e}")
                return False , str(e)
```

4.4.1.5. Módulo de introducción de datos

El módulo `dataLoger` está enfocado a introducir los datos de los diferentes archivos dentro de la base de datos, empleando para ello las funciones de los módulos `yamlParser` y `SQLHandler`. Este módulo contiene una clase dedicada a introducir la información dentro de cada base de datos manejada por la aplicación, una para la base de datos de los archivos de entrada de datos de la oferta, otra para la base de datos de los archivos de entrada de datos de la demanda y otra para la base de datos de los archivos de resultados arrojados por el simulador.

Cada una de las clases que componen este módulo posee una función principal desde la cual se ejecutan todas las demás funciones encargadas de añadir la información a las bases de datos. Estas subfunciones emplean funciones del módulo encargado de la comunicación con las bases de datos para insertar los datos de cada uno de los archivos en su correspondiente tabla dentro de la base de datos diseñada para ese archivo.

Un ejemplo de estas subfunciones es la función `logStationData`. Esta función primero extrae los datos de la clave raíz `stations` empleando la función `getStationsData` perteneciente a la clase `Parser` del módulo `yamlParser`.

Una vez obtenidos los datos de la clave raíz `stations` se emplean las funciones destinadas a introducir los datos en la base de datos, en este caso, en la tabla `STATIONS` de la base de datos de los archivos de entrada de datos de la oferta, que se encuentran en la clase `SqlSupply` dentro del módulo `SQLHandler`.

Listado 4.54: Función `logStationData`

```
def logStationsData(self):
    """
    Introduce los datos de las estaciones a la base de datos de la oferta
    :return:
    """

    data = self.yml.getStationsData()
    self.sqlSupply.insertStationsData(data)
    self.sqlSupply.insertAuxStationsData(data, self.testID)
```

4.4.1.6. Módulo de configuración

El módulo `configManager` se encarga de gestionar la configuración del programa, concretamente, de las sentencias SQL almacenadas para su uso posterior. Estas sentencias se guardan en un archivo JSON.

El archivo de configuración se genera en la carpeta `config`, ubicada dentro del directorio de trabajo del programa, con el nombre `config.json`. Tanto el directorio como el archivo se crean automáticamente al iniciar la aplicación por primera vez o si, al arrancarla, el archivo no existe. Este comportamiento se define en la generación de la clase `Config` dentro del módulo `configManager`.

Por defecto, el archivo de configuración se genera con un conjunto de sentencias diseñado para mostrar los archivos almacenados en cada base de datos. Estas sentencias se guardan bajo la clave `SQL_Querys` dentro de una lista de diccionarios. Las claves de estos diccionarios corresponden al nombre asignado a cada sentencia en el archivo de configuración, y su valor es otro diccionario que contiene dos claves: `db` y `query`, las cuales indican la base de datos en la que se ejecutará la sentencia y la propia sentencia a ejecutar, respectivamente.

Esto se realiza mediante la función `generateDefaultConfig` (Listado 4.55).

Listado 4.55: Función `generateDefaultConfig`

```
def generateDefaultConfig(self):
    """
    Esta funcion genera la configuracion inicial para la aplicacion, la cual
    contiene querys de SQL utiles
    y bastante usadas en las bases de datos que maneja el programa.
    :return: None
    """
    try:
        data = {
            'SQL_Querys':{
                "Mostrar test de oferta":{"db":"oferta","query":"SELECT * FROM
                TESTS"},
                "Mostrar test de demanda": {"db": "demanda", "query": "SELECT *
                FROM TESTS"},
                "Mostrar test de resultados": {"db": "resultados", "query": "
                SELECT * FROM TESTS"}
            }
        }
        self.configData = deepcopy(data)
        with open(self.configFilePath,'w') as configFile:
            json.dump(data,configFile,indent=4)
    except Exception as e:
        print(e)
```

Mediante el uso de `addSQLQuery` (Listado 4.56) se pueden agregar sentencias al archivo de configuración. Esta función tomará como parámetros el nombre que recibe la sentencia SQL, que servirá como clave dentro del diccionario que almacena las sentencias, la base de datos en la que se ejecutará la sentencia y la sentencia SQL en sí. Con estos parámetros se genera una entrada dentro del diccionario y se introduce una clave nueva, que corresponde con el nombre que se ha pasado como argumento y, bajo esta clave, se genera otro diccionario con las claves "db" y "query", donde se guardan la base de datos donde se va a ejecutar la sentencia y la sentencia SQL respectivamente.

Listado 4.56: Función `addSQLQuery`

```
def addSQLQuery(self,name,db,query):
    """
    Esta funcion se encarga de agregar querys de SQL a la lista almacenada en el
    archivo de configuracion
    :return: None
    """
    try:
        self.configData["SQL_Querys"].update({str(name):{"db":str(db),"query":str(
        query)}})
        self.saveConfig()
    except Exception as e: print(e)
```

De igual modo, usando la función `removeSQLQuery` (Listado 4.57) se eliminan sentencias SQL que se encuentran ya almacenadas en el archivo de configuración. Para ello, esta función se vale del nombre que se le ha asignado a la sentencia SQL para eliminar la entrada que contiene la sentencia SQL dentro del archivo de configuración.

Listado 4.57: Función `removeSQLQuery`

```
def removeSQLQuery(self,name):
    """
    Esta funcion se encarga de eliminar querys de SQL a la lista almacenada en el
    archivo de configuracion
    :return: None
    """
    try:
        self.configData["SQL_Querys"].pop(name)
        self.saveConfig()
    except Exception as e: print(e)
```

4.4.2. Interfaz gráfica

Para construir la interfaz gráfica del programa se ha empleado el paquete Tkinter [13] incluido en Python. Este paquete actúa como una envoltura sobre la biblioteca Tcl/Tk, escrita en C, permitiendo a Python comunicarse con el intérprete Tool Command Language (Tcl) mediante el módulo interno `_tkinter`. Cuando se emplea Tkinter, lo que realmente ocurre es que Python genera una cadena de comandos Tcl, que mediante el intérprete de Tcl, invoca a las funciones del sistema gráfico a través de Tk o Ttk. Estos gestionan directamente los elementos de la interfaz a través del sistema gráfico del sistema operativo.

Tkinter es multiplataforma, es decir, puede funcionar en varios sistemas operativos sin necesidad de cambiar el código, en la mayoría de las ocasiones, ya que se encuentra implementado sobre Application Programming Interface (API) nativas de cada sistema operativo, Graphics Device Interface (GDI) y USER32 para sistemas con Windows, Cocoa o Quartz para sistemas con MacOS y X11 para sistemas Linux/Unix.

A continuación, se desarrollará una explicación de algunos de los elementos de Tkinter empleados en la construcción de la interfaz para la aplicación que gestiona el flujo de datos entre los archivos y las bases de datos que componen este TFG, apoyándose en un programa de ejemplo que contiene un contador y un generador de ventanas de mensajes. Esto se ha decidido así debido a que el entorno tiene muchos más elementos que complicarían la explicación,

mientras que con un ejemplo más sencillo se puede entender el funcionamiento de Tkinter y cómo se insertan y manejan los diferentes elementos.

4.4.2.1. Ejemplo de uso de Tkinter

El ejemplo se compone de una clase llamada `UI` que se encarga de la gestión de la presentación de la ventana, así como del comportamiento del programa y en la que se encuentran todas las funciones que hacen que el programa tenga el funcionamiento esperado.

La construcción de la interfaz comienza con la declaración de la clase `UI`, mediante su invocación en el script. Esta invocación hace que la clase `UI` ejecute su constructor, definido en la función `__init__` (Listado 4.58).

Listado 4.58: Función `__init__` del ejemplo de Tkinter

```
def __init__(self):
    self.counter = 0
    self.root = tk.Tk()
    self.root.withdraw()
    self.root.title("Ejemplo Tkinter")
    self.root.resizable(False, False)  # Restriccion del redimensionamiento de la
    ventana principal
    self.root.protocol("WM_DELETE_WINDOW",
                       self.onCloseEvent)
    self.init_ui()  # Inicializacion de la interfaz
    self.root.deiconify()
    self.root.mainloop()
```

En este constructor se definen todas las variables de la clase y se generan los elementos que componen la ventana del programa, como el nombre de la ventana, dimensiones de la ventana, el comportamiento de la ventana al cerrarla, etc. El elemento principal de la interfaz se encuentra en la variable `self.root` y de este dependen todos los elementos de la interfaz. Empleando el elemento principal de la interfaz, en el caso de este ejemplo, se define el título de la ventana, empleando el método `title`, el ajuste que desactiva la posibilidad de modificar el tamaño de la ventana, ya sea en altura o en anchura. También está definido el comportamiento de la ventana al usar el botón para cerrar la ventana, usando el método `protocol` pasando a dicho método el nombre del evento, en este caso `WM_DELETE_WINDOW`, al que se quiera asociar una función, en este caso `onCLoseEvent` (Listado 4.59), que se ejecuta al cerrar la ventana. Esta función destruye el elemento principal, lo que cierra la ventana y detiene la ejecución del programa.

Listado 4.59: Función `onCloseEvent` del ejemplo de Tkinter

```
def onCloseEvent(self):
    self.root.destroy()
```

La función `init_ui` (Listado 4.60) tiene como cometido construir todos los elementos que componen la interfaz, como botones, contenedores, entradas de texto, etc. En este caso concreto, se define el contenedor principal, sobre el que se construyen los demás elementos, los marcos con etiqueta que contienen los elementos para el contador y para el generador de ventanas de información. El contenedor del contador alberga una etiqueta, que es la que muestra el valor del contador, y tres botones: uno para sumar 1 al valor del contador, otro para restar 1 al valor del contador y, por último, un botón para reiniciar el contador a 0. Por otra parte,

el contenedor del generador de ventanas de información contiene: una entrada de texto de una sola línea para el título de la ventana, con su correspondiente etiqueta para indicar que es la entrada del título, una entrada de texto para el mensaje que se mostrará en la ventana que, además, cuenta con su etiqueta correspondiente que indica que se trata del cuerpo del mensaje, y, por último, el botón que genera la ventana al presionar dicho botón.

Listado 4.60: Función `init_ui` del ejemplo de Tkinter

```
def init_ui(self):
    # Contenedor principal
    self.mainFrame = tk.Frame(self.root)
    self.mainFrame.grid(row=0, column=0, padx=5, pady=5, sticky="nsew")

    # Contenedor para el contador
    self.counterFrame = tk.LabelFrame(self.mainFrame, text="Contador")
    self.counterFrame.grid(row=0, column=0, padx=5, pady=5, sticky="nsew")

    # Contenedor para para mostrar el valor del contador
    self.labelForCounter = tk.Label(self.counterFrame,text=f"Valor del contador: {
    self.counter}")
    self.labelForCounter.grid(row=0,column=0, padx=5, pady=5)

    self.addButton = tk.Button(self.counterFrame, text="Añadir 1", command= lambda
    : self.addOneToCounter())
    self.addButton.grid(row=0, column=1, padx=5, pady=5, sticky="nsw")

    self.diffButton = tk.Button(self.counterFrame, text="Restar 1", command=
    lambda: self.diffOneToCounter())
    self.diffButton.grid(row=0, column=2, padx=5, pady=5, sticky="nsw")

    self.resetButton = tk.Button(self.counterFrame, text="Reiniciar contador",
    command= lambda: self.resetCounter())
    self.resetButton.grid(row=0, column=3, padx=5, pady=5, sticky="nsw")

    # Contenedor para el generador de ventanas de mensaje
    self.messageFrame = tk.LabelFrame(self.mainFrame, text="Generar ventana de
    informacion")
    self.messageFrame.grid(row=1, column=0, padx=5, pady=5, sticky="nsew")

    self.titleLabel = tk.Label(self.messageFrame, text="Titulo del mensaje:")
    self.titleLabel.grid(row=0, column=0, padx=5, pady=5, sticky="nsw")

    # Entrada de línea de texto para el título de la ventana
    self.titleEntry = tk.Entry(self.messageFrame, width=66)
    self.titleEntry.grid(row=1, column=0, padx=5, pady=5)

    self.textLabel = tk.Label(self.messageFrame, text="Cuerpo del mensaje:")
    self.textLabel.grid(row=2, column=0, padx=5, pady=5, sticky="nsw")

    # Entrada de texto para el cuerpo del mensaje de la ventana
    self.messageText = tk.Text(self.messageFrame,width=50, height=8)
    self.messageText.grid(row=3, column=0, padx=5, pady=5)
```

```
self.showButton = tk.Button(self.messageFrame, text="Generar ventana", command
=
lambda:self.generateMsgInfoWinodw())
self.showButton.grid(row=4, column=0, padx=5, pady=5, sticky="nsew")

self.root.update_idletasks()
width = self.root.winfo_reqwidth()
height = self.root.winfo_reqheight()
screenHeight = self.root.winfo_screenheight()
screenWidth = self.root.winfo_screenwidth()
posX = (screenWidth - width) // 2
posY = (screenHeight - height) // 2
self.root.geometry(f"{width}x{height}+{posX}+{posY}")
```

El contenedor principal, que depende del elemento principal de la interfaz, está almacenado en la variable `self.mainFrame` y para crearlo se emplea el método `Frame` de Tkinter. Esto genera un espacio en el que se pueden colocar los demás elementos que componen la interfaz, ya sean botones, entradas de texto, etiquetas o incluso otros contenedores. Después, para colocarlo en su posición correspondiente dentro de la ventana, se ha usado el método `grid`, tanto en este ejemplo como en la interfaz del programa objeto de este TFG de la que se hablará más adelante. Este permite colocar los elementos en una estructura de filas y columnas, como si de una tabla se tratara.

Además del método `grid`, existen otros métodos, como `pack`, que organiza los elementos de forma automática dentro del contenedor, o como `place`, que coloca los elementos en una posición específica empleando sus coordenadas exactas dentro del contenedor o ventana.

El contenedor principal se ha colocado en la primera columna (`column=0`) de la primera fila (`row=0`) de la ventana, con una separación de los bordes en el eje horizontal de 5 píxeles (`padx=5`) y una separación de los bordes en el eje vertical de 5 píxeles (`pady=5`) y se encuentra centrado dentro de la celda en la que se ubica (`sticky="nsew"`).

El contenedor con etiqueta en el que se encuentra el contador se define mediante el método `LabelFrame`. Esto genera, como en el caso de `Frame`, un espacio en el que colocar los elementos de la interfaz, pero en este caso además, incluye una etiqueta y un borde que indican a qué se destina dicho contenedor, en este caso, al contador y los botones que lo controlan. Este contenedor con etiqueta depende del contador principal. Al igual que el contenedor principal, este elemento se ha ubicado en su posición usando el método `grid` y ha sido ubicado en la primera columna de la primera fila, con una separación en los ejes horizontal y vertical de 5 píxeles y se encuentra centrado en dicha celda.

Dentro de este contenedor se encuentra la etiqueta que sirve para mostrar qué valor posee el contador en cada momento. Esta etiqueta se define mediante el método `Label`, en el que se indica a qué contenedor pertenece, en este caso al contenedor del contador (`self.counterFrame`), y qué texto tiene que mostrar. La etiqueta se encuentra en la primera columna de la primera fila, separada en los ejes horizontal y vertical 5 píxeles y centrada en el eje vertical, pero se encuentra alineada a la izquierda en el eje horizontal.

También, dentro de este contenedor, se encuentran los botones que controlan el contador, los cuales se definen usando `Button`, en el que se indica el contenedor al que pertenece, el texto que aparece en el botón y, por último, la función que se ejecuta al presionar el botón, en el caso del botón que suma 1 al contador, `addOneToCounter`, en el caso del botón que resta 1 al contador, `diffOneToCounter`, y en el caso del botón que reinicia el contador a 0, `resetCounter`.

El contenedor con etiqueta en el que aparecen las entradas de texto, las etiquetas para las

entradas de texto y el botón con los que se generará la ventana de información se encuentra definido también con `LabelFrame`. Este también depende del contenedor principal, al igual que el del contador, pero en la etiqueta aparece el texto "Generar ventana de información".

La entrada de texto para el título de la ventana se define mediante el método `Entry` en el que se especifican el contenedor al que pertenece y la anchura de la entrada de texto.

La entrada de texto para el cuerpo del mensaje se define mediante el método `Text` en el que se especifican el contenedor del que depende, la anchura y la altura del área de entrada de texto.

Por último, el botón que genera la ventana ejecuta la función `generateMsgInfoWinodw` (Listado 4.64) al ser presionado.

Finalmente, la última parte de la función `init_ui` se encarga de redimensionar la ventana y centrarla en la pantalla, de modo que la aplicación tenga el tamaño adecuado para mostrar todos los elementos. Para ello, se comienza llamando al método `update_idletasks` de `self.root`, lo que permite actualizar todas las tareas pendientes y obtener las dimensiones reales que requieren los elementos dispuestos en la ventana (medidas con `winfo_reqwidth` y `winfo_reqheight`). Posteriormente, se extraen las dimensiones totales de la pantalla mediante `winfo_screenwidth` y `winfo_screenheight`, lo que posibilita calcular las coordenadas (`posX` y `posY`) necesarias para centrar la ventana. Finalmente, con el método `geometry`, se asigna tanto el tamaño final como la posición exacta de la ventana, garantizando que ésta se ajuste perfectamente a su contenido y se visualice centrada.

La Figura 4.60 muestra la interfaz resultante de la ejecución del método `init_ui` de la clase `UI`. Tal como se ha explicado en los párrafos anteriores, en esta interfaz se aprecian dos contenedores principales. En primer lugar, el contenedor del contador, que agrupa la etiqueta destinada a mostrar el valor actual del contador y los botones para sumar, restar o reiniciar su valor, dispuestos con márgenes de 5 píxeles en ambos ejes y con la alineación especificada en cada uno de ellos.

En segundo lugar, se encuentra el contenedor que contiene las entradas de texto y los botones para generar la ventana de información. En este contenedor se definen la entrada para el título (usando `Entry`), el área de texto para el cuerpo del mensaje (definida con `Text`) y, además, un botón que activa la función `generateMsgInfoWinodw`.

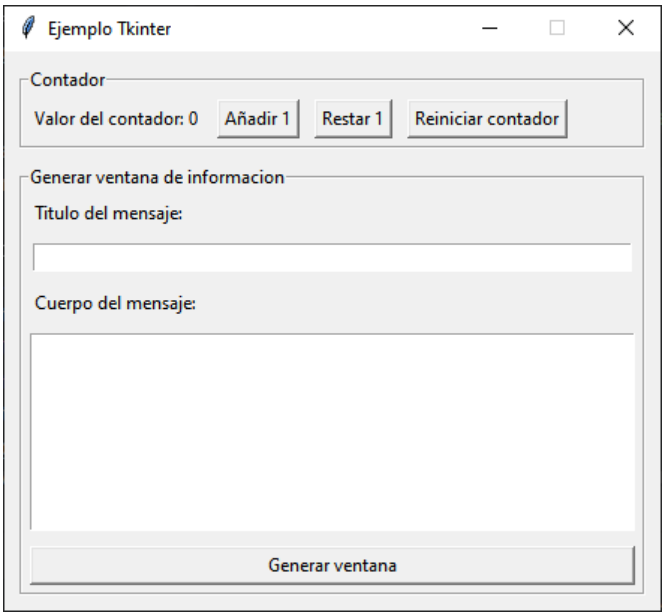


Figura 4.60: Ventana de la aplicación de ejemplo

Teniendo la interfaz de la aplicación de ejemplo, ya se puede comprobar que el funcionamiento sea el esperado, es decir, que todos los botones invoquen correctamente las funciones asociadas a ellos.

Primero se prueba que el botón del contador que añade unidades a este, invoque correctamente a la función que tiene asociada, es decir, la función `addOneToCounter` (Listado 4.61) y que sume 1 al contador cada vez que este botón se pulse.

La función `addOneToCounter` primero suma 1 al valor del contador en el momento de su invocación. Tras esto, actualiza el texto de la etiqueta que muestra el valor del contador y, por último, actualiza toda la interfaz para que los cambios realizados sean visibles para el usuario.

Listado 4.61: Función `addOneToCounter` del ejemplo de Tkinter

```
def addOneToCounter(self):
    self.counter+=1
    self.labelForCounter.config(text=f"Valor del contador: {self.counter}")
    self.root.update_idletasks()
```

Para la prueba se ha pulsado 5 veces el botón "Añadir 1" en la interfaz, cuyo resultado puede apreciarse en la Figura 4.61.

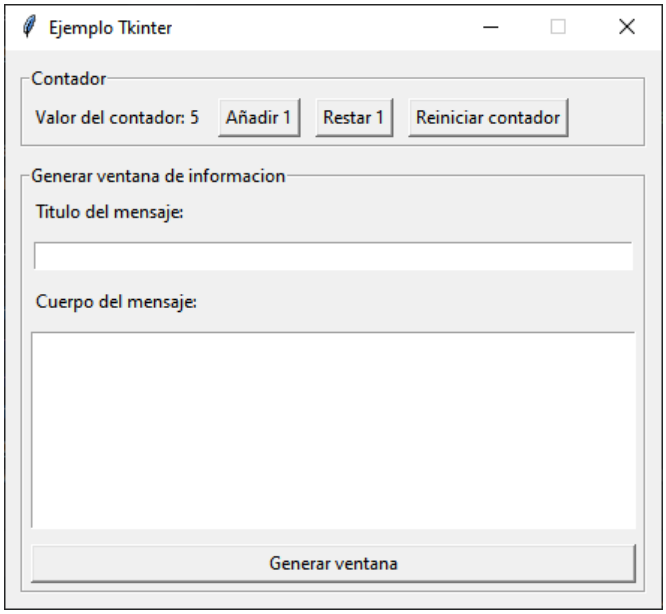


Figura 4.61: Prueba del botón "Añadir 1"

Después, se ha comprobado que el botón "Resta 1" llame correctamente a la función que tiene asociada, en este caso `diffOneToCounter` (Listado 4.62). Esta función funciona de forma similar a la anterior, pero en lugar de sumar 1 al valor del contador que posea en el momento en el que se pulsa el botón, esta función resta 1 al valor actual del contador, cambia el texto que aparece en la etiqueta y actualiza toda la interfaz para que se reflejen los cambios.

Listado 4.62: Función `diffOneToCounter` del ejemplo de Tkinter

```
def diffOneToCounter(self):
    self.counter-=1
    self.labelForCounter.config(text=f"Valor del contador: {self.counter}")
    self.root.update_idletasks()
```

Para comprobar que funciona, se ha pulsado una vez el botón que resta 1 al contador cuando el contador tenía un valor de 5, como se muestra en la Figura 4.61. Después de haber presionado el botón, el valor del contador baja a 4, como se puede ver en la Figura 4.62

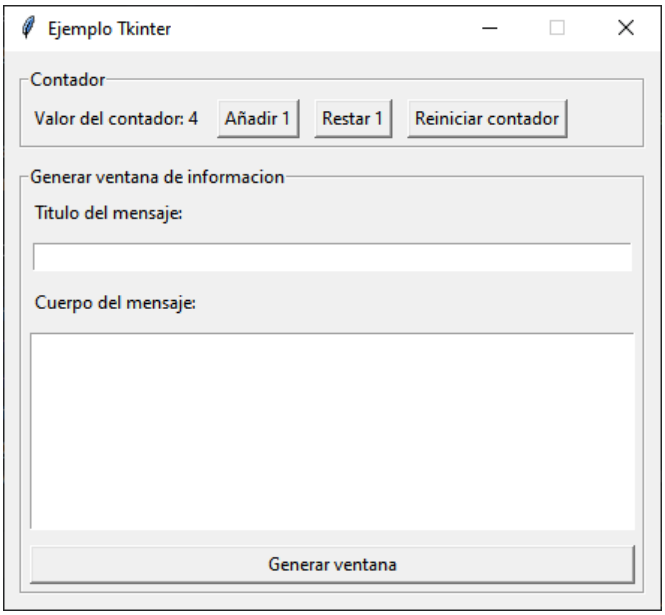


Figura 4.62: Prueba del botón "Restar 1"

Tras esto, se ha comprobado el funcionamiento del botón que reinicia el contador y lo pone a 0. Este botón lanza la función `resetCounter` (Listado 4.63), que establece el valor de la variable que almacena el valor del contador a 0, actualiza la etiqueta que muestra el valor del contador y actualiza la ventana para que se reflejen todos los cambios realizados.

Listado 4.63: Función `resetCounter` del ejemplo de Tkinter

```
def resetCounter(self):
    self.counter = 0
    self.labelForCounter.config(text=f"Valor del contador: {self.counter}")
    self.root.update_idletasks()
```

Una vez pulsado el botón "Reiniciar contador" el contador pasa de tener un valor de 4 a tener un valor de 0, quedando la ventana como en la Figura 4.60, donde se veía el programa recién iniciado.

Por último, se prueba el funcionamiento del generador de ventanas de mensaje, escribiendo para ello un título para dicha ventana y un texto que aparecerá como mensaje en la ventana. Tanto el título como el cuerpo del mensaje se pueden ver en la Figura 4.63.

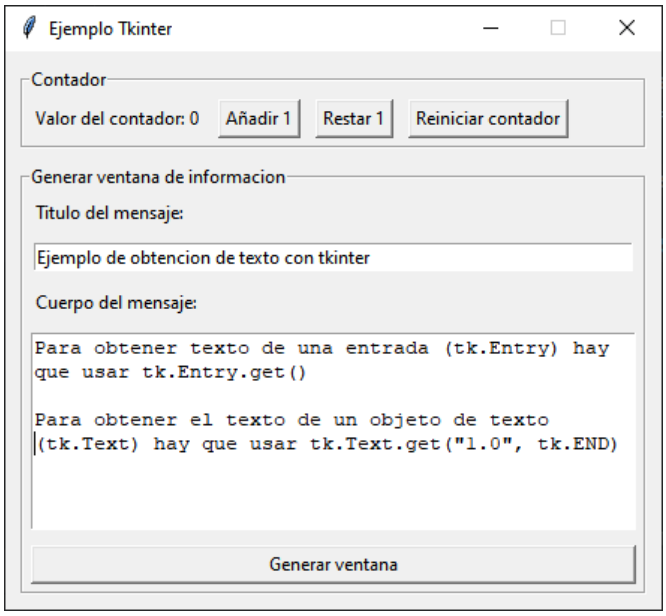


Figura 4.63: Ventana principal con los campos de título y cuerpo del mensaje rellenos

Una vez obtenidos el título y el cuerpo del mensaje escritos, se presiona el botón "Generar ventana". Esto ejecuta la función `generateMsgInfoWinodw` (Listado 4.64), encargada de generar la ventana de mensaje con los datos introducidos en la interfaz.

Listado 4.64: Función `generateMsgInfoWinodw` del ejemplo de Tkinter

```
def generateMsgInfoWinodw(self):
    title = self.titleEntry.get()
    message = self.messageText.get("1.0", tk.END)
    messagebox.showinfo(title=title, message=message)
```

La ventana generada, con los datos que aparecen en la Figura 4.63, se muestra en la Figura 4.64.

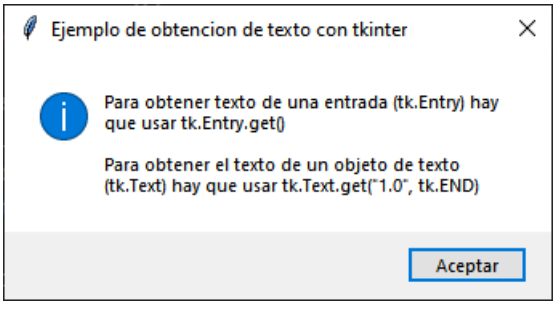


Figura 4.64: Ventana de mensaje generada

4.4.2.2. Interfaz del programa principal

La interfaz del programa objeto de este TFG se encuentra definida en el módulo `UI.py`. Al igual que en el ejemplo anterior, esta interfaz cuenta con una clase denominada `UI`, encargada de generar y gestionar todos los aspectos relacionados con la presentación y el funcionamiento de la aplicación. El constructor (Listado 4.65) de dicha clase no solo incluye lo necesario para construir la interfaz, como el título de la ventana, el protocolo para su cierre y el ícono que se muestra en la esquina superior izquierda, sino que también establece las referencias y crea

instancias de los distintos módulos que componen la aplicación. De este modo, la interfaz puede acceder a cada uno de ellos, facilitando la interacción y la coordinación entre la lógica de la interfaz y el resto de los módulos.

Listado 4.65: Constructor de la clase UI

```
def __init__(self):
    self.root = tk.Tk()  # Inicializacion de la ventana
    self.root.withdraw()
    self.root.title("TFG - Gestor de base de datos")  # Establecimiento del nombre
    de la ventana
    icon = tk.PhotoImage(file=os.path.join(getattr(sys, '_MEIPASS', os.path.
    dirname(os.path.abspath(__file__))),"icon.png"))
    self.root.iconphoto(False,icon)
    self.root.columnconfigure(0, weight=1)  # Activacion del autoajuste de la
    columna 0
    self.root.resizable(False, False)  # Restriccion del redimensionamiento de la
    ventana principal
    self.root.protocol("WM_DELETE_WINDOW",
                       self.onCloseEvent)  # Registro de la función a ejecutar al
                       cerrar la aplicacion
    self.supplyLoger = SupplyLoger  # Inicializacion del objeto SupplyLoger
    self.demandLoger = DemandLoger  # Inicializacion del objeto DemandLoger
    self.resultsLoger = ResultsLoger  # Inicializacion del objeto ResultsLoger
    self.ymlParser = Parser()  # Inicializacion del módulo yamlParser
    self.ymlWriter = Writer()  # Inicializacion del módulo yamlWriter
    self.sqlSupply = SqlSupply()  # Inicializacion del objeto SqlSupply del módulo
    SQLHandler
    self.sqlDemand = SqlDemand()  # Inicializacion del objeto SqlDemand del módulo
    SQLHandler
    self.sqlResults = SqlResults()  # Inicializacion del objeto SqlResults del
    módulo SQLHandler
    self.config = Config()  # Inicializacion del módulo configManager
    self.csvReader = csvReader()  # Inicializacion del objeto csvReader del módulo
    csvHandler
    self.csvWriter = csvWriter  # Inicializacion del objeto csvWriter del módulo
    csvHandler
    self.init_ui()  # Inicializacion de la interfaz
    self.root.deiconify()
```

La distribución de la interfaz del programa se muestra en la Figura 4.65. En ella se pueden apreciar, por ejemplo, los botones destinados a la importación de los datos de los archivos en cada una de las bases de datos, los botones para la exportación de datos a un archivo desde las bases de datos, así como los que permiten eliminar los datos asociados a un archivo específico. También se encuentra un apartado dedicado a la interacción con las bases de datos mediante sentencias SQL, en el que se pueden ejecutar sentencias previamente almacenadas, cargar un archivo con extensión `.sql` que contenga la sentencia SQL o, alternativamente, escribir la sentencia directamente en la zona de texto destinada para ello.

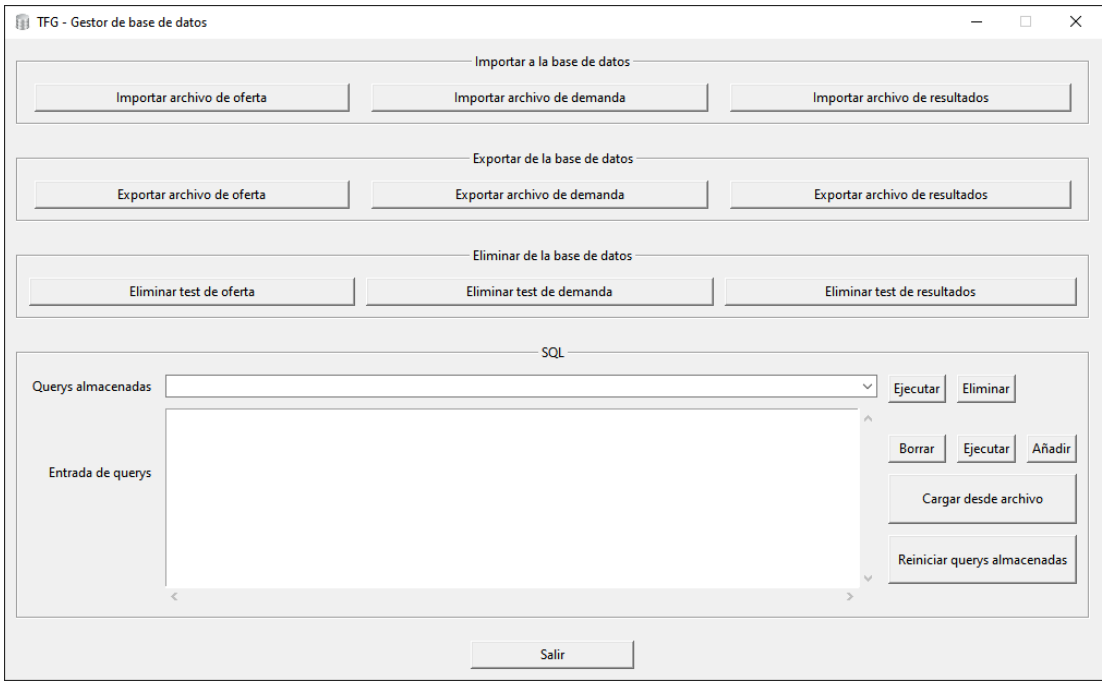


Figura 4.65: Ventana principal de la aplicación

Las sentencias que son cargadas desde un archivo o escritas directamente en la interfaz se someten a un proceso de validación antes de ser ejecutadas. Esto evita que se ejecuten sentencias en la base de datos que presenten errores, como por ejemplo, errores sintácticos. Si se detecta algún error durante la validación, se notificará al usuario mediante una ventana emergente el error presente en la sentencia. En las Figuras 4.66 y 4.67 se puede apreciar una sentencia con un error; en este caso, la columna "campoInexistente" no existe dentro de la base de datos de los archivos de la oferta, y su correspondiente mensaje indicando que la columna requerida por el usuario no existe en la base de datos.

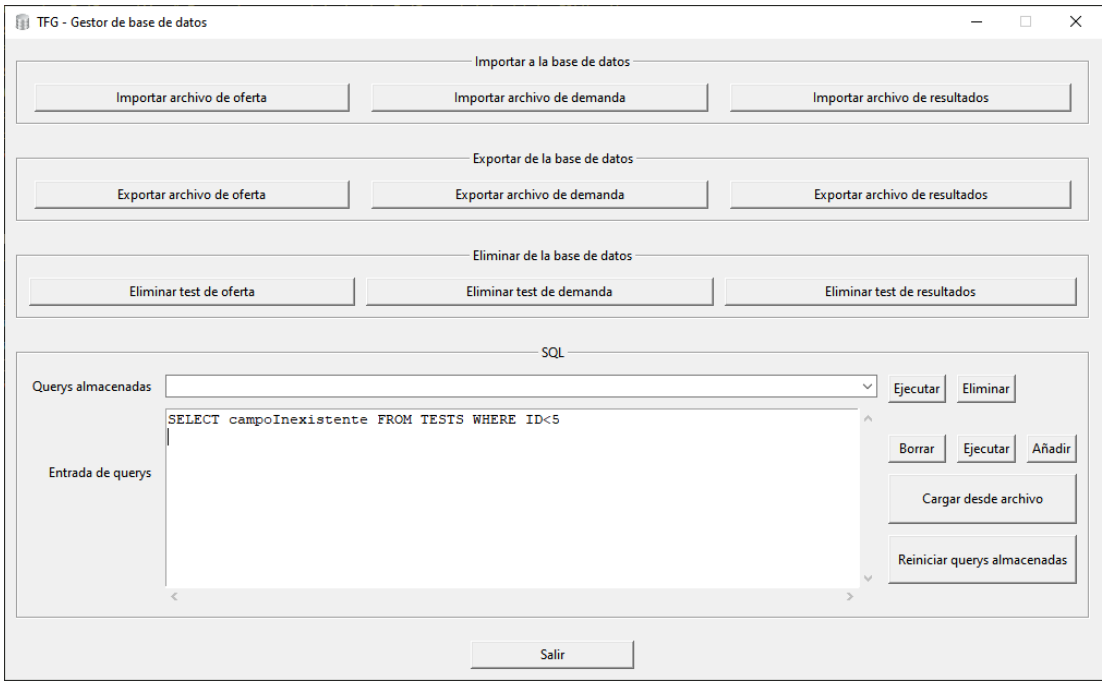


Figura 4.66: sentencia con error de columna faltante

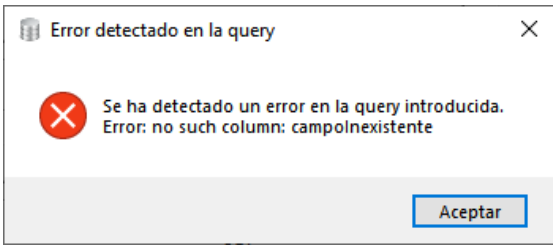


Figura 4.67: Mensaje de error al ejecutar la sentencia con error de la Figura 4.66

Capítulo 5

Resultados y discusión

En esta sección se realizan una serie de comprobaciones de funcionamiento de la interfaz y de la base de datos, con la finalidad de comprobar que todo funciona correctamente y sin errores.

Este capítulo está estructurado en dos secciones. La Sección 5.1 muestra los tests realizados para comprobar el correcto funcionamiento de la interfaz. La Sección 5.2 testea que las bases de datos funcionan correctamente, sin omitir ninguna información.

5.1. Prueba de la interfaz

En esta sección se va a analizar el funcionamiento general de la interfaz mediante una serie de pruebas centradas en sus funciones principales.

5.1.1. Importación de archivos

Cuando el usuario pulsa cualquiera de los botones para importar archivos a su correspondiente base de datos, ya sea la base de datos de los archivos de entrada de datos de la oferta, la de los archivos de entrada de datos de la configuración de la demanda o la de los archivos de resultados, el programa pide al usuario que seleccione el archivo que desea introducir en la base de datos.

Después de seleccionar el archivo, se pregunta al usuario si quiere añadir alguna observación a la entrada que se va a crear en la tabla `TESTS` de la base de datos correspondiente (Figura 5.1).

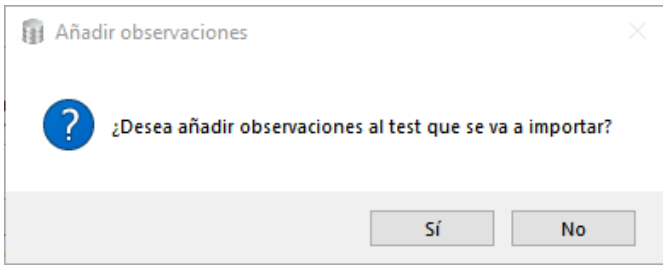


Figura 5.1: Ventana de confirmación para añadir observaciones a la entrada del archivo.

En caso de que la respuesta por parte del usuario sea positiva, se muestra otra ventana (Figura 5.2) en la que el usuario puede escribir la observación que quiere que se almacene en la columna `OBSERVATIONS` en la tabla `TESTS` de la base de datos. Después de introducir las

observaciones, el programa intenta introducir los datos del archivo seleccionado en la base de datos.

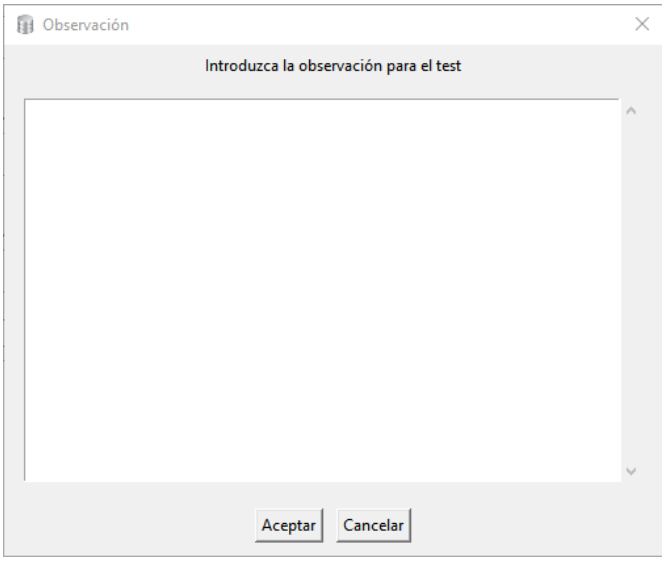


Figura 5.2: Ventana para añadir observaciones.

En caso de que la respuesta por parte del usuario sea negativa, se procede a introducir los datos del archivo seleccionado en la base de datos sin añadir ninguna observación.

Si el programa no ha encontrado problemas al introducirlo, notificará al usuario mediante una ventana de información (Figura 5.3) que la importación ha sido exitosa. Si por el contrario, la importación ha fallado, también se le notificará al usuario mediante una ventana de error (Figura 5.4). Por último, si el archivo que se intenta introducir en la base de datos ya se encuentra dentro de la base de datos, el programa mostrará que ha habido un error al importar el archivo, haciendo hincapié en que puede que el archivo ya se encuentre en la base de datos (Figura 5.5). Con estas comprobaciones se ha verificado la detección de los potenciales errores en este proceso.

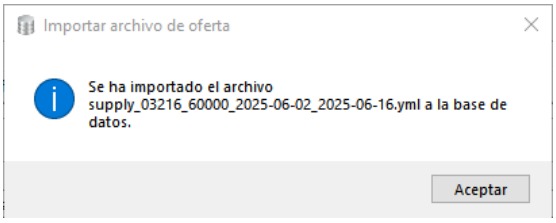


Figura 5.3: Ventana de notificación para importación exitosa.

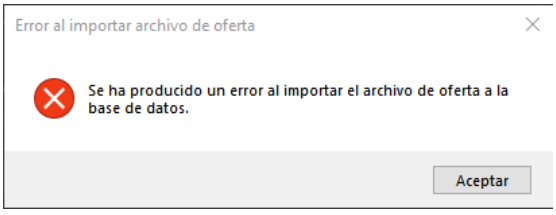


Figura 5.4: Ventana de notificación para importación fallida.

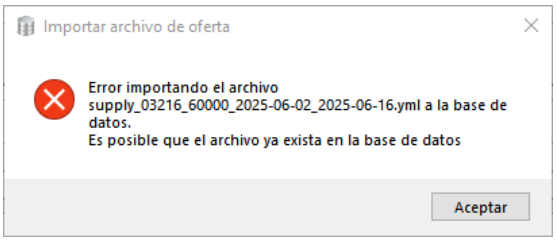


Figura 5.5: Ventana de notificación para archivo duplicado.

5.1.2. Exportación de archivos

En caso de que el usuario quiera exportar uno de los archivos almacenados en la base de datos ha de presionar uno de los botones destinados a la exportación de los archivos. Después de presionar uno de los botones, el programa muestra una ventana (Figura 5.6) con una lista con los archivos que se encuentran almacenados en la base de datos para que el usuario seleccione cual de ellos es el que quiere exportar. Acto seguido, se le pide al usuario seleccionar la carpeta en la que se va a guardar el archivo exportado. Si la base de datos no contiene ningún archivo almacenado, el programa muestra un mensaje de error indicando que la base de datos se encuentra vacía.

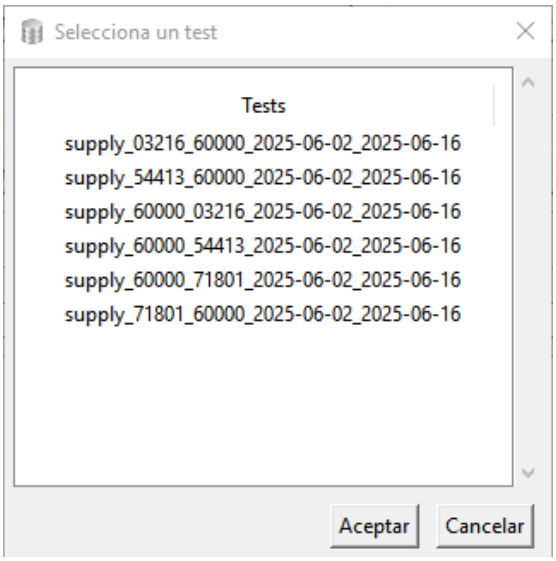


Figura 5.6: Ventana de selección de archivo.

Una vez seleccionado el archivo a exportar y la ubicación en la que se va a guardar, el programa comienza a recabar la información almacenada en la base de datos, apoyándose en las tablas auxiliares para saber qué información de la que contienen las tablas de datos pertenece al archivo en cuestión. Una vez recabada la información del archivo, se le da el mismo formato que poseen los datos al procesar un archivo Yaml. Tras esto, el programa crea el archivo en la ubicación especificada anteriormente e inserta la información a la que previamente se le ha dado el mismo formato que a un archivo Yaml.

5.1.3. Eliminación de archivos de la base de datos

En caso de que el usuario quiera eliminar un archivo de la base de datos, se debe pulsar uno de los botones dentro de la sección "Eliminar de la base de datos". Tras presionar uno de los botones, aparece una ventana (Figura 5.6) para seleccionar qué archivo se desea eliminar.

Una vez seleccionado, el programa lanza una serie de sentencias SQL que se encargan de eliminar el archivo y todos los datos pertenecientes a este, que no estén presentes en ningún otro archivo. Por ejemplo, si en el archivo que se va a eliminar aparece la estación de Atocha y esta se encuentra en otro archivo de los almacenados en la base de datos, no será eliminada. Por el contrario, si, por ejemplo, la estación de Chamartín solo aparece en el archivo que se va a eliminar, esta sí que será eliminada de la tabla `STATIONS`.

5.1.4. Ejecución de sentencias SQL

El programa también permite la interacción con las bases de datos, mediante sentencias SQL. Hay 3 formas diferentes de ejecutar estas sentencias: desde las sentencias almacenadas en la memoria del programa, escribiendo la sentencia directamente en el cuadro de texto dentro de la sección "SQL" y ejecutándola en la base de datos a la que vaya dirigida la sentencia, o cargando la sentencia directamente desde un archivo y ejecutándola en la base de datos objetivo. Para la selección de la base de datos se genera una ventana (Figura 5.7) para seleccionar cuál de las tres es el objetivo de la sentencia SQL.

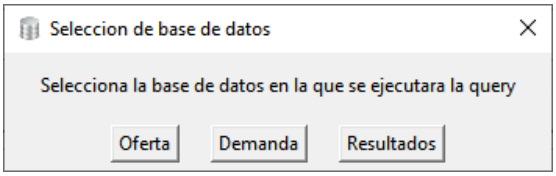


Figura 5.7: Ventana de selección de la base de datos objetivo.

Por defecto, en la aplicación hay tres sentencias guardadas en memoria (Figura 5.8). Las tres sentencias son para mostrar los archivos almacenados en cada una de las bases de datos. Para usarlas, hay que seleccionar la que se quiera ejecutar del menú desplegable y pulsar el botón ejecutar, que aparece a su derecha.

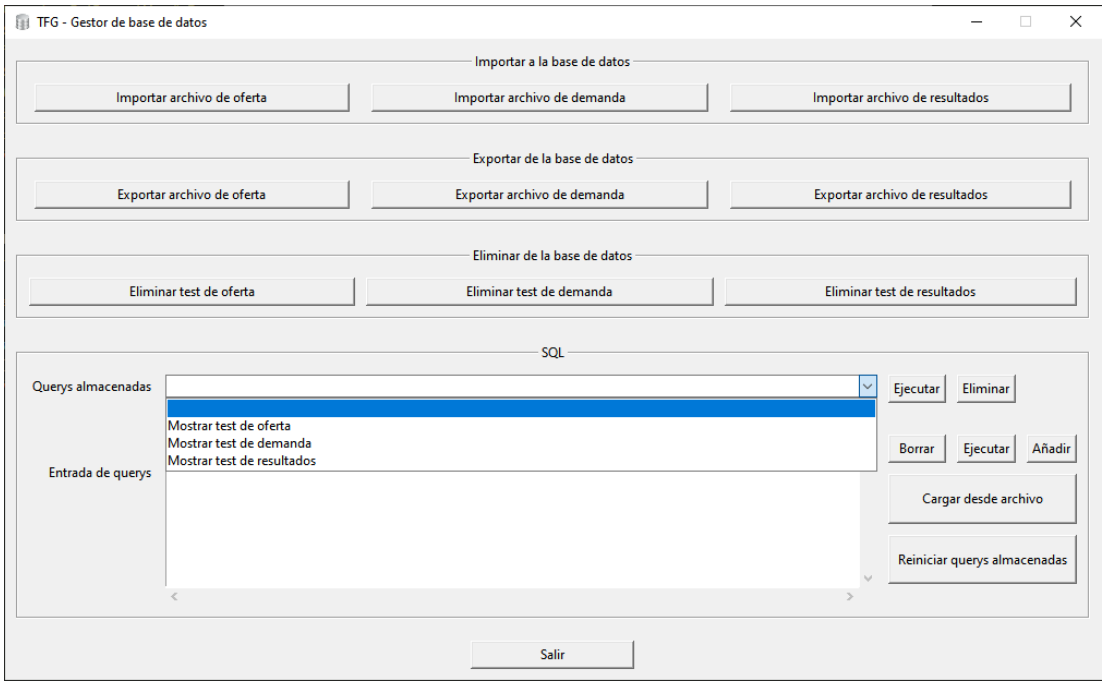


Figura 5.8: Desplegable con las sentencias almacenadas en la aplicación.

Para este ejemplo, se usa la sentencia almacenada "Mostrar test de oferta", cuya sentencia

SQL es la que aparece reflejada en el siguiente listado:

Listado 5.1: sentencia "Mostrar test de oferta"

```
SELECT *
FROM TESTS
```

Debido a que las sentencias que se incluyen por defecto se encargan de seleccionar datos en la base de datos mediante el comando SQL `SELECT`, los datos que devuelva la base de datos se muestran en una ventana aparte, en la que se puede exportar la tabla generada a un archivo CSV, con un nombre personalizado definido por el usuario. En este caso, en la Figura 5.9 se puede observar el resultado de la ejecución de la sentencia "Mostrar test de oferta".

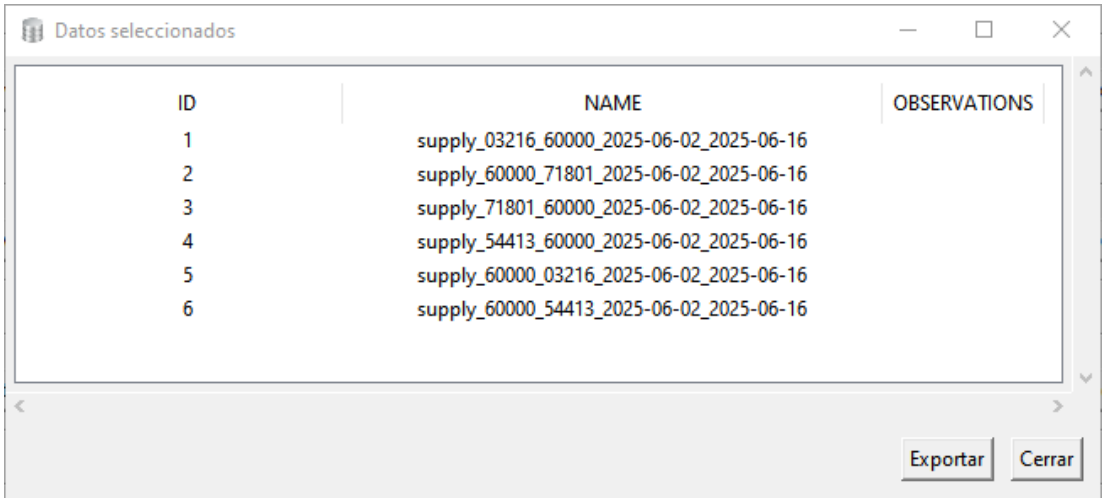


Figura 5.9: Ventana con los resultados de la sentencia "Mostrar test de oferta".

Como ya se ha mencionado anteriormente, el usuario de la aplicación también puede escribir sentencias directamente en la interfaz mediante la entrada de texto. El funcionamiento es el siguiente: el usuario introduce la sentencia en el cuadro de texto, y después pulsa el botón "Ejecutar". Acto seguido aparece la ventana para seleccionar la base de datos en la que ejecutar la sentencia (Figura 5.7). En este caso, como ejemplo, se va a ejecutar la siguiente sentencia SQL:

Listado 5.2: Selección de tests con id < 5

```
SELECT * FROM TESTS WHERE ID < 5
```

Esta sentencia selecciona los archivos cuyo campo `ID` tenga un valor inferior a 5. El resultado de la sentencia aparece en la Figura 5.10

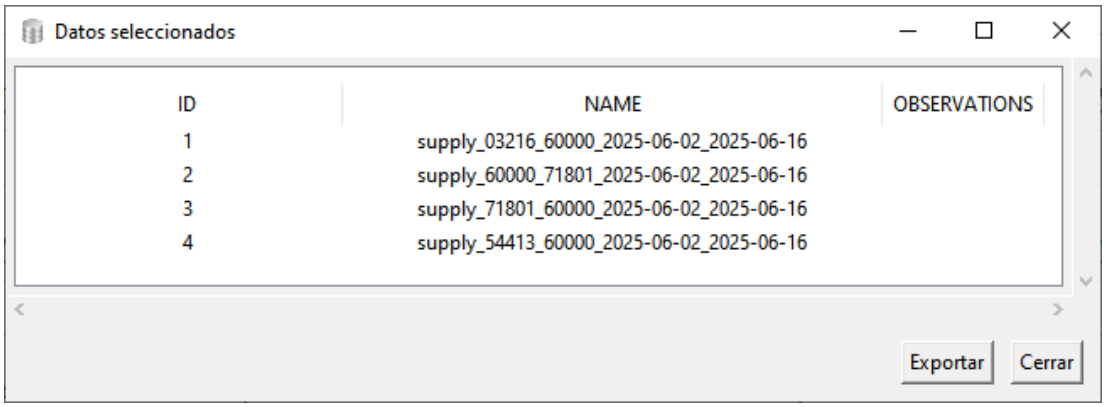


Figura 5.10: Ventana con los resultados de la sentencia del Listado 5.2.

Además, se pueden importar sentencias desde archivos con extensión `.sql`. Al importar estos archivos, la sentencia que contengan se muestra en el cuadro de texto. Para ejecutar la sentencia se sigue el mismo procedimiento que en el caso de redactar la sentencia dentro de la interfaz. Como ejemplo, se van a seleccionar las estaciones pertenecientes al ramal con el identificador de valor 10, perteneciente al corredor español, requiriendo a la base de datos el valor de las columnas `ID`, `NAME` y `CITY` de la tabla `STATIONS` (Figura 4.11).

Para ello, se emplea la sentencia SQL del Listado 5.3.

Listado 5.3: Estaciones del ramal con identificador 10

```
SELECT
    STATIONS.ID,
    STATIONS.NAME,
    STATIONS.CITY
FROM
    STATIONS
WHERE
    STATIONS.ID IN (SELECT path.value
            FROM CORRIDOR_STATIONS,
            json_each(CORRIDOR_STATIONS.STATIONS) AS path
            WHERE CORRIDOR_STATIONS.ID = 10)
```

Al ejecutar la sentencia del Listado 5.3, la aplicación muestra la ventana de la Figura 5.11, con los datos solicitados a la base de datos de los archivos de entrada de datos de la oferta, en este caso.

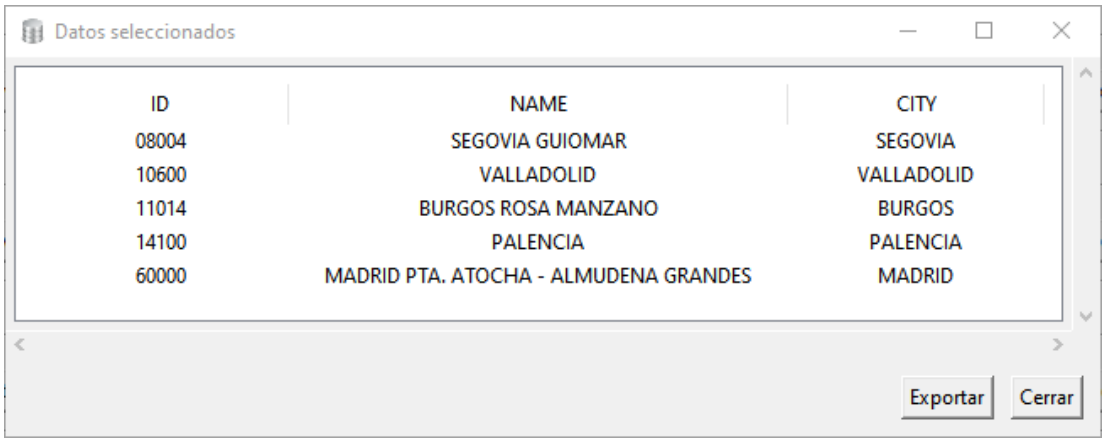


Figura 5.11: Estaciones pertenecientes al ramal con identificador 10.

En la Figura 5.12 se pueden ver todos los ramales del corredor español. El ramal que se encuentra seleccionado corresponde con el ramal que posee el identificador con valor 10. Se puede apreciar que las estaciones que tienen el identificador contenido en el ramal aparecen en la Figura 5.11, aunque con un orden distinto. Esto se debe a que en el ramal, las estaciones se ordenan según el recorrido; se inicia por la estación de inicio y las siguientes se listan según el trayecto. En cambio, los datos que muestra la ventana de la aplicación se ordenan siempre de manera ascendente según el identificador.



Figura 5.12: Ramales del corredor español.

5.1.5. Gestión de sentencias SQL

Las sentencias SQL introducidas en la entrada de texto pueden ser almacenadas en la aplicación para su posterior uso, sin la necesidad de volver a introducirlas. Para ello, una vez escrita la sentencia, pulsamos el botón "Añadir" y, acto seguido, se pide al usuario que seleccione a qué base de datos quiere afectar con esa sentencia cuando sea lanzada desde la memoria y, por último, se pide al usuario que le añada un nombre a esa sentencia. Una vez hecho esto, la sentencia recién guardada aparece en el desplegable de las sentencias almacenadas. Como ejemplo, se ha añadido la sentencia del Listado 5.2 a la memoria de la aplicación. Al pulsar el botón de añadir, se muestra una ventana para poner el nombre a la sentencia a guardar; en este caso, se le ha puesto el nombre "Seleccionar test con id < 5". Tras nombrar la sentencia, se selecciona a qué base de datos afecta la sentencia y, una vez hecho esto, la sentencia aparece en el menú desplegable junto a las otras sentencias almacenadas en memoria, como se puede ver en la Figura 5.13.

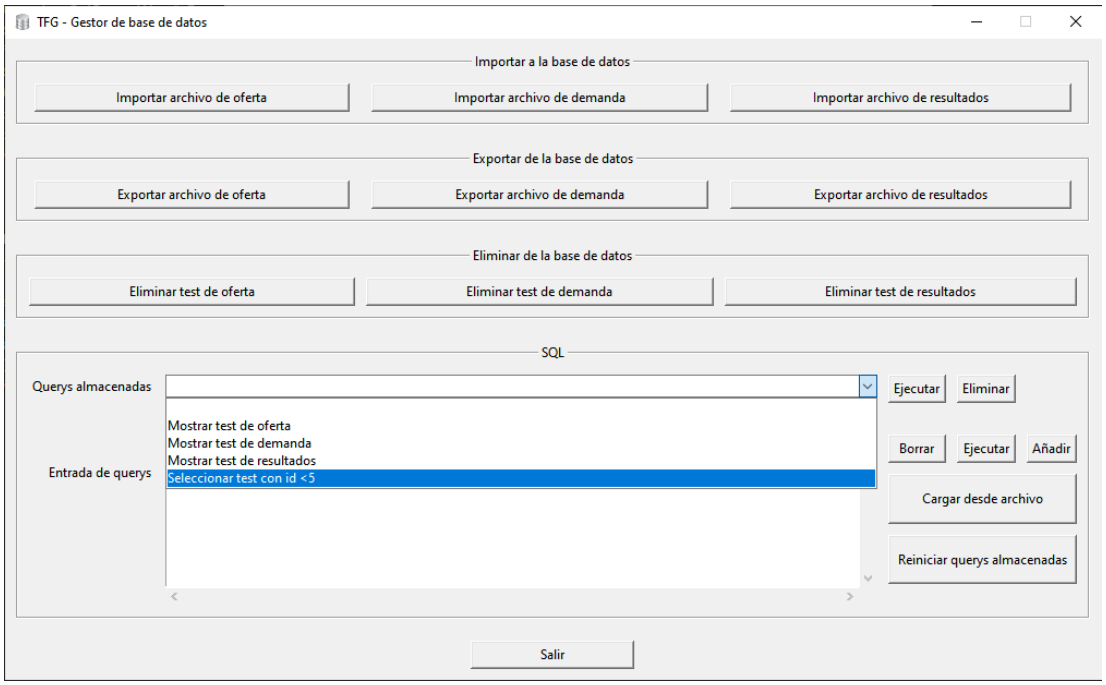


Figura 5.13: Nueva lista de sentencias guardadas.

5.1.6. Generación de CSV a partir de los datos devueltos por sentencias SQL

Empleando la salida de la sentencia SQL del Listado 5.3, se va a generar un archivo CSV con el que verificar el correcto comportamiento de la funcionalidad para exportar los datos devueltos por la ejecución de una sentencia SQL (Figura 5.11). Para generar el archivo se pulsa el botón "Exportar" de la ventana presente en la Figura 5.11. Acto seguido, aparece una ventana para escribir el nombre que recibe el archivo una vez creado. Después, se muestra una ventana con la que seleccionar la carpeta en la que se va a guardar el archivo con los datos exportados. La Tabla 5.1 contiene los datos del archivo exportado. El archivo se encuentra en el siguiente enlace[1] del repositorio de GitHub.

ID	NAME	CITY
08004	SEGOVIA GUIOMAR	SEGOVIA
10600	VALLADOLID	VALLADOLID
11014	BURGOS ROSA MANZANO	BURGOS
14100	PALENCIA	PALENCIA
60000	MADRID PTA. ATOCHA - ALMUDENA GRANDES	MADRID

Tabla 5.1: Datos exportados de las estaciones del ramal con identificador 10

5.2. Prueba de las bases de datos

En esta sección, se pretende comprobar el correcto funcionamiento de las bases de datos, es decir, que las bases de datos almacenen la información extraída de los archivos de forma correcta. Para ello, se han importado varios archivos con la aplicación, para después exportarlos y comprobar que los archivos exportados desde la aplicación contienen la misma información

[1] **Archivo exportado:** https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos/blob/master/Archivos%20Yaml%20y%20CSV/Exportados/Vistas%20exportadas/Estaciones%20del%20ramal%20con%20id%2010.csv

que los archivos importados. De esta forma, se van a comparar los archivos originales con los archivos exportados desde las bases de datos empleando la aplicación para la gestión de las bases de datos objeto de este TFG para, de este modo, verificar el correcto funcionamiento de las bases de datos.

Para esta tarea se han empleado dos pequeños *scripts* en Python: uno que compara dos archivos Yaml y otro que compara archivos CSV. Esto se debe a que realizar dichas comparaciones de manera manual resultaría no solo muy tedioso, sino además propenso a errores, dada la gran cantidad de elementos que habría que revisar para confirmar que ambos archivos contienen los mismos datos.

El *script* que se encarga de comparar los archivos Yaml revisa que en ambos archivos se encuentren las mismas claves, tanto las claves raíz como lo que depende de éstas, y que los datos en cada una de las claves sean iguales, tanto en valor como en tipo de dato. En cuanto al script que se encarga de los archivos CSV, este compara cada fila teniendo en cuenta el identificador de cada una ellas y revisa que los datos de las filas que contienen el mismo identificador sean iguales. Ambos *scripts* muestran la información en la consola y, además, copian esta salida a un archivo de texto para poder revisar los posibles fallos aunque se haya cerrado la consola que contenía los mensajes indicando las diferencias entre los dos archivos comparados.

Para verificar el funcionamiento de los scripts, se ha realizado una prueba empleando 2 archivos Yaml (Listados 5.4 y 5.5).

Listado 5.4: Archivo Yaml de prueba "Letras a números.yml"

```
letras_a_numeros:
    -a:1
     b:2
     c:3
     d:4
```

Listado 5.5: Archivo Yaml de prueba "Letras a números 2.yml"

```
letras_a_numeros:
    -a:1
     b:2
     c:5
     d:4

otra_clave:
    - null
```

Primero se compara el archivo del Listado 5.4 consigo mismo para verificar que, al introducir dos archivos idénticos, el script no detecta ningún fallo. Al comparar el archivo "Letras a números.yml" consigo mismo, se constata que el *script* verifica que ambos archivos de entrada son iguales, lo que es lógico dado que se trata del mismo archivo empleado como entrada dos veces. Esto se evidencia en la salida de la consola, que se muestra en el Listado 5.6.

Listado 5.6: Salida de consola al comparar "Letras a números.yml" consigo mismo.

```
Selecciona el primer archivo YAML (referencia)
YAML1: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Yaml prueba
    comparador/Letras a números.yml
Selecciona el segundo archivo YAML (a comparar)
```

```
YAML2: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Yaml prueba
    comparador/Letras a números.yml

=== RESULTADO DE LA COMPARACIÓN ===
¡No se encontraron diferencias entre los archivos YAML!
```

Posteriormente, se ha comparado el archivo del Listado 5.4 con el archivo del Listado 5.5. En este caso, el script detecta que hay una diferencia e imprime en la consola que ha encontrado una diferencia entre los archivos y muestra las diferencias entre estos. La diferencia entre estos dos archivos es que en el diccionario, el valor de "c" en el primer archivo es de 3, mientras que en el segundo archivo es de 5 y, además, existe otra clave en el segundo archivo llamada "otra_clave" que contiene una lista con un único elemento de valor "null". La salida de la consola se encuentra en el Listado 5.7

Listado 5.7: Salida de consola al comparar "Letras a números.yml" con "Letras a números 2.yml"

```
Selecciona el primer archivo YAML (referencia)
YAML1: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Yaml prueba
    comparador/Letras a números 2.yml
Selecciona el segundo archivo YAML (a comparar)
YAML2: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Yaml prueba
    comparador/Letras a números.yml

=== RESULTADO DE LA COMPARACIÓN ===
Se encontraron las siguientes diferencias:
- letras_a_numeros: YAML1 -> '-a:1 b:2 c:5 d:4', YAML2 -> '-a:1 b:2 c:3 d:4'
- otra_clave: Clave faltante en YAML2
```

Todos los archivos empleados en las pruebas de la aplicación se encuentran subidos en el siguiente repositorio de GitHub[2].

Primero se van a comparar los archivos Yaml de configuración de la oferta. Los archivos originales que se han introducido, con ayuda de la aplicación, a la base de datos para los archivos de entrada de datos de la oferta, se encuentran en el siguiente enlace[3]. Los archivos exportados de la base de datos se encuentran en el siguiente enlace[4].

Listado 5.8: Salida de la consola para los archivos supply_03216_60000_2025-06-02_2025-06-16

```
Selecciona el primer archivo YAML (referencia)
YAML1: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/inputData/Oferta/supply_03216_60000_2025-06-02_2025-06-16.yaml
Selecciona el segundo archivo YAML (a comparar)
YAML2: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/outputData/Oferta/supply_03216_60000_2025-06-02_2025-06-16.yml

=== RESULTADO DE LA COMPARACIÓN ===
¡No se encontraron diferencias entre los archivos YAML!
```

[2] Archivos Yaml y CSV: https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos/tree/master/Archivos%20Yaml%20y%20CSV

[3] Archivos originales de entrada de datos de la oferta: https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos/tree/master/Archivos%20Yaml%20y%20CSV/Originales/Oferta

[4] Archivos exportados de entrada de datos de la oferta: https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos/tree/master/Archivos%20Yaml%20y%20CSV/Exportados/Oferta

Listado 5.9: Salida de la consola para los archivos supply_54413_60000_2025-06-02_2025-06-16

```
Selecciona el primer archivo YAML (referencia)
YAML1: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/inputData/Oferta/supply_54413_60000_2025-06-02_2025-06-16.yaml
Selecciona el segundo archivo YAML (a comparar)
YAML2: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/outputData/Oferta/supply_54413_60000_2025-06-02_2025-06-16.yml

=== RESULTADO DE LA COMPARACIÓN ===
¡No se encontraron diferencias entre los archivos YAML!
```

Listado 5.10: Salida de la consola para los archivos supply_60000_03216_2025-06-02_2025-06-16

```
Selecciona el primer archivo YAML (referencia)
YAML1: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/inputData/Oferta/supply_60000_03216_2025-06-02_2025-06-16.yaml
Selecciona el segundo archivo YAML (a comparar)
YAML2: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/outputData/Oferta/supply_60000_03216_2025-06-02_2025-06-16.yml

=== RESULTADO DE LA COMPARACIÓN ===
¡No se encontraron diferencias entre los archivos YAML!
```

Listado 5.11: Salida de la consola para los archivos supply_60000_54413_2025-06-02_2025-06-16

```
Selecciona el primer archivo YAML (referencia)
YAML1: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/inputData/Oferta/supply_60000_54413_2025-06-02_2025-06-16.yaml
Selecciona el segundo archivo YAML (a comparar)
YAML2: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/outputData/Oferta/supply_60000_54413_2025-06-02_2025-06-16.yml

=== RESULTADO DE LA COMPARACIÓN ===
¡No se encontraron diferencias entre los archivos YAML!
```

Listado 5.12: Salida de la consola para los archivos supply_60000_71801_2025-06-02_2025-06-16

```
Selecciona el primer archivo YAML (referencia)
YAML1: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/inputData/Oferta/supply_60000_71801_2025-06-02_2025-06-16.yaml
Selecciona el segundo archivo YAML (a comparar)
YAML2: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/outputData/Oferta/supply_60000_71801_2025-06-02_2025-06-16.yml

=== RESULTADO DE LA COMPARACIÓN ===
¡No se encontraron diferencias entre los archivos YAML!
```

Listado 5.13: Salida de la consola para los archivos supply_71801_60000_2025-06-02_2025-06-16

```
Selecciona el primer archivo YAML (referencia)
YAML1: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/inputData/Oferta/supply_71801_60000_2025-06-02_2025-06-16.yaml
```

```
Selecciona el segundo archivo YAML (a comparar)
YAML2: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/outputData/Oferta/supply_71801_60000_2025-06-02_2025-06-16.yml

=== RESULTADO DE LA COMPARACIÓN ===
¡No se encontraron diferencias entre los archivos YAML!
```

Al examinar las salidas de la consola correspondientes a las distintas comparaciones (Listados 5.8, 5.9, 5.10, 5.11, 5.12 y 5.13), se comprueba que los datos de los archivos originales, importados y luego exportados mediante el programa se mantienen idénticos. En otras palabras, todos los archivos exportados conservan exactamente la misma información que los que se importaron a la base de datos, lo que demuestra que la base de datos no corrompe ni modifica los datos.

En el caso del archivo de demanda, el archivo original se encuentra en el siguiente enlace[5]. El archivo exportado de la base de datos se encuentra en el siguiente enlace[6].

Listado 5.14: Salida de la consola para los archivos demand_data

```
Selecciona el primer archivo YAML (referencia)
YAML1: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/inputData/Demanda/demand_data.yml
Selecciona el segundo archivo YAML (a comparar)
YAML2: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/outputData/Demanda/demand_data.yml

=== RESULTADO DE LA COMPARACIÓN ===
¡No se encontraron diferencias entre los archivos YAML!
```

Como se puede observar en la salida de la consola (Listado 5.14) a la hora de comparar el archivo "demand_data" original con el que ha sido exportado de la base de datos por la aplicación se puede comprobar que ambos archivos contienen los mismos datos, pero hay una pequeña diferencia en cuanto a formato entre ambos archivos, aunque esto no afecta a la hora de leer y de procesar el archivo. La diferencia de formato en cuestión se debe a que el archivo original almacena las listas como si fueran listas de Python; es decir, en la misma línea. Sin embargo, el archivo exportado convierte las listas al formato que emplea Yaml. Como ya se ha dicho, esto no afecta a la hora de procesar los datos de los archivos, pero sí a la hora de modificarlos y de visualizarlos por una persona, debido a que la estructura que emplea Yaml es, probablemente, menos visual que emplear listas como las de Python. A continuación se muestra un fragmento de ambos archivos que ilustra lo descrito anteriormente:

Listado 5.15: Fragmento del archivo demand_data original

```
userPattern:
   -variables:
      - name: origin
        type: fuzzy
        support: [0, 100]
        sets: [very_near, mid_range, far, far_away]
        very_near: [0, 0, 10, 20]
```

[5] Archivo original de entrada de datos de la demanda: https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos/tree/master/Archivos%20Yaml%20y%20CSV/Originales/Demanda

[6] Archivo exportado de entrada de datos de la demanda: https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos/tree/master/Archivos%20Yaml%20y%20CSV/Exportados/Demanda

```
        mid_range: [10, 20, 50, 60]
        far: [50, 60, 70, 80]
        far_away:  [70, 80, 100, 100]
```

Listado 5.16: Fragmento del archivo demand_data exportado por la aplicación

```
userPattern:
   -variables:
      - name: origin
        type: fuzzy
        support:
        - 0
        - 100
        sets:
        - very_near
        - mid_range
        - far
        - far_away
        very_near:
        - 0
        - 0
        - 10
        - 20
        mid_range:
        - 10
        - 20
        - 50
        - 60
        far:
        - 50
        - 60
        - 70
        - 80
        far_away:
        - 70
        - 80
        - 100
        - 100
```

Por último, se van a comparar los archivos CSV de los resultados. El archivo original se encuentra en el siguiente enlace[7]. El archivo exportado de la base de datos se encuentra en el siguiente enlace[8].

Listado 5.17: Salida de la consola para los archivos output_fuzzy

```
Selecciona el primer archivo CSV (referencia)
CSV1: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/inputData/Resultados/output_fuzzy.csv
```

[7] Archivo original de resultados: https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos/tree/master/Archivos%20Yaml%20y%20CSV/Originales/Resultados

[8] Archivo exportado de resultados: https://github.com/Sergioba99/TFG-Gestor_De_Bases_de_Datos/tree/master/Archivos%20Yaml%20y%20CSV/Exportados/Resultados

```
Selecciona el segundo archivo CSV (a comparar)
CSV2: D:/Sergio/universidad/24-25/TFG Base de datos/TFG/Material TFG/Aplicacion
    compilada/outputData/Resultados/output_fuzzy.csv

=== RESULTADO DE LA COMPARACIÓN ===
¡No se encontraron diferencias entre los archivos CSV!
```

En la salida de la consola del Listado 5.17, se puede comprobar que los dos archivos CSV comparados presentan los mismos datos. En este caso se trata del archivo de resultados importado a la base de datos con el archivo exportado de la base de datos empleando la aplicación. Al igual que sucedía con los datos solicitados a la base de datos mediante la sentencia que devolvía las estaciones presentes en el ramal con el identificador de valor 10 (Listado 5.3), los datos de ambos CSV no tienen el mismo orden, debido a que al solicitar los datos para regenerar el archivo CSV los datos se ordenan de manera ascendente por el valor del identificador.

Capítulo 6

Conclusiones

A continuación, se analizará el cumplimiento de los objetivos durante el desarrollo del TFG.

Se ha investigado el funcionamiento de las bases de datos relacionales así como herramientas como SQLiteStudio. Esto ha permitido entender cómo funciona una base de datos relacional y plantear posibles diseños para las diferentes bases de datos necesarias en la consecución del trabajo. También se han estudiado los diferentes comandos SQL, que permiten la interacción con las bases de datos. Esto ha permitido realizar una serie de pruebas para familiarizarse con la creación e interacción con las bases de datos, empleando para ello la herramienta SQLiteStudio, la cual emplea como motor de base de datos SQLite3. De esta manera, se ha podido comprender cómo emplear el lenguaje SQL para realizar todas las tareas necesarias referentes a las bases de datos.

Se ha realizado el diseño de las tres bases de datos necesarias para albergar los datos que se importan desde los diferentes archivos: una para los archivos que emplea el simulador para la entrada de datos de la oferta, otra para los archivos de entrada de datos de la demanda del simulador y, por último, otra que almacena los diferentes archivos de resultados que arroje el simulador. Para el diseño se ha empleado la herramienta SQLiteStudio para generar las tres bases de datos con sus correspondientes tablas.

Se ha diseñado y desarrollado una herramiento en lenguaje Python que permite importar la información de los diferentes archivos del simulador a su base de datos correspondiente, así como regenerar estos archivos empleando la información contenida en dichas bases de datos. Este programa se ha dividido en diferentes módulos que realizan diferentes tareas. Para la lectura, escritura y procesado de datos de los archivos, se han realizado una serie de módulos que permiten leer y procesar la información contenida en archivos Yaml y CSV para su posterior inserción en las bases de datos y, además, estos módulos también permiten la creación de los archivos empleando la información obtenida de las bases de datos. También existe un módulo que es el encargado de crear estas bases de datos y manejar todo el flujo de información, permitiendo insertar, consultar y eliminar los datos almacenados mediante sentencias SQL.

Se ha estudiado el diseño y desarrollo de interfaces gráficas utilizando la librería nativa de Python Tkinter. Esta librería permite crear diferentes elementos, como marcos, botones, etiquetas, etc, que al unirse, componen una interfaz gráfica. Esto ha permitido la creación de una interfaz sencilla para que los usuarios puedan importar y exportar archivos, borrar los archivos de las bases de datos y ejecutar sentencias SQL dentro de la propia interfaz. Esto último permite al usuario de la aplicación consultar información almacenada en las bases de datos empleando sentencias SQL, teniendo además la posibilidad de exportar la vista generada a un archivo CSV.

Los objetivos conseguidos de los marcados inicialmente son los siguientes:

- Diseño e implementación de tres bases de datos funcionales para almacenar los datos de los diferentes archivos de configuración de la oferta y la demanda, así como los de resultados.
- Diseño y desarrollo de los módulos necesarios para leer y procesar los datos provenientes de los archivos para introducirlos posteriormente en la base de datos y, además, generar estos archivos usando la información proveniente de las bases de datos.
- Diseño e implementación del módulo encargado de la gestión de las bases de datos y de los datos almacenados en estas.
- Creación y desarrollo de una interfaz sencilla que permita interactuar con los diferentes módulos permitiendo así, importar, exportar y borrar archivos de las bases de datos y, también, realizar consultas a las bases de datos mediante sentencias SQL.

A lo largo de la realización de este TFG, se han abordado y cumplido también una serie de objetivos de carácter académico y técnico que han aportado un conjunto de competencias nuevas no estudiadas en la carrera. Estas son las siguientes:

- Diseñar y gestionar bases de datos relacionales mediante el uso de sentencias SQL.
- Procesar archivos Yaml utilizando PyYaml en Python.
- Procesar archivos CSV utilizando la librería csv de Python.
- Diseñar y crear interfaces gráficas utilizando la librería de Tkinter.
- Integrar bases de datos relacionales a programas desarrollados en Python utilizando la librería de sqlite3.
- Separar los programas, desarrollados en Python, en diferentes módulos que se comuniquen entre sí.
- Utilizar repositorios de código como GitHub para el control de versiones de un programa.

Los conocimientos adquiridos durante la realización del TFG no solo han contribuido al desarrollo de este proyecto, sino que también resultarán de gran utilidad en futuros proyectos profesionales y académicos.

Todo el trabajo desarrollado en este TFG ha permitido comprobar la utilidad del uso de bases de datos dentro del simulador ROBIN.

Finalmente, se plantean las siguientes líneas posibles de trabajos futuros:

- Optimizar el código de la aplicación para que su ejecución sea lo más rápida posible.
- Migrar las bases de datos a un servidor y modificar el código para que se empleen esas bases de datos.
- Mejorar el aspecto de la interfaz y añadir mejoras y funciones a esta.
- Implementar las bases de datos directamente al simulador para que al ejecutar una prueba, este almacene los archivos empleados en las simulaciones directamente a las bases de datos.

Apéndice A

Esquemas de diseño relacional

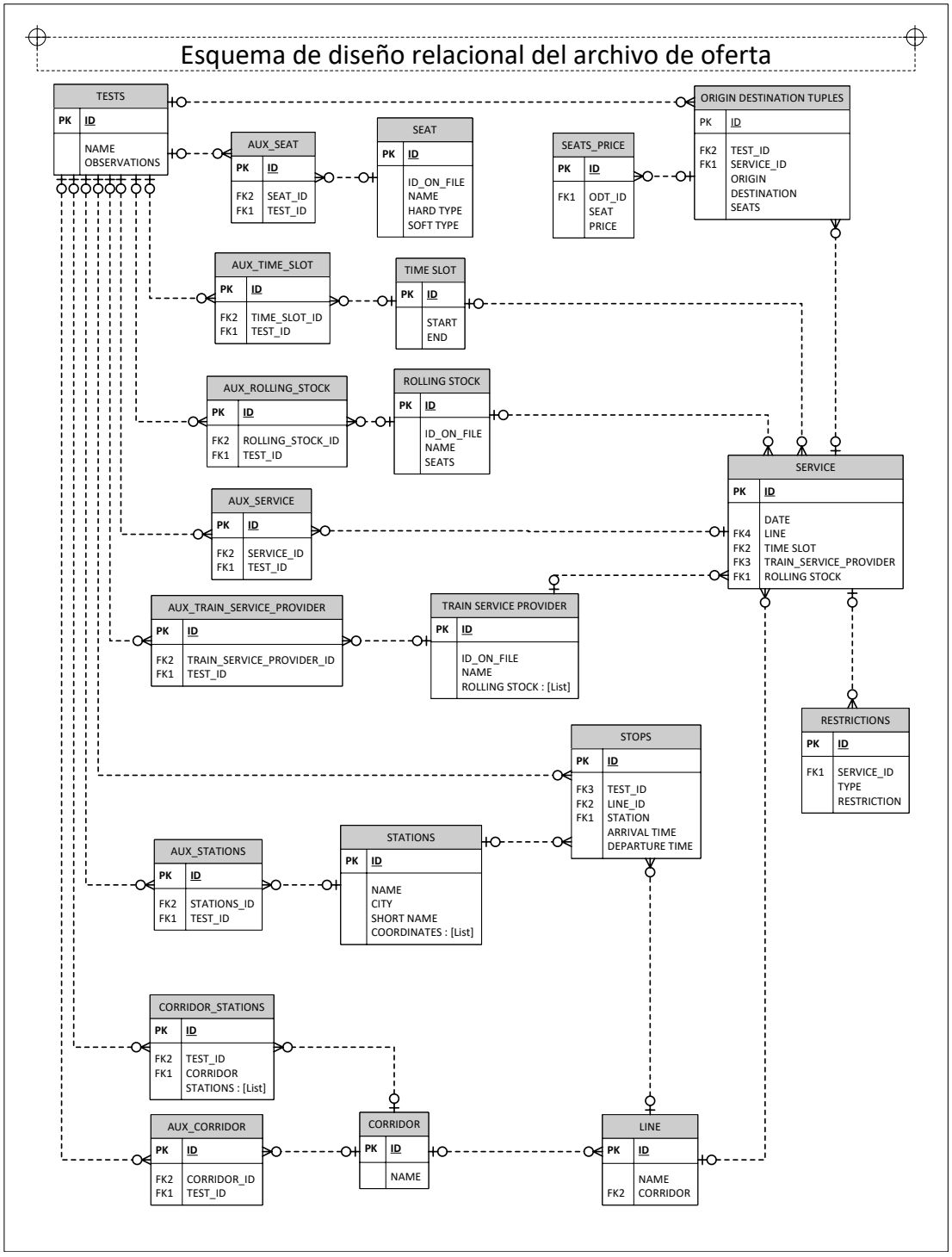


Figura A.1: Esquema de diseño relacional para la base de datos de los archivos de entrada de datos de la oferta

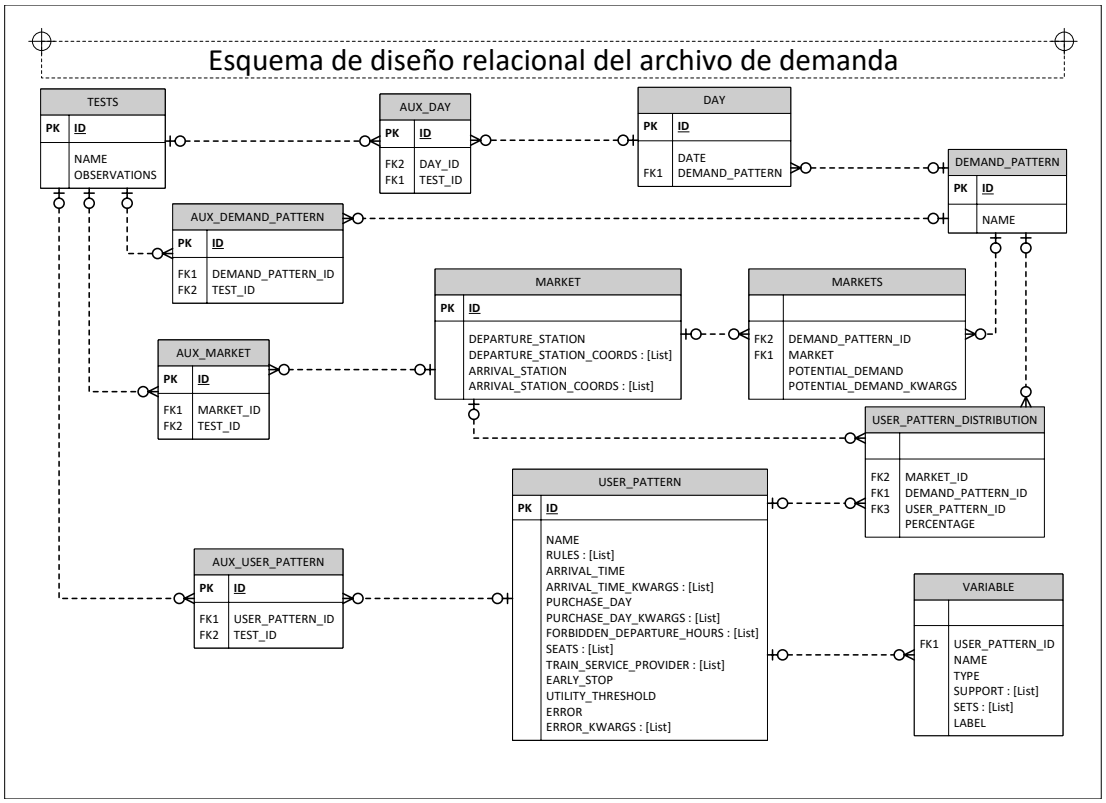


Figura A.2: Esquema de diseño relacional para la base de datos de los archivos de entrada de datos de la demanda

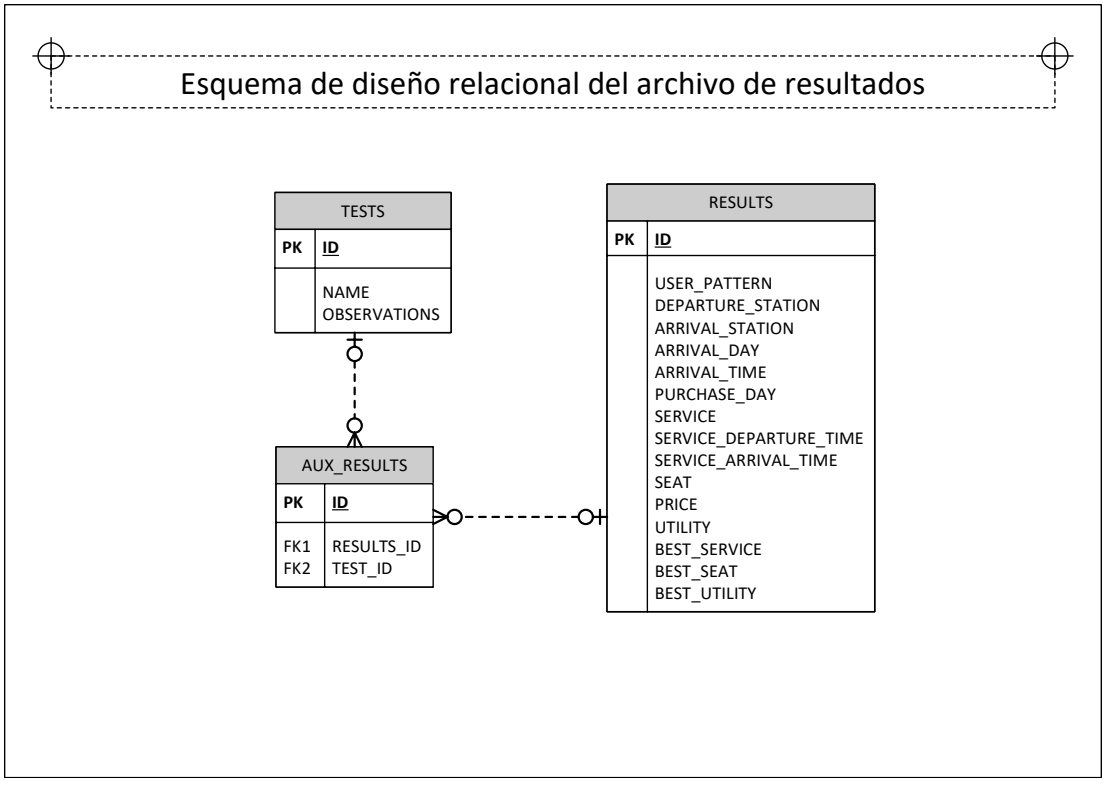


Figura A.3: Esquema de diseño relacional para la base de datos de los archivos de resultados

Apéndice B

Estructura de los archivos manejados por la aplicación

B.1. Estructura de los archivos Yaml de entrada de datos de la oferta

Listado B.1: Plantilla estructural de los archivos de entrada de datos de la oferta

```yaml
#Lista con los datos de las estaciones
stations:
- id: <Identificador de la estación>
  name: <Nombre de la estación>
  city: <Ciudad en la que se encuentra la estación>
  short_name: <Nombre corto de la estación>
  coordinates:
    latitude: <Latitud a la que se encuentra la estación>
    longitude: <Longitud a la que se encuentra la estación>

#Lista con los asientos ofertados
seat:
- id: <Identificador del asiento>
  name: <Nombre del identificador>
  hard_type: <Tipo de asiento físico>
  soft_type: <Tipo de asiento según los servicios incluidos>

#Lista con la información de los corredores ferroviarios
corridor:
- id: <Identificador del corredor>
  name: <Nombre del corredor>

  #Lista de diccionarios que define el corredor
  stations:
  - org: <Estación de origen>
    des: # Si hay más estaciones, esto es un diccionario
    - org: <Estación de origen>
      des: [] # Para indicar el final del corredor
              # se usa una lista vacia
```

```
#Lista con los datos de las líneas
line:
- id: <Identificador de la línea>
  name: <Nombre de la línea>
  corridor: <Corredor al que pertenece la línea>
  stops: # Lista con las paradas de la línea
    - station: <Identificador de la estación de llegada>
      arrival_time: <Tiempo relativo de llegada a la estación>
      departure_time: <Tiempo relativo de salida de la estación>

#Lista con los trenes en servicio
rollingStock:
- id: <Identificador del tren>
  name: <Nombre del tren>
  seats: # Lista con los tipos de asientos físicos que tiene el tren
  - hard_type: <Tipo de asiento físico>
    quantity: <Cantidad del tipo de asiento>

#Lista con los proveedores de servicios ferroviarios
trainServiceProvider:
- id: <Identificador del proveedor>
  name: <Nombre del proveedor>

  #Lista de los trenes que posee el proveedor de servicios ferroviarios
  rolling_stock:
  - <Identificador del tren 1>
  - <Identificador del tren n>

#Lista con todas las franjas horarias
timeSlot:
- id: <Identificador de la franja horaria>
  start: <Hora de inicio de la franja>
  end: <Hora de finalización de la franja>

#Lista con la información de los servicios
service:
- id: <Identificador del servicio>
  date: <Fecha en la que se da el servicio>
  line: <Linea a la que pertenece el servicio>
  train_service_provider: <Proveedor encargado del servicio>
  time_slot: <Franja horaria de inicio del servicio>
  rolling_stock: <Tren que va a cumplir el servicio>

  #Lista con la información de los trayectos entre estaciones del servicio
  origin_destination_tuples:
  - origin: <Estación de origen>
    destination: <Estación de destino>

    #Lista con el precio de cada asiento
    seats:
```

```
    - seat: <Identificador del asiento>
      price: <Precio del asiento para el trayecto>
  capacity_constraints: <Restricción de capacidad, null en caso de no existir>
```

B.2. Estructura de los archivos Yaml de entrada de datos de la demanda

Listado B.2: Plantilla estructural de los archivos de entrada de datos de la demanda

```
#Lista de mercados para la simulación
market:
  - id: <Identificador del mercado>
    departure_station: <Nombre de la estación de salida>
    departure_station_coords: <Coordenadas de la estación de salida>
    arrival_station: <Nombre de la estación de llegada>
    arrival_station_coords: <Coordenadas de la estación de llegada>
#Lista con los patrones de usuario
userPattern:
  - id: <Identificador del patrón de usuario>
    name: <Nombre del patron de usuario>
    #Lista de reglas difusas
    rules:
      R0: <Regla difusa número 0>
      Rn: <Regla difusa número n>
    #Lista de variables lingüísticas
    variables:
        #Variable tipo "fuzzy"
      - name: <Nombre de la variable>
        type: fuzzy
        support: <Dominio de la variable lingüística>
        sets: [Conjunto_1, Conjunto_2, Conjunto_n]
        Conjunto_1: <Valores que definen el Conjunto_1>
        Conjunto_n: <Valores que definen el Conjunto_n>
        #Variable tipo "categorical"
      - name: <Nombre de la variable>
        type: categorical
        labels: [Etiqueta_1,Etiqueta_2,Etiqueta_n]
    arrival_time: <Función para generar la distribución del tiempo de llegada>
    arrival_time_kwargs: # Lista con los argumentos de la función de arrival_time
      arg_1: <Valor del argumento 1>
      arg_n: <Valor del argumento n>
    purchase_day: <Función que genera los días de antelación de la compra del
        billete>
    purchase_day_kwargs: # Argumentos para la función purchase_day
      arg_1: <Valor del argumento 1>
      arg_n: <Valor del argumento n>
    forbidden_departure_hours: # Franja horaria en la que el usuario prefiere no
        empezar el viaje
      start: <Hora de inicio de la franja>
      end: <Hora de finalización de la franja>
    #Lista de diccionarios para representar la utilidad de cada asiento
```

```
    #para el patrón de usuario
    seats:
      - id: 1
        utility: <Valor de utilidad para el asiento con id = 1>
      - id: n
        utility: <Valor de utilidad para el asiento con id = n>
    #Lista de diccionarios para representar la utilidad de cada proveedor
    #de servicios ferroviarios para el patrón de usuario
    train_service_providers:
      - id: 1
        utility: <Valor de utilidad para el proveedor con id = 1>
      - id: 2
        utility: <Valor de utilidad para el proveedor con id = 2>
      - id: n
        utility: <Valor de utilidad para el proveedor con id = n>
    early_stop: <Probabilidad de que el usuario compre un billete útil sin
        realizar una búsqueda exhaustiva>
    utility_threshold: <Umbral de utilidad para el patron de usuario>
    error: <Función para generar la distribución del error>
    error_kwargs:
      arg_1: <Valor del argumento 1>
      arg_n: <Valor del argumento n>
#Lista de patrones de demanda
demandPattern:
  - id: <Identificador del patrón de demanda>
    name: <Nombre del patrón de demanda>
    markets: # Lista de mercados a los que afecta el patrón de demanda
      - market: <Identificador del mercado>
        potential_demand: <Función para calcular la posible demanda>
        potential_demand_kwargs: # Argumentos para la función de potential_demand
            arg_1: <Valor del argumento 1>
            arg_n: <Valor del argumento n>
        #Lista con la distribución de los patrones de usuario para el
        #patrón de demanda actual
        user_pattern_distribution:
          - id: <Identificador del patrón de usuario 1>
            percentage: <Porcentaje del tipo de usuario esperado>
day:
  - id: <Identificador del día>
    date: <Fecha de la simulación>
    demandPattern: <Patrón de demanda empleado para el día simulado>
```

Apéndice C

Introducción a la lógica difusa

La lógica difusa surge como una extensión de la lógica clásica para afrontar la realidad imprecisa de nuestro entorno. Mientras que la lógica tradicional solo admite valores binarios (verdadero o falso), en la lógica difusa cada afirmación puede matizarse con un grado de verdad entre 0 y 1. La capacidad de graduar la pertenencia a un concepto, como por ejemplo "barato" o "cercano", brinda a la lógica difusa la posibilidad de emular con más fidelidad el razonamiento humano.

A continuación, se expondrán, de manera general, los elementos esenciales de la lógica difusa.

C.1. Conjuntos difusos

Un conjunto difuso introduce una función de membresía que asigna a cada elemento "x" un valor real entre 0 y 1, a diferencia de un conjunto clásico, cuya función indica si un elemento pertenece o no al conjunto. En la práctica, las funciones de membresía más utilizadas son las trapezoidales y las triangulares, debido a que se definen con pocos parámetros y describen con claridad regiones de pertenencia total y parcial. Además, su formulación matemática facilita el cálculo y la visualización. En la Figura C.1 se puede ver un ejemplo de un conjunto trapezoidal y un conjunto triangular.

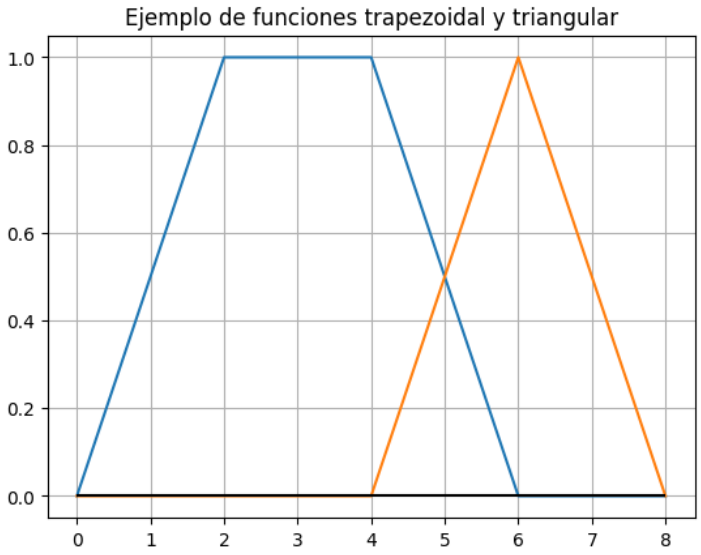


Figura C.1: Ejemplo de funciones trapezoidal, en azul, y triangular, en naranja.

C.2. Variables lingüísticas

Una variable lingüística es un modo de nombrar información numérica usando palabras en lugar de cifras. Estas vienen definidas por un nombre, como por ejemplo "temperatura", el rango de valores que puede tomar, como por ejemplo de 0 a 35 °C y un conjunto de etiquetas, como por ejemplo, "fría", "templada" y "caliente". Cada etiqueta lleva asociado un conjunto, con su correspondiente función de membresía (triangular, trapezoidal, etc.). De esta manera, decir que la temperatura es "templada", no sería una verdad absoluta, sino que sería un valor real comprendido entre 0 y 1, como por ejemplo 0,6. Esto reflejaría cuantitativamente la pertenencia del dato dentro de esa categoría, en este caso que la temperatura sea templada.

C.3. Proposiciones difusas

Una proposición difusa es una afirmación del tipo: "variable ES término", por ejemplo, si la "distancia ES corta" o la "velocidad ES alta". Estas proposiciones no son ni verdaderas ni falsas de forma inequívoca, como pasaba en la lógica clásica, sino que devuelven un grado de verdad igual al valor de la función de membresía correspondiente. Estas proposiciones pueden ser proposiciones atómicas, si estas no se pueden subdividir en otras proposiciones, o proposiciones compuestas o moleculares, si están conformadas por una o más proposiciones atómicas. Un ejemplo de una proposición compuesta sería "distancia ES corta Y velocidad ES alta", que estaría formada por las proposiciones atómicas "distancia ES corta" y "velocidad ES alta".

C.4. Reglas difusas

Las reglas difusas son los enunciados condicionales que se emplean en lógica difusa para ligar expresiones imprecisas con términos lingüísticos. Por lo general, tienen la siguiente estructura: `IF <antecedente> THEN <consecuente>`, donde el antecedente puede ser una proposición atómica o una proposición compuesta y el consecuente asigna valores de pertenencia o una proposición atómica, dependiendo del modelo empleado. Por ejemplo, en el modelo Mamdani, el antecedente sería una proposición atómica o compuesta y el consecuente sería una proposición atómica, como por ejemplo, `IF velocidad alta AND distancia lejana THEN freno bajo`.

C.5. Modelos difusos

Dentro de la lógica difusa se han desarrollado distintos modelos que definen cómo se representa el conocimiento, cómo se formulan las reglas y cómo se obtiene una salida a partir de las entradas. Cada modelo responde a necesidades distintas según el contexto de la aplicación: control de sistemas, predicción, clasificación, entre otros. A continuación se presentan algunos de los modelos más utilizados en lógica difusa.

C.5.1. Modelo de Mamdani

El modelo de Mamdani [18] es el más conocido. Cada regla está formada por un antecedente, que puede ser una proposición atómica o una proposición compuesta, y por un consecuente, que es siempre una proposición atómica, con lo que una regla difusa siguiendo el modelo de Mamdani tiene la siguiente forma: `IF <antecedente> THEN <consecuente>`.

Para obtener una salida, primero se evalúa el antecedente. Si la regla difusa tiene una proposición compuesta y esta tiene un operador `AND`, se toma el valor mínimo entre los valores de pertenencia de las proposiciones atómicas que componen la proposición compuesta, por otro lado, si el operador fuera un `OR`, se toma el valor máximo entre los valores de pertenencia de las proposiciones atómicas de la proposición compuesta. El resultado de esta evaluación es el grado de activación de la regla.

Después, el grado de activación de la regla se emplea como límite superior para la salida, por lo que si un valor supera este umbral, se recorta al nivel del grado de activación. Una vez hecho esto para todos los puntos del dominio, se combinan todas las salidas obtenidas realizando, por ejemplo, el máximo punto a punto de los valores recortados.

Por último, con el área obtenida de combinar las salidas, se calcula el centroide de este para obtener la salida de la regla difusa (defuzzificación).

C.5.2. Modelo Tsukamoto

El modelo de Tsukamoto [19] es parecido al de Mamdani, pero obliga a los conjuntos de salida a ser monótonos, es decir, que su función de pertenencia debe crecer o decrecer de forma continua sin cambiar de sentido. Esto permite que cada regla difusa dé como salida un valor directamente. Después, se realiza la media ponderada de todas las salidas de las reglas para obtener el valor de salida. De esta manera se evita el tener que realizar la defuzzificación de la salida.

C.6. Modelo TSK (Takagi-Sugeno-Kang)

El modelo TSK (Takagi-Sugeno-Kang) [20] es una evolución del modelo de Mamdani, pero con un planteamiento mucho más matemático y orientado al rendimiento. La premisa que sigue este modelo es que, en lugar de emplear como salida un conjunto difuso, cada regla devuelva la salida como un valor numérico directamente a partir de una fórmula. Por ejemplo:

$$\texttt{IF Velocidad Alta AND Carga Media THEN Salida} = 0.3 \times \texttt{Velocidad} + 0.6 \times \texttt{Carga} + 2$$

Donde el antecedente corresponde con la proposición compuesta `Velocidad Alta AND Carga Media` y la salida es el resultado del cálculo de la fórmula $0.3 \times \texttt{Velocidad} + 0.6 \times \texttt{Carga} + 2$, `suponiendo una velocidad de 100 y una carga de 50, la salida sería 62`

Para poder obtener una salida, primero se calcula el grado de activación de cada una de las reglas difusas que componen el sistema. Después se calcula el valor de cada una de las funciones de las reglas difusas. Por último, se calcula la salida como la media ponderada de todas las salidas de las funciones.

$$y = \frac{\sum_{i=1}^{n} w_i \cdot z_i}{\sum_{i=1}^{n} w_i} \tag{C.1}$$

Donde:

- w_i es el grado de activación de la regla i, calculado a partir del antecedente de la regla difusa.
- z_i es la salida de la regla i, obtenida evaluando la función del consecuente con los valores de entrada.
- y es la salida global del sistema, obtenida como media ponderada de las salidas individuales.

C.7. Modelos difusos acumulativos

El modelo difuso acumulativo [21] se basa en el modelo TSK, pero sustituye la fórmula que se emplea para calcular la salida de cada una de las reglas y en su lugar se utiliza un valor fijo. Además, cada regla del sistema contribuye de manera directa a la salida final, debido a que esta se calcula sumando los valores proporcionales de salida de cada una de las reglas difusas que tiene el sistema, en lugar de realizar la media ponderada de los valores de la salida de cada una de las reglas como en el modelo TSK clásico. Dicho esto, se puede reescribir la regla de ejemplo empleada en la sección del modelo TSK C.6 para que se adapte a este modelo cambiando la fórmula para calcular la salida por un número, por ejemplo, 60. De esta forma, la regla resultante es:

```
IF Velocidad Alta AND Carga Media THEN 60
```

Para calcular los valores de salida de la regla del ejemplo se hace lo siguiente:

1. Se calcula el grado de activación de la regla con la fórmula:

$$\alpha = \mu(A_1 \text{ en } x_1) \times \mu(B_1 \text{ en } y_1) \tag{C.2}$$

   Donde:

   - α es el valor del grado de activación de la regla difusa.
   - $\mu(A_1 \text{ en } x_1)$ es el valor de la pertenencia para la proposición atómica A (`Velocidad Alta`) en x_1.
   - $\mu(B_1 \text{ en } y_1)$ es el valor de la pertenencia para la proposición atómica B (`Capacidad media`) en y_1.
   - x_1 es el valor conocido de la entrada para la velocidad en el punto 1.
   - y_1 es el valor conocido de la entrada para la carga en el punto 1.

2. Empleando el valor de α y el valor del consecuente, en este caso 60, se calcula la contribución de la regla multiplicando el valor de α por 60.

$$\mu_{R_1} = \alpha \times 60 \tag{C.3}$$

3. Si hubiera más reglas difusas, se seguiría el mismo procedimiento descrito en 1 y 2 para calcular la contribución de cada una de las reglas.

4. Después de calcular la contribución de cada una de las reglas, estas se suman para obtener el valor de la decisión.

$$DV = \sum_{i=1}^{n} (\mu_{R_i}) \tag{C.4}$$

   Para el caso del ejemplo, como solo tenemos una regla, el valor sería directamente el valor de contribución de la regla, por lo que $DV = \alpha \times 60$.

Este es el modelo empleado por ROBIN en el momento de redacción de este TFG.

Bibliografía

[1] E. del Castillo-Herrera et al., «ROBIN: Rail mOBIlity simulatioN,» en *Proceedings of the 25th Euro Working Group on Transportation Meeting (EWGT 2023)*, ép. Transportation Research Procedia, vol. 78, Santander, España: Elsevier, sep. de 2023, págs. 158-165. DOI: 10.1016/j.trpro.2024.02.021.

[2] P. S. Foundation. «Python 3.13.» Último acceso: Mayo de 2025. (oct. de 2024), dirección: https://www.python.org/.

[3] F. Romano, *Learning Python*. Birmingham, UK: Packt Publishing Ltd., dic. de 2015, ISBN: 978-1-78355-171-2.

[4] R. van Hattem, *Mastering Python*. Birmingham, UK: Packt Publishing Ltd., abr. de 2016, ISBN: 978-1-78528-972-9.

[5] «PyYAML.» Último acceso: Mayo de 2025. (mayo de 2006), dirección: https://pyyaml.org/.

[6] Independently published, *YAML: Quick start including data processing and generation with Python and PHP*. Independently published, 2019, ISBN: 9781703404609.

[7] D. Wittmann, *Mastering YAML: From Basics to Application*. Independently Published, 2023, pág. 78, ISBN: 9798871512531.

[8] SQLite Consortium. «SQLite3.» Último acceso: Octubre de 2024. (ago. de 2000), dirección: https://www.sqlite.org/.

[9] P. S. Foundation. «sqlite3 — DB-API 2.0 interface for SQLite databases (Python 3.13).» Último acceso: Mayo de 2025. (oct. de 2024), dirección: https://docs.python.org/es/3.13/library/sqlite3.html.

[10] R. H. Sianipar, *Learn SQLite with Python: Building Database-Driven Desktop Projects*. Independently published, 2019, ISBN: 9781696314794.

[11] P. S. Foundation. «csv — CSV File Reading and Writing (Python 3.13).» Último acceso: Mayo de 2025. (oct. de 2024), dirección: https://docs.python.org/es/3.13/library/csv.html.

[12] A. Martelli et al., *Modern Python Cookbook*. Packt Publishing, 2019, ISBN: 9781786469250.

[13] P. S. Foundation. «Tkinter — Python interface to Tcl/Tk (Python 3.13).» Último acceso: Mayo de 2025. (oct. de 2024), dirección: https://docs.python.org/es/3.13/library/tkinter.html.

[14] B. A. Meier, *Python GUI Programming Cookbook, Second Edition: Use recipes to develop responsive and powerful GUIs using Tkinter*, 2.ª ed. Birmingham, UK: Packt Publishing, mayo de 2017, ISBN: 9781787129450.

[15] A. D. Moore, *Python GUI Programming with Tkinter: Design and build functional and userfriendly GUI applications*. Packt Publishing, 2021, ISBN: 9781801818865.

[16] «SQLiteStudio.» Último acceso: Noviembre de 2024. (mayo de 2007), dirección: https://sqlitestudio.pl/.

[17] JetBrains. «PyCharm 2024.3.» (dic. de 2024), dirección: https://www.jetbrains.com/pycharm/.

[18] E. Mamdani y S. Assilian, «An experiment in linguistic synthesis with a fuzzy logic controller,» *International Journal of Man-Machine Studies*, vol. 7, n.º 1, págs. 1-13, 1975, ISSN: 0020-7373. DOI: https://doi.org/10.1016/S0020-7373(75)80002-2. dirección: https://www.sciencedirect.com/science/article/pii/S0020737375800022.

[19] J.-S. R. Jang, C.-T. Sun y E. Mizutani, *Neuro-Fuzzy and Soft Computing: A Computational Approach to Learning and Machine Intelligence*. Upper Saddle River, New Jersey: Prentice Hall, 1997, Véase explicación del modelo de Tsukamoto en la p. 84, ISBN: 9780132610667.

[20] T. Takagi y M. Sugeno, «Fuzzy identification of systems and its applications to modeling and control,» *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-15, n.º 1, págs. 116-132, 1985. DOI: 10.1109/TSMC.1985.6313399.

[21] D. Muñoz-Valero, J. Moreno-Garcia, J. A. López-Gómez, E. A. Villarrubia-Martin y L. Jimenez-Linares, «A knowledge-driven fuzzy logic framework for supporting decision-making entities,» *Applied Soft Computing*, vol. 181, pág. 113 415, 2025, ISSN: 1568-4946. DOI: https://doi.org/10.1016/j.asoc.2025.113415. dirección: https://www.sciencedirect.com/science/article/pii/S1568494625007264.